

M5cet 5.3

Šifrovaná komunikace peer-to-peer v prohlížeči: zprávy, soubory, hovory a sdílení polohy mezi lidmi, kteří znají společné heslo místnosti. Server zprostředkuje spojení, podrží zprávy pro nepřítomné a nikdy nevidí obsah.

Tato dokumentace popisuje verzi **5.3.0**: příkazy jako rozhovor — vstupní body modelu (start, odpověď na jeho zprávu, tlačítka, formuláře, chyby a libovolný počet webhooků s vlastními URL), historie volání v `m5.model`, výsledek jako seznam výstupů (text, zvuk, video, notifikace, tlačítka, formuláře, kód pro prohlížeč v izolovaném rámu), form builder v konzoli a víc funkcí v jednom toku vizuálního tvůrce; nástroje konzole (5.2) (Functions, AI a řeč, telefonie a SIP, Layout a Menu builder) jako moduly s výchozím přístupem, přístupovými skupinami, hlavní skupinou a granty na části modulu, rychlé kontroly přístupu s logem povolení i odmítnutí, webhooky s plným logem, replayem a asynchronními režimy, aktivační znaky psaní zprávy (`/`, `@`, `#`) a vestavěné příkazy `/help`, `/whois`, `/dns`, `/web`, `/mail` a `/domain`; konzole Functions (5.1) s profesionálním editorem kódu (CodeMirror, našeptávač SDK, šablony, nápověda) a vizuálním tvůrcem funkcí z uzlů a drátů, živé běhy s odpovídáním na otázky funkce, řeč zdarma a offline přímo na serveru (Whisper a hlasy Piper); M5cet Functions (5.0) — programovatelné moduly v JavaScriptu a Pythonu spouštěné z chatu přes /klíčové-slovo, webhookem, plánem nebo API; AI a řeč (4.14) — poskytovatelé Claude, OpenAI, Open WebUI, Perplexity, Ollama, llama.cpp, GPT4All, Hugging Face a ElevenLabs se zašifrovanými klíči, modely, limity, zkušebnou a žurnálem všech volání, asistent v aplikaci s odpověďmi psanými průběžně; Layout builder pro celou aplikaci — okno Místnost, okna, dialogy a panely jako rozvržení, varianty pro skupiny uživatelů a šablony vzhledu, historie uložených verzí s rozdíly a návratem, sloučení vlastního rozvržení s novým výchozím po aktualizaci aplikace, vložení HTML jako prvků a kontrola přístupnosti; opravené konektory AI a řeči (Claude 5, HuggingFace Inference Providers) a jejich zapínání v konzoli; Layout builder jako GUI designer (lišta, okno chatu, zprávy, psaní a widget příjemců jako stromy prvků s paletou, našeptáváním a náhledem samotné aplikace); Server-enhanced jen s přihlášením passkey, jedinečné uživatelské jméno účtu jako primární klíč, ověřené přihlášení krok za krokem (passkey, globální klíč, databáze, trezor) a adresy /signin a /signup, ochranu před opuštěním připojené místnosti, kontrolu verzí aplikace a knihoven proti serveru s opravou jedním tlačítkem, moduly zapínané pro skupiny uživatelů a Menu builder v konzoli se šablonovacím jazykem; dále okno Místnost se záložkami Light · P2P / Server-enhanced a výběrem uložených připojení, sdílení uloženého připojení pozvánkou, uložená připojení přihlášených uživatelů (klíč, jméno, server, statistiky a log), šablony vzhledu iOS 27, Windows 11 a pět studiových, jejich konfiguraci v administraci — a pod tím vším signalizační protokol v2, šifrování v3 (Argon2id, slepá ID místností, klíče odesílatelů s forward secrecy, párové klíče, E2EE hovorů), binární přenos a relay souborů, cluster přes Redis, účty s více passkeys a obnovou, role v administraci, neměnný audit, zálohy, metriky a alerty.

Node.js 24

TypeScript 7

React 19

WebRTC DataChannel

AES-GCM 256

ECDSA / ECDH P-256

Argon2id

Ed25519

WebAuthn + PRF

SQLite + SQLCipher

Pro koho je co

Uživatel — režimy, co šifrování chrání.

Vývojář — architektura, protokol v2, API.

Provozovatel — nasazení, administrace, audit, bezpečnost.

Co je nového v 5.3

Příkazy jako rozhovor: **vstupní body** — execute, **response** (odpověď na zprávu modelu), **button**, **form**, **error** a víc **webhooků** s vlastními URL, každý se svými vstupy. `m5.model` zná celé sezení. Funkce vrací **seznam výstupů** — zvuk, video, notifikace, tlačítka a formuláře (form builder) i kód pro prohlížeč v izolovaném rámu; každý se vykreslí samostatně.

Přehled v seznamu změn 5.3.

Co je nového v 5.2

Nástroje jako moduly: Functions, AI & speech, Telephony & SIP, Layout a Menu builder mají výchozí přístup, přístupové skupiny (allow/deny), hlavní skupinu `mod-``<modul>` a granty jen na části modulu (`model:dns*`, `-provider:openai`, `number:+420*`); log přístupů. **Webhooky** s plným logem, proměnnými, replayem i na konceptu a režimy sync/async/auto. V chatu opět funguje `/`, přibyly `@` a `#` a vestavěné příkazy `/help`, `/whois`, `/dns`, `/web`, `/mail`, `/domain`. Přehled v seznamu změn 5.2.

Co je nového v 5.1

Konzole Functions přepracovaná: **editor kódu** s barevným zvýrazněním, našeptávačem SDK `m5` se signaturami a poli k vyplnění, šablonami, nápovědou a kontrolou; **vizuální tvůrce** — funkce z uzlů a drátů bez psaní kódu, hodnoty uzlů přímo na plátně, balíček i model jedním klikem; živé běhy s odpovídáním na `prompt / form`. **Řeč zdarma a offline:** Whisper a hlasy Piper (i česky) přímo na serveru, plus předvolby Groq, Speeches, Kokoro a whisper.cpp. Přehled v seznamu změn 5.1.

Co je nového v 5.0

M5cet Functions — programovatelné moduly: operátor napíše model v **JavaScriptu** (QuickJS) nebo **Pythonu** (Pydide) v odděleném procesu s limity, publikuje ho a v chatu ho kdokoli spustí přes `/klíčové-slovo` (jako roboti v messengerech), případně přes webhook, plán nebo API. SDK `m5` (výstupy, session, cache, HTTP se SSRF ochranou, DNS, crypto vč. JWT/PGP/SSH/X.509, kódy, AI a agenti), IDE a tutoriál v konzoli, živé `prompt / form` v chatu. Přehled v seznamu změn 5.0.

Co je nového ve 4.14

AI a řeč od základu: poskytovatelé (Claude, OpenAI, Open WebUI, Perplexity, Ollama, llama.cpp, GPT4All, Hugging Face, ElevenLabs) s klíči zašifrovanými na serveru, modely načtené od poskytovatele, limity (výchozí 0 = AI vypnutá), zkušebna, žurnál všech volání a asistent v aplikaci se streamovanými odpověďmi. První etapa frameworku funkcí. Přehled v seznamu změn 4.14.

Co je nového ve 4.13

Layout builder navrhuje celou aplikaci: vedle hlavní obrazovky i okno Místnost, okna, dialogy a panely (44 rozvržení v sekcích). Varianty pro skupiny a šablony vzhledu, historie s rozdíly a návratem, sloučení s novým výchozím rozvržením po aktualizaci, vložení HTML a kontrola přístupnosti. Přehled v seznamu změn 4.13.

Oprava 4.0.6

AI a řeč: Anthropic bez parametru `temperature`, který modely Claude 5 odmítají; HuggingFace přes `router.huggingface.co` (dřívější `api-inference.huggingface.co` zanikl); srozumitelné chyby spojení; moduly AI a Speech se zapínají v konzoli (změny 4.0.6).

Co je nového ve 4.0.5

Layout builder je GUI designer: lišta nahoře, okno chatu, příchozí / odchozí / systémové zprávy, pole pro psaní a widget příjemců jsou stromy prvků z palety (panely, oblasti, texty, tlačítka, pole, ikony, obrázky, HTML, živé části aplikace, šablony). Každé pole našeptává (třídy, atributy, CSS, proměnné, akce), náhled je aplikace sama. Přehled v seznamu změn 4.0.5.

Co je nového ve 4.0

Přihlášení a identita: Server-enhanced vyžaduje passkey, účet má jedinečné uživatelské jméno (jméno v místnosti je jen přezdívkou), přihlášení ověří passkey, globální klíč i databázi a všechno zapíše do auditu; /signin a /signup. Ochrana navigace a kontrola verzí. Moduly a skupiny a Menu builder v konzoli. Přehled v seznamu změn 4.0.

Co je nového ve 3.3

Okno Místnost: typ připojení jako záložky v záhlaví, na *Server-enhanced* výběr z uložených připojení a ozubené kolo k jejich správě, tlačítka a sdílení vždy dole, během spojení nic nepřepnete. Sdílení uloženého připojení pozvánkou. Přehled v seznamu změn 3.3.

Co je nového ve 3.2

Uložená připojení s výchozím připojením, automatickým připojením po přihlášení, jiným obslužným serverem, statistikami a logem; 13 šablon vzhledu včetně iOS 27 a Windows 11, světlý / tmavý tón podle systému a pět stylů ikon; vše řízené z konzole (Client & addons); rychlejší načítání. Přehled v seznamu změn 3.2.

Co je nového ve 3.1

Šifrování v3 s forward secrecy a slepým ID místnosti, binární přenos, relay souborů a E2EE hovorů, více instancí, více passkeys a obnovovací kód, role, neměnný audit, metriky a alerty. Přehled v seznamu změn 3.1.

Vyhledávání: stiskněte `/` a pište — výsledky se hledají v nadpisech i textu, `↑` `↓` a `Enter` skočí na místo. Tlačítko **PDF** stáhne tutéž dokumentaci k tisku.

Obsah

1. Úvod
2. Režimy a funkce
3. Architektura
4. Signalizační protokol v2

5. Šifrování v2
6. Šifrování v3
7. Soubory, relay a hovory
8. Fronta zpráv pro nepřítomné
9. Účty a passkeys
10. Přihlášení a identita
11. Úložiště
12. Ochrana navigace a kontrola verzí
13. Okno Místnost
14. Uložená připojení
15. Šablony vzhledu
16. Administrace
17. Menu builder
18. Layout builder — GUI designer
19. AI a řeč
20. Funkce — programovatelné moduly
21. Monitoring a audit
22. API reference
23. Více instancí
24. Nasazení
25. Bezpečnostní opatření serveru
26. Návrhy a roadmapa
27. Změny v 5.3
28. Změny v 5.2
29. Změny v 5.1
30. Změny v 5.0
31. Změny ve 4.14
32. Změny ve 4.13
33. Změny ve 4.0.6
34. Změny ve 4.0.5
35. Změny ve 4.0.0
36. Změny ve 3.3.0
37. Změny ve 3.2.0
38. Změny ve 3.1.0
39. Změny ve 3.0.0

Režimy a funkce

Aplikace běží ve dvou režimech. Oba šifrují stejně; liší se tím, co smí dělat server.

REŽIM	CO DĚLÁ SERVER	CO TO PŘINÁŠÍ
light	Jen signalizace (spojení WebRTC). Nic neukládá.	Nejmenší stopa. Zpráva pro někoho, kdo není online, čeká v lokální frontě prohlížeče.
server-enhanced	Navíc: úložiště, relay pro nepřítomné, Web Push, synchronizace nastavení, proxy souborů, když P2P nejde. Od 4.0 jen s přihlášením passkey (Přihlášení a identita).	Zprávy dorazí i nepřítomnému (přihlášenému passkey), historie přežije zavření okna, soubory projdou i přes striktní NAT.

Uchování chatu

VOLBA	KDE JE HISTORIE	KDO JI PŘEČTE
ephemeral	Nikde. Nové spojení začíná prázdné.	—
session	V sessionStorage prohlížeče, šifrovaná neexportovatelným klíčem z IndexedDB.	Jen tato záložka prohlížeče.
server s passkey	V trezoru účtu a v jeho SQLCipher databázi; zapečetěno v prohlížeči klíčem z PRF passkey.	Jen držitel passkey (zero-knowledge).
server bez passkey	Do 3.3 v dočasné databázi relace (TTL 1 den). Od 4.0 ho aplikace bez přihlášení nenabízí a uchování přepne na session.	—

Funkce v kostce

Zprávy

Místnost i soukromí příjemci, odpovědi, přeposílání, stav (odesláno → uloženo → doručeno → přečteno), zprávy *klikací*, *mizějící* a *zapečetěné* vlastním kódem.

Soubory

Malé přímo ve zprávě, velké po kusech po DataChannelu; když P2P nejde, přes šifrovanou proxy serveru. Opakování ztracených kusů, ověření otisku celého souboru.

Hovory a média

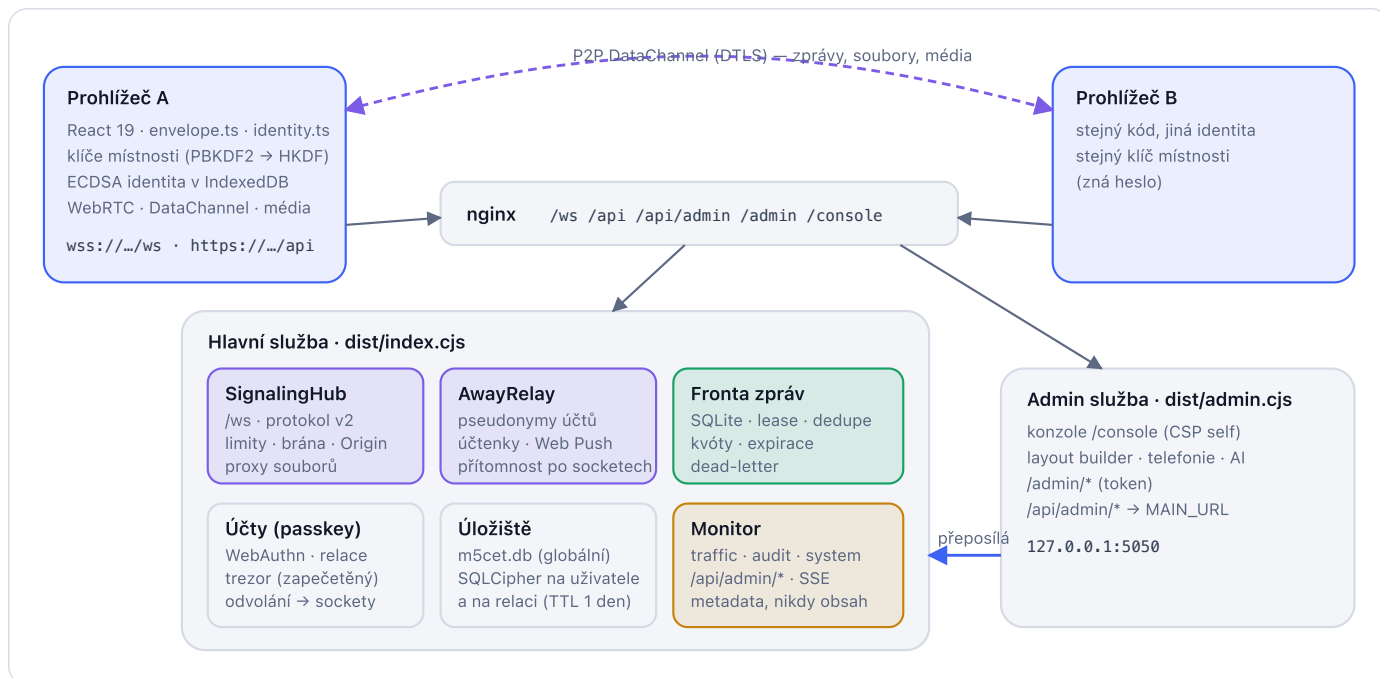
Audio a video přes WebRTC; DTLS otisky chráněné zapečetěnou signalizací a každý rámec navíc šifrovaný klíčem páru (E2EE hovorů). Sdílení polohy, NFC, přepis řeči, AI asistent a telefonie jako volitelné moduly.

Účty

Passkey (WebAuthn) s rozšířením PRF: z něj se v prohlížeči odvodí klíč trezoru i klíč vlastní databáze. Server nezná žádný z nich. Více passkeys a obnovovací kód pro případ ztráty zařízení.

Architektura

Tři běžící části: **prohlížeče** (veškerá kryptografie), **hlavní služba** (Express + WebSocket, všechny živý stav) a **admin služba** (konzole a nástroje). Před nimi obvykle nginx.



Obr. 1 — Části systému a kudy tečou data. Fialové: signalizace a relay; zelené: fronta; oranžové: monitoring.

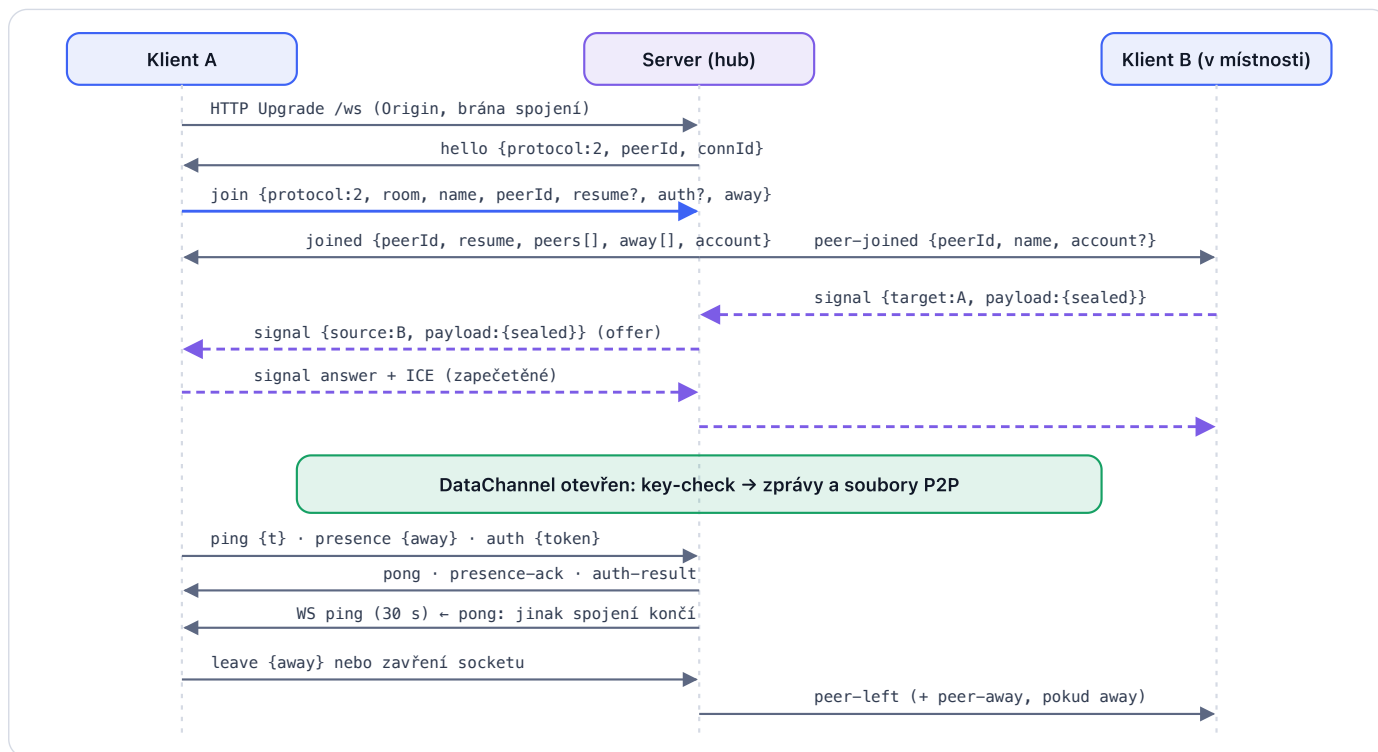
Kde je co v kódu

OBLAST	SOUBORY
Signalizace, protokol v2	<code>server/signaling/hub.ts</code> , <code>frames.ts</code> , <code>limits.ts</code> , <code>refs.ts</code>
Relay pro nepřítomné	<code>server/signaling/relay.ts</code>
Fronta zpráv	<code>server/accounts/mailqueue.ts</code> (SQLite), <code>memqueue.ts</code> (paměť)
Účty, WebAuthn	<code>server/accounts/store.ts</code> , <code>routes.ts</code> , <code>webauthn.ts</code> , <code>cbor.ts</code>
Úložiště	<code>server/storage/*</code> — <code>service.ts</code> , <code>global-store.ts</code> , <code>user-store.ts</code> , <code>keys.ts</code>
Monitoring a audit	<code>server/monitor/traffic.ts</code> , <code>audit.ts</code> , <code>system.ts</code> ; API <code>server/admin-api.ts</code>
Admin služba a konzole	<code>server/admin.ts</code> , <code>admin-ui/public/</code> (<code>console.js</code> , <code>builder-kit.js</code> , <code>menu-builder.js</code> , <code>layout-builder.js</code> , <code>modules.js</code> , <code>legacy-tools.js</code>)
AI a řeč	<code>server/ai/</code> (<code>providers/*</code> — Anthropic, OpenAI-kompatibilní, Ollama, ElevenLabs; <code>config.ts</code> — poskytovatelé, zašifrované klíče, modely, limity; <code>service.ts</code> — výběr modelu, skupiny, limity; <code>journal.ts</code> — žurnál volání v SQLite; <code>routes.ts</code> , <code>admin-routes.ts</code>), <code>admin-ui/public/ai-console.js</code> , <code>client/src/lib/ai.ts</code> , <code>client/src/components/AiPanel.tsx</code> , <code>client/src/lib/markdown.ts</code> + <code>components/Markdown.tsx</code>
Rozvržení (Layout builder)	<code>client/src/lib/layout-tree.ts</code> (model, paleta, validace), <code>client/src/lib/layouts/*</code> (výchozí stromy, kontrakty, ukázky), <code>client/src/components/LayoutView.tsx</code> (vykreslení, kompilované), <code>client/src/components/LayoutProvider.tsx</code> (rozvržení pro přihlášeného diváka), <code>client/src/layout-preview.tsx</code> a <code>layout-preview-parts.tsx</code> (náhled), <code>client/src/lib/layout-merge.ts</code> (3-way), <code>layout-diff.ts</code> , <code>html-to-tree.ts</code> , <code>layout-a11y.ts</code> , <code>server/layout.ts</code> , <code>server/layout-history.ts</code> , <code>server/layout-archive.json</code> (vydané výchozí stromy), <code>server/layout-catalog.ts</code>
Kryptografie v prohlížeči	<code>client/src/lib/envelope.ts</code> , <code>identity.ts</code> , <code>crypto.ts</code> (v1), <code>file-transfer.ts</code> , <code>message-kinds.ts</code>
Validace přichozích dat	<code>client/src/lib/validate.ts</code>
Aplikace	<code>client/src/App.tsx</code> a komponenty v <code>client/src/components/</code>

Signalizační protokol v2

Jeden WebSocket na `/ws`, rámce jako JSON objekty s polem `type`. Verze 2 přinesla přesnou validaci, limity, pseudonymy, rámec `auth` a zapečetěnou signalizaci.

Životní cyklus spojení



Obr. 2 — Připojení, výměna zapečetěné signalizace a udržování spojení.

Co se stane s každým rámcem

- Velikost** — nad 256 KB (`MAX_FRAME_BYTES`) knihovna ws spojení zavře kódem `1009`.
- Parsování** (`parseFrame`) do přesného tvaru: typy, délky, formát ID (`[A-Za-z0-9_.-]{1,96}`), base64, počty. Neznámý typ → `error {code:"unknown-type"}`, špatný tvar → `invalid-frame`.
- Limit třídy** — token bucket na socket. Při překročení `rate-limited {frame, retryAfterMs}`; 20 odmítnutí za minutu → zavření `1008`.
- Akce a záznam** do monitoru provozu (typ, bajty, spojení, peer, účet, hash místnosti, stav).

Rámce klient → server

TYPE	POLE	VÝZNAM	TŘÍDA LIMITU
<code>join</code>	<code>protocol</code> , <code>room</code> , <code>name</code> , <code>peerId?</code> , <code>resume?</code> , <code>auth?</code> , <code>away</code>	Vstup do místnosti; <code>auth</code> = session token účtu	signaling
<code>auth</code>	<code>token</code> <code>null</code> , <code>away</code>	Přihlásit / odhlásit účet bez opuštění místnosti	signaling
<code>leave</code>	<code>away</code>	Odchod; <code>away:true</code> = zůstat jako nepřítomný (relay)	signaling
<code>signal</code>	<code>target</code> , <code>payload</code>	SDP/ICE pro jednoho peera; ve v2 <code>{sealed:{v,iv,ciphertext}}</code>	signaling
<code>ping</code>	<code>t</code>	Aplikační heartbeat a měření RTT	heartbeat
<code>presence</code>	<code>away</code>	Stránka odložena / zpět (lifecycle)	presence
<code>relay</code>	<code>messageId</code> , <code>to[]</code> , <code>envelope</code> , <code>expiresAt?</code>	Šifrová obálka pro nepřítomné členy (odkazy z <code>away[]</code>)	relay
<code>relay-ack</code>	<code>ids[]</code> (≤ 500)	Potvrzení zpracování doručených položek	relay
<code>receipt</code>	<code>messageIds[]</code> (≤ 200), <code>state</code>	Přečteno / doručeno — jen pro zprávy, které server tomuto čtenáři doručil	receipt
<code>storage</code>	<code>id</code> , <code>op</code> , <code>payload</code> , <code>auth?</code> , <code>session?</code>	Operace úložiště po WebSocketu (viz úložiště)	storage
<code>command-poll</code> / <code>command-ack</code>	<code>deviceId</code> / <code>commandId</code> , <code>result?</code>	Příkazy operátora pro zařízení	command
<code>proxy-meta</code> <code>chunk</code> <code>end</code> <code>cancel</code> <code>need</code>	<code>transferId</code> , ...	Proxy souborů přes server (když P2P nejde)	proxy (+ bajty)

Rámce server → klient

TYPE	KDY
<code>hello</code>	Hned po otevření: <code>protocol</code> , navržené <code>peerId</code> , <code>connId</code> , <code>limits</code>
<code>joined</code>	Odpověď na <code>join</code> : přidělené <code>peerId</code> , <code>resume</code> , <code>peers[]</code> , <code>away[]</code> , <code>account</code>
<code>peer-joined</code> / <code>peer-left</code> / <code>peer-updated</code>	Změny v místnosti (<code>peer-updated</code> = přihlášení/odhlášení bez odchodu)
<code>peer-away</code> / <code>peer-back</code> / <code>peer-gone</code>	Člen s účtem je nepřítomen / zpět / odhlášen (relay pro něj končí)
<code>signal</code> , <code>signal-undeliverable</code>	Signál od peera / cíl v místnosti není
<code>relay-status</code>	<code>stored</code> <code>forwarded</code> <code>duplicate</code> <code>delivered</code> <code>read</code> <code>rejected</code> pro odeslanou zprávu
<code>relay-deliver</code>	Položky z fronty (po 50) — čekají na <code>relay-ack</code>
<code>presence-ack</code> , <code>auth-result</code> , <code>account-revoked</code>	Stav přítomnosti a účtu na tomto socketu
<code>rate-limited</code> , <code>error</code>	Odmítnutý rámec (<code>code</code> : <code>invalid-frame</code> , <code>unknown-type</code> , <code>not-in-room</code> , <code>proxy-refused</code> , <code>server-error</code>)
<code>replaced</code> , <code>closed-by-server</code>	Stejný klient se připojil znovu (resume) / operátor spojení ukončil
<code>proxy-*</code> , <code>proxy-ack</code>	Přeposlané rámce proxy (s <code>from</code>) a přijetí přenosu
<code>admin-command</code> , <code>storage-result</code> , <code>pong</code>	Příkaz operátora, výsledek operace úložiště, odpověď na ping

Datagramy — rozložení klíčových rámců

☐ hlavička / směrování (vidí server) ☐ bezpečnostní pole ☐ šifrový text (server nepřečte)

join

type	protocol	room	name	
"join"	2	1–64 znaků	≤ 48, bez řídicích znaků	
peerId?		resume?	auth?	away
požadované ID		tajemství z minulého joined	session token (≤ 200)	bool

joined

type	protocol	peerId	resume
"joined"	2	přidělené — převzít!	32 B base64url, jednorázově
peers[]		away[]	account
{peerId, name, joinedAt, account?}		{account, name, since}	{account, away} {invalid} null

signal (zapečetěný)

type	target / source	sealed.v	sealed.iv	sealed.ciphertext
"signal"	peer ID	2	12 B	AES-GCM(klíč „signal“, SDP ICE) · AAD „m5cet/2 signal místnost od pro“

relay

type	messageId		to[]	expiresAt?
"relay"	ID zprávy		≤ 50 odkazů účtů (pseudonymy)	TTL zprávy
envelope.v	envelope.id	envelope.iv	envelope.ciphertext	
2	= messageId	12 B	AES-GCM(klíč „message“, {b, pk?, s?}) · AAD „m5cet/2 msg místnost id“ · ≤ 190 KB	

relay-deliver — položka

id	seq	kind	messageId	from	
mq-... (pro ack)	pořadí v místnosti	message status		{peerId, name, account?}	
envelope			status?	storedAt	attempts
přesně to, co odesílatel poslal			{state, at, recipientName}		

proxy-chunk

type	transferId	seq	v	iv	ciphertext	from
"proxy-chunk"		0 ... n-1	2		AES-GCM(klíč souboru, kus) · AAD „m5cet/2 chunk přenos seq počet“	přidá server

Limity

TŘÍDA	KAPACITA (BURST)	DOPLŇOVÁNÍ / S	RÁMCE
signaling	120	10	join, leave, signal, auth
relay	60	2	relay, relay-ack
receipt	30	1	receipt
presence	10	0,2	presence
storage	120	10	storage
proxy	400	60	proxy-* + bajty: 8 MB burst, 2 MB/s
heartbeat	6	0,5	ping
command	20	0,5	command-poll, command-ack
other	30	1	neplatné rámce

Brána spojení (při upgradu): 30 nových spojení za minutu a 20 současně na adresu, 5 000 celkem. Adresa se bere z `X-Forwarded-For` jen od důvěryhodné proxy (`TRUST_PROXY`). **Zpětný tlak**: příjemci s více než 8 MB ve frontě socketu server přestane posílat kusy souborů (klient si je vyžádá znovu), nad 32 MB je spojení ukončeno.

Peer ID, resume a pseudonymy

- Server v `hello` navrhne ID; klient smí v `join` požádat o vlastní. Je-li v místnosti obsazené jiným živým socketem, dostane nové — **převzetí cizího ID už není možné** (dřív šlo odklonit cizí signalizaci).
- `joined.resume` je jednorázové tajemství. Při novém připojení do stejné místnosti ho klient pošle spolu se svým ID; server porovná SHA-256 v konstantním čase, starý socket dostane `replaced` a ID přejde na nové spojení.
- Členové s účtem se místnosti ukazují jako `account = base64url(HMAC-SHA256(tajemství, místnost || 0 || ID účtu))[0..22]`. Stejná osoba má v každé místnosti jiný odkaz. Pole `accountId` nese totéž (alias pro klienty 2.x).

Příklad: minimální klient

```
const ws = new WebSocket("wss://chat.example.com/ws");
let me = null;
ws.onmessage = (e) => {
  const f = JSON.parse(e.data);
  if (f.type === "hello") ws.send(JSON.stringify({ type: "join", protocol: 2, room: "brno", name: "Alice", peerId: f.peerId, away: false }));
  if (f.type === "joined") { me = f.peerId; /* uložit f.resume pro reconnect */ }
  if (f.type === "rate-limited") setTimeout(retry, f.retryAfterMs);
  if (f.type === "error") console.warn(f.code, f.message);
};
setInterval(() => ws.send(JSON.stringify({ type: "ping", t: Date.now() })), 25_000);
```

Kompatibilita: klient 3.0 přijímá jen zapečetěné signály. Klient 2.x se po nasazení sám nabídne k obnovení; dokud se neobnoví, P2P spojení s ním nevznikne a jeho zprávy v1 klient 3.0 sice přečte, ale 2.x nepřečte zprávy v2.

Šifrování v2

Veškerá kryptografie běží v prohlížeči přes WebCrypto. Server dostává jen šifrované texty a metadata potřebná ke směrování.

Hierarchie klíčů místnosti



Obr. 3 — Z jednoho hesla čtyři nezávislé klíče; žádný z nich neopustí prohlížeč.

Svázání s kontextem (AAD)

Každé šifrování AES-GCM nese *associated data* — řetězec, který se nešifruje, ale je chráněn tagem. Šifrovaný text proto jde otevřít jen tam, kam patří.

ÚČEL	AAD	ČEMU BRÁNÍ
zpráva	<code>m5cet/2 msg <místnost> <id></code>	přenesení do jiné místnosti, záměně ID, přehrání pod jiným ID
signál	<code>m5cet/2 signal <místnost> <od> <pro></code>	přesměrování nebo podvržení SDP serverem (MITM hovoru)
meta souboru	<code>m5cet/2 file-meta <přenos></code>	záměně metadat mezi přenosy
kus souboru	<code>m5cet/2 chunk <přenos> <seq> <počet></code>	přeskládání kusů, vložení kusu z jiného souboru
konec souboru	<code>m5cet/2 file-end <přenos></code>	podvržení otisku
řádek historie na serveru	<code>m5cet:server-row:v1:<id></code>	záměně řádků v úložišti relace

Obálka zprávy

Envelope v2				DataChannel i relay
v	id	iv	ciphertext	
2	ID zprávy (v AAD)	12 B náhodně	AES-GCM-256(JSON { b: tělo, pk?: veřejný klíč, s?: podpis }) + 16 B tag	
tělo b (uvnitř šifrování)				jen příjemci
id	text / attachment	senderId, senderName	createdAt, ttl, flags	to, replyTo, forwardedFrom
= envelope.id	obsah, příloha data:			

Podpis `s` = ECDSA-P256-SHA256 přes `AAD || b` klíčem identity odesílatele; `pk` je jeho veřejný klíč (SPKI). Obojí je **uvnitř** šifrování — server nevidí, kdo co podepsal, a nemůže tak propojit identitu zařízení napříč místnostmi.

```
// client/src/lib/envelope.ts
const keys = await deriveRoomKeys("brno", "correct horse battery");
const me = await loadIdentity(); // ECDSA P-256, IndexedDB
const env = await sealMessage(keys, "msg-1", { id: "msg-1", text: "ahoj", /* ... */ }, me);
// → { v: 2, id: "msg-1", iv: "...", ciphertext: "..." }
const { payload, version, signer } = await openMessage(keys, env);
// signer = { publicKey, valid: true } · version = 2
```

Identita zařízení, TOFU a bezpečnostní čísla

- Každý profil prohlížeče má pár ECDSA P-256; soukromý klíč je **neexportovatelný** (umí podepsat, nejde přečíst ani kódem aplikace) a leží v IndexedDB `m5cet-identity`.
- **Trust on first use**: první ověřený klíč pro jméno v místnosti se připne (`m5cet:pins:v1`). Zpráva pod stejným jménem s jiným klíčem se označí **jiný klíč** a zobrazí se varování; neplatný podpis **neplatný**; starší klient **nepodepsáno**.
- **Bezpečnostní číslo** dvou lidí = 12 skupin po 5 číslicích ze SHA-512 (1 025 kol) seřazených veřejných klíčů — obě strany vidí totéž a mohou ho porovnat mimo aplikaci.
- Otisk vlastního zařízení i ověření odesílatele ukazuje *Informace o zprávě*.

Kontrola klíče a replay

Po otevření DataChannelu si peeři pošlou `{kind:"key-check", v:2, check}`. Nesouhlasí-li, aplikace řekne „jiné heslo místnosti“ místo nečitelných zpráv. Hodnota je odvozená z tajemství místnosti a k útoku na heslo nedává víc než libovolný šifrový text. ID přijatých zpráv si klient pamatuje (20 000 posledních) a opakovaně zahodí.

Soubory

1. Odesílatel vytvoří `transferId`, odvodí **klíč souboru** (HKDF, salt = ID přenosu) a pošle zapečetěná, podepsaná metadata.
2. Každý kus (32 KiB) šifruje s AAD `chunk|přenos|seq|počet` a zároveň počítá jeho SHA-256.
3. Konec přenosu nese zapečetěné a podepsané `{ root, totalChunks, size }`, kde `root` = SHA-256 zřetěžených otisků kusů.
4. Příjemce ověří: metadata před alokací (velikost, počet kusů ≤ 2 000 000, konzistence), každý kus proti AAD, na konci součet bajtů, `root` a shodu podpisu s metadaty. Ztracené kusy si vyžádá (`file-need` / `proxy-need`, max. 3 kola).
5. Jméno souboru se očistí (bez cest a řídicích znaků, BiDi) a **MIME typ se omezí na bezpečné** (obrázky, audio, video, PDF, text). HTML nebo SVG přijde jako `application/octet-stream` — nikdy se nespustí jako stránka v originu aplikace.

Zapečetěné zprávy

Text zprávy se navíc šifruje kódem předaným mimo aplikaci. Ve 3.0: kód **12 znaků** z 31znakové abecedy bez záměnných znaků (≈ 59 bitů, zobrazení `XXXX-XXXX-XXXX`), losování bez modulo biasu, PBKDF2 **600 000** iterací, při zadání se ignoruje velikost písmen, mezery a pomlčky. Zprávy z 2.x (6 znaků, 150 000 iterací) jdou dál otevřít. Text ani kód zapečetěné zprávy se nikdy neukládají mimo prohlížeč.

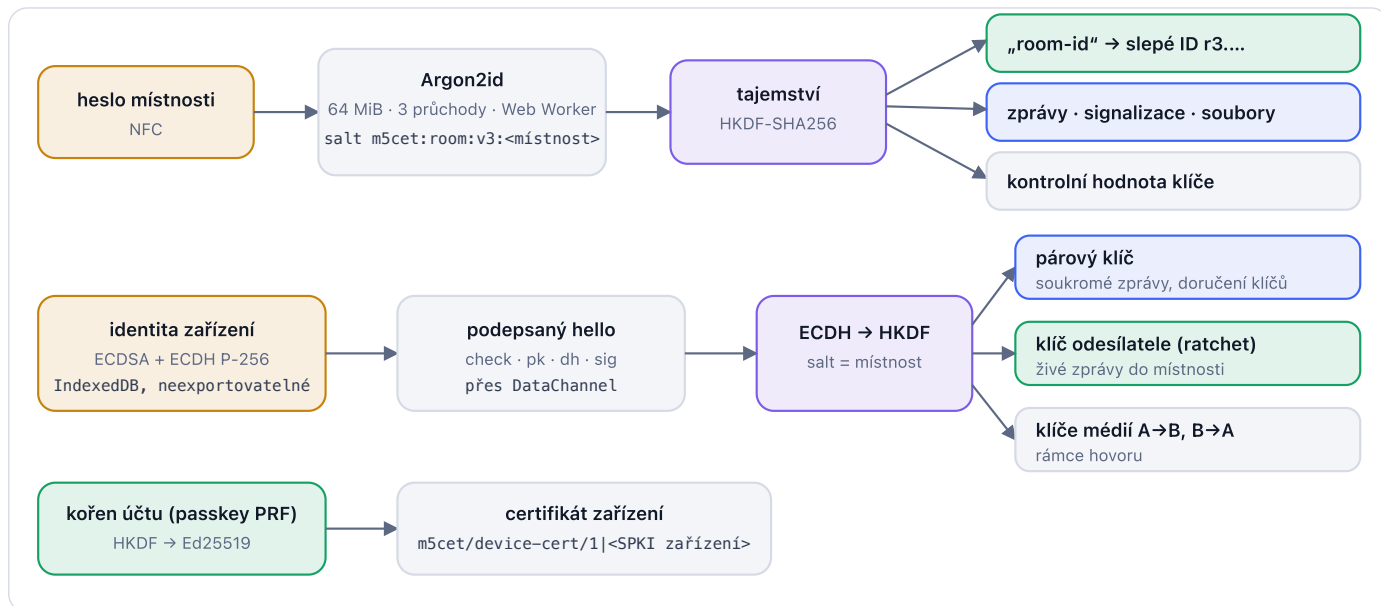
Co kdo vidí

ÚDAJ	SERVER	ČLEN MÍSTNOSTI	POZOROVATEL SÍTĚ
Obsah zpráv, souborů, hovorů	ne	ano (má klíč)	ne
Kdo zprávu napsal (podpis)	ne	ano, ověřeně	ne
SDP / ICE, DTLS otisky	ne (zapečetěno)	ano	ne
Název místnosti	ano (směrování) — od 3.1 ne (slepé ID)	ano	ne (TLS)
Kdo je v místnosti, kdy, IP	ano (monitor ukládá IP zkrácenou)	jména	IP spojení
Kdo komu píše přes relay	ano — do auditu jen se zapnutým auditem komunikace	—	ne
Heslo místnosti, klíče, passkey	nikdy	heslo ano	ne

Ve 3.0 otevřené, ve 3.1 vyřešené: klíč místnosti byl sdílený a statický — kdo heslo zná (nebo ho později získá), přečetl by i starší zachycený provoz. Verze 3.1 šifruje živé zprávy klíči odesílatelů s ratchetem a umí vyloučit člena bez změny hesla — viz Šifrování v3.

Šifrování v3 3.1

Verze 3.1 mění první krok (Argon2id), skrývá před serverem název místnosti a živé zprávy už nešifruje sdíleným klíčem místnosti, ale klíči odesílatelů s ratchetem a párovými klíči. Vše z kapitoly Šifrování v2 (AAD, podpisy, ověřené soubory) platí dál.



Obr. 3b — Klíče místnosti (nahore), klíče párů a odesílatelů (uprostřed), identita účtu (dole).

Argon2id ve Web Workeru

PBKDF2 ztěžuje hádání hesla jen časem; grafická karta ho spočítá paralelně. Argon2id (64 MiB paměti, 3 průchody, `hash-wasm`) stojí útočníka paměť na každý pokus. Výpočet běží v `kdf.worker.ts`, stránka nezamrzne; kde worker nejde spustit, poběží v hlavním vlákně. WebAssembly potřebuje v CSP `'wasm-unsafe-eval'` (nic jiného se nepovoluje). Obálky v2 (PBKDF2, např. zprávy čekající ve frontě z 3.0) se otevrou klíči odvozenými při první potřebě.

Slepé ID místnosti

Klient se připojuje pod `r3.<base64url(HKDF(tajemství, "room-id"), 192 b)>`. Server tak směřuje podle hodnoty, ze které název nezjistí; lidé se stejným názvem a jiným heslem se ani nepotkají. Historie na serveru se čte pod slepým ID, se zpětnou cestou pro data uložená 3.0 pod názvem.

Hello a párové klíče

hello DataChannel, po otevření						
kind	v	check	pk	dh	sig	caps
"hello"	3	kontrolní hodnota	ECDSA klíč zařízení (SPKI)	ECDH P-256 klíč	podpis m5cet/hello/1 místnost od pro check dh	bin, media bin, media

Podpis váže DH klíč na klíč zařízení, který si protistrany pinují. Jiná kontrolní hodnota = jiné heslo (aplikace to řekne), neplatný podpis = varování. Párový klíč = `HKDF(ECDH, salt = místnost, info = "m5cet/pair/1" + oba klíče zařízení seřazené)` — znají ho právě tato dvě zařízení. Soukromá zpráva vybraným příjemcům se zapečetí pro každého zvlášť (`sk: "pair"`); jiný člen místnosti ji neotevře ani s heslem.

Klíče odesílatele a forward secrecy

Každý odesílatel drží řetězový klíč `CK` a po každé zprávě ho posune:

```

MKn  = HMAC(CKn, 0x01)    // klíč zprávy n (AES-GCM), po použití se zahodí
CKn+1 = HMAC(CKn, 0x02)    // další článek; starý se přepíše
obálka = { v: 3, id, sk: <ID řetězu>, n, iv, ciphertext }
```

- Aktuální `CK` dostane každý peer zapečetěný **párovým klíčem** (`{kind:"sender-key", iv, ct}`) — server ho nevidí, jiný člen ho nepřečte.
- Kdo později získá heslo, **starší zachycené zprávy nepřečte**: klíče už přečtených zpráv nejdou z ničeho v paměti znovu spočítat.
- Řetěz se vymění po **500 zprávách, hodině**, nebo když někdo **odejde**; noví dostanou jen ti, kdo zůstali. Mimo pořadí se přijme až 1 000 přeskočených zpráv.

- **Vyloučit z mých zpráv** (Informace o účastníkovi): spojení skončí, klíč zařízení se na relaci zablokuje a řetěz se vymění.
- Klíčem místnosti se dál šifrují zprávy pro **nepřítomné** (relay), fronta odchozích zpráv, stav mikrofonu, signalizace a soubory — tam příjemce párový kanál mít nemusí.

Identita účtu a bezpečnostní čísla

- Z kořene účtu (PRF passkey) se odvodí podpisový klíč **Ed25519** (`HKDF info "m5cet:account-sign:v1"`). Podepíše klíč zařízení (`m5cet/device-cert/1|<SPKI>`); podepsané zprávy nesou klíč účtu i certifikát.
- Pin se od 3.1 váže na **účet**, ne na jméno: nové zařízení téhož účtu nevyvolá varování, cizí klíč pod stejným jménem ano. Odznak zprávy: **ověřený účet** , **ověřeno bezpečnostním číslem** .
- **Bezpečnostní číslo** (60 číslic) se dá porovnat i jako **QR kód** (`M5CET-SN:1:<číslice>`) — zobrazit, naskenovat kamerou druhého (`BarcodeDetector`), potvrdit „ověřeno“.
- Každá zpráva v detailu říká, čím byla zapečetěná: klíčem odesílatele, párovým klíčem, nebo klíčem místnosti.

Co kdo vidí ve 3.1

ÚDAJ	3.0	3.1
Název místnosti pro server	ano	ne — slepé ID
Starší provoz po úniku hesla	čitelný	nečitelný (živé zprávy; relay a fronta dál pod klíčem místnosti)
Soukromá zpráva pro ostatní členy	čitelná s heslem	nečitelná — párový klíč
Rámce hovoru pro prvek, který ukončí SRTP (SFU, relay)	čitelné	nečitelné — E2EE rámců
Kdo komu psal (fronta v DB)	čitelně v globální DB	zapečetěné master klíčem, odesílatel jen jako hash
Text probuzení na zamčené obrazovce	jméno, místnost	neutrální

Soubory, relay a hovory 3.1

Binární rámce

Kus souboru už necestuje jako JSON s base64 (o třetinu víc dat a práce na obou stranách), ale jako jedna binární zpráva — po DataChannelu i po signalizačním socketu:

binary chunk client/src/lib/binary-frames.ts · server/signaling/binary.ts							
0x4D „M“	type 0x01 P2P · 0x11 relay	version	L délka ID	transferId L bajtů	seq u32 BE	IV 12 B	ciphertext + tag AES-GCM, AAD chunk přenos seq počet

Schopnost si strany sdělí: peer v `hello` (`caps: ["bin"]`), klient v `join` (`features: ["bin"]`), server v úvodním `hello` (`features` a `limits.proxy`). Starší klient dostane tentýž kus jako JSON — server ho převede, ale neotevře.

Relay souborů, když P2P nejde

Za striktním NAT bez TURN se DataChannel neotevře. Aplikace pak pošle soubor šifrovaně přes server (`proxy` transport) — dřív přenos rovnou odmítla. Server kusy jen předává (žádná kopie v paměti), hlídá vlastnictví přenosu a strop 64 souběžných přenosů. Odesílatel se řídí rozpočtem, který server ohlásí (2 MiB/s, 60 rámců/s na socket, s rezervou): posílá na **80 %**, polovinu burstu, a čeká, dokud vlastní buffer socketu neklesne pod 1 MiB. Relay tak kus nikdy neodmítne ani spojení neodpojí.

E2EE hovorů

Hovor běží DTLS-SRTP mezi prohlížeči a signalizace, která ho sestavuje, je zapečetěná. Navíc se **každý zakódovaný rámec** zvuku i obrazu zapečetí v workeru (`RTCRtpScriptTransform`) klíčem páru pro daný směr — `HKDF(ECDH, info "m5cet/media/1|odesílatel|příjemce")`. Rámec projde i prvem, který SRTP ukončí (SFU, záznamový relay), a zůstane nečitelný.

rámec hovoru client/src/lib/media-frames.ts				
čitelný prefix Opus 1 B · VP8 10 / 3 B (klíčový / delta)	AES-GCM ciphertext + tag AAD = prefix	IV sůl 4 B + čítač 8 B	n délka prefixu	0x6D 0xE3 značka

- Transformace odesílání i příjmu se nastaví **na celý transceiver najednou** (při přidání stopy nebo příchodu cizí) — Chrome pozdější transformaci druhého směru tiše ignoruje.
- Bez klíče rámec projde beze změny (hovor chrání DTLS-SRTP); rámec, který nejde otevřít, se zahodí. Panel *Hlas* ukazuje stav za poslední sekundu: **E2EE** jen když byl každý rámec oběma směry zapečetěný a otevřený.
- Do hovoru se může přidat kterákoli strana: vyjednávání je „perfect negotiation“ (kolizi nabídek vyhraje iniciátor spojení).

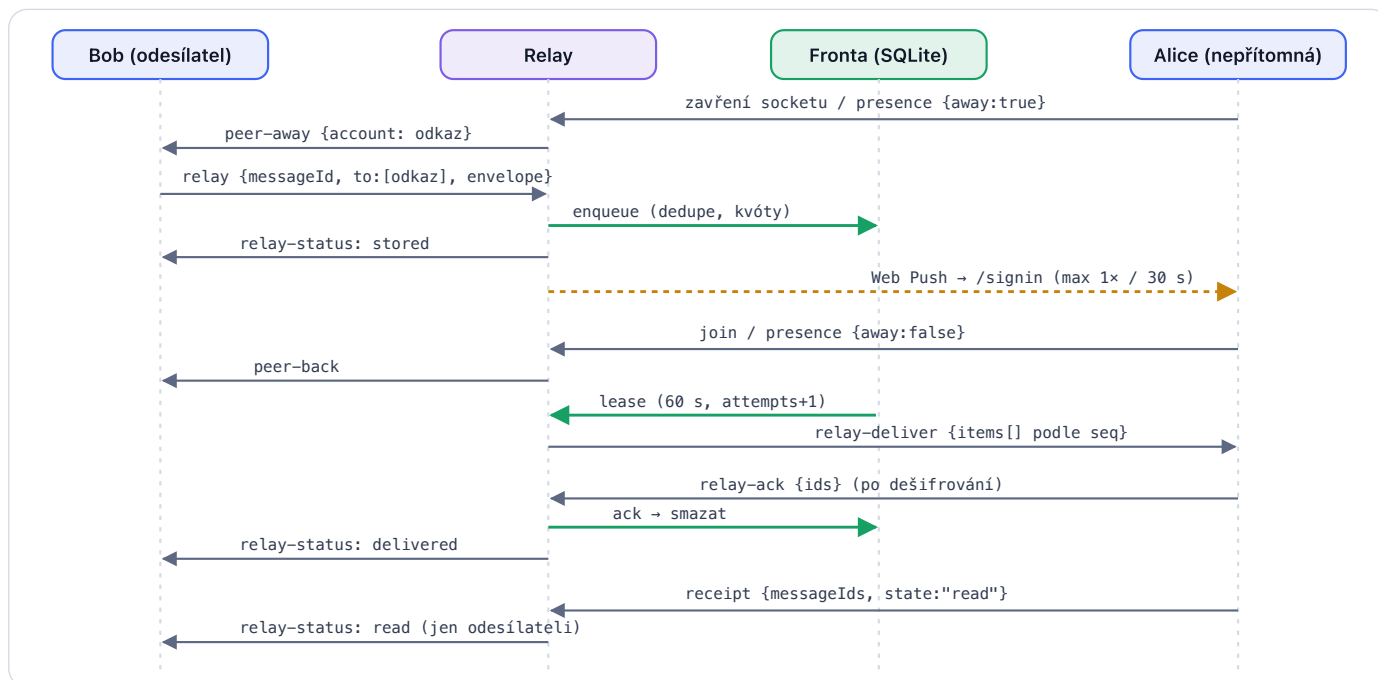
TURN s krátkodobými přístupy

S `TURN_SECRET` (coturn `use-auth-secret`) vydá `/api/turn` jméno `<expirace>:<náhoda>` a heslo `base64(HMAC-SHA1(secret, jméno))`, platné `TURN_TTL_SECONDS` (výchozí hodina). Klient si je obnoví 5 minut před koncem. Statické `TURN_USERNAME` / `TURN_CREDENTIAL` dál fungují, ale server při startu varuje — jsou čitelné pro každého návštěvníka.

Fronta zpráv pro nepřítomné

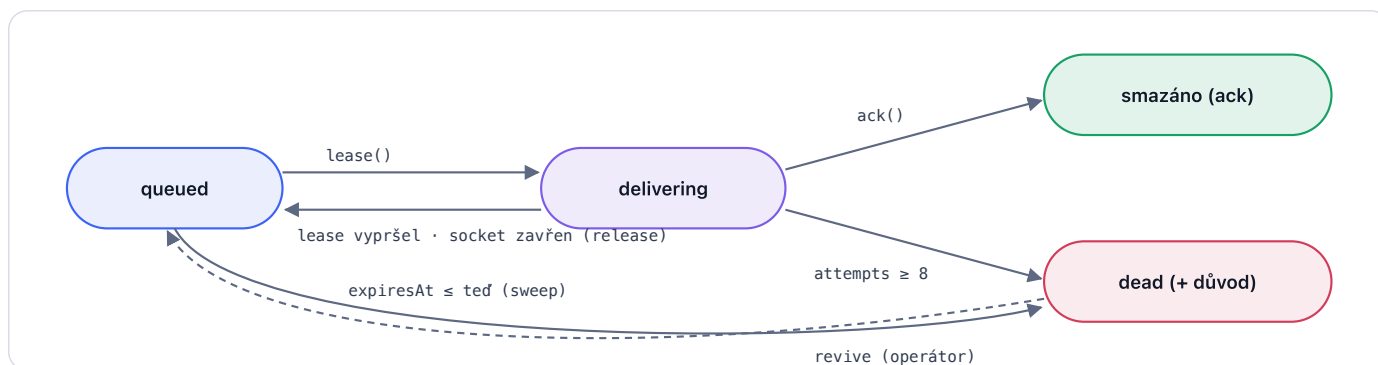
Když člen s účtem (passkey) a zapnutým uchováním na serveru odejde nebo odloží stránku, zůstane v místnosti jako *nepřítomný*. Ostatní mu dál píší; server šifrovou obálku uloží, odpoví „uloženo“, probudí ho Web Push a po návratu mu vše předá — v pořadí a s potvrzením.

Tok relay



Obr. 4 — Od odeslání nepřítomné po „přečteno“. Je-li odesílatel mezitím pryč, stav čeká v jeho vlastní frontě jako položka `status`.

Stavy položky



Obr. 5 — Doručení položku nemaže; smaže ji až potvrzení příjemce.

VLASTNOST	HODNOTA	POZNÁMKA
Pořadí	<code>seq</code> na (účet, místnost)	doručení v pořadí odeslání
Deduplikace	(účet, druh, messageId)	opakovaný relay = <code>duplicate</code>
Položek na účet	1 000	<code>maxItemsPerAccount</code>
Bajtů na účet	8 MB	<code>maxBytesPerAccount</code>
Velikost položky	130 000 B	<code>maxItemBytes</code>
Od jednoho odesílatele	300 položek	jeden člen nezaplní cizí schránku
Lease	60 s	po zavření socketu uvolněn hned
Pokusů	8	pak <code>dead</code>
Život	30 dní (<code>RELAY_RETENTION_DAYS</code>) nebo TTL zprávy	kratší z obou
Dead-letter	7 dní	pak se maže

Poctivé účtenky

Server si ke každé relayované zprávě pamatuje (v paměti, až 50 000 zpráv / 30 dní) odesílatele a příjemce. `receipt` přijme jen od účtu, kterému zprávu doručil, a pošle ho jen jejímu odesílateli. Ve 2.x mohl kdokoli v místnosti označit cizí zprávu jako přečtenou a vložit položky do cizí schránky.

Lokální fronta (light režim)

Bez serveru čeká zpráva, pro kterou není otevřený kanál, v paměti prohlížeče — už zašifrovaná. Zkouší se při otevření kanálu, po návratu do okna a jednou za minutu (200 zpráv, 60 pokusů, 24 h, respektuje TTL). Protože peer ID je od 3.0 stabilní i po výpadku (resume), obálky ve frontě zůstávají platné.

Účty a passkeys

Účet je passkey (WebAuthn) — žádné heslo. Server uloží jen veřejné klíče a počítaadla podpisů; bez rozšíření **PRF** účet nevznikne, protože z něj prohlížeč odvozuje **kořen účtu** a z něj klíče:

- `HKDF(kořen, "m5cet:profile:v1")` → klíč trezoru (profil + historie chatu, zapečetěné před odesláním),
- `HKDF(kořen, "m5cet:userdb:v1")` → klíč vlastní SQLCipher databáze; server ho drží jen v paměti, dokud je uživatel přihlášený,
- `HKDF(kořen, "m5cet:account-sign:v1")` → podpisový klíč účtu Ed25519 (identita účtu).

Více passkeys a obnovovací kód 3.1

Kořenem je PRF prvního passkey. Pro každý další passkey (max. 10) i pro obnovovací kód uloží server kořen **zapečetěný** (AES-GCM, AAD `m5cet:account-root:v1`, klíč `HKDF(PRF nebo tajemství kódu, "m5cet:root-wrap:passkey:v1" | "...recovery:v1")`) — přečíst ho umí jen ten, kdo passkey nebo kód drží. Přidání passkey i vytvoření kódu nejdřív potvrdí stávající passkey.

MECHANISMUS	CHOVÁNÍ
Obnovovací kód	26 znaků Crockford base32 (130 bitů), zobrazí se jednou; z kódu se odvodí ID, důkaz a tajemství (HMAC). Server zná jen ID a <code>SHA-256(důkaz)</code> . Obnova: <code>recovery/start</code> (10 pokusů za hodinu z adresy) vydá lístek na 10 minut a zapečetěný kořen, prohlížeč ho otevře, zaregistruje nový passkey a <code>recovery/finish</code> ho přidá k účtu.
Relace	náhodný token (32 B), na serveru jen SHA-256 v <code>sessions.json</code> — přežijí restart ; klouzavá platnost 12 h od posledního použití, nejdéle 7 dní, max. 20 na účet
Zařízení	okno účtu ukazuje relace (klient, adresa /24, poslední použití) a každou jde ukončit; aktuální je označená
Výzvy	náhodné, jednorázové, 2 minuty, vázané na obřad (registrace, přihlášení, přidání passkey, obnova)
Odvolání	odhlášení (jedno zařízení), „všude“, smazání účtu, zásah operátora → <code>account-revoked</code> na všech dotčených socketech (i na jiných instancích), relay pro účet končí, databáze se pro toto zařízení zamkne
RP ID	<code>WEBAUTHN_RP_ID</code> , jinak host z <code>PUBLIC_BASE_URL</code> , jinak hlavička Host (v produkci server varuje)
Push	jen endpointy známých služeb (FCM, Mozilla, Windows, Apple; rozšíření <code>PUSH_ENDPOINT_HOSTS</code>); text probuzení je neutrální

Přihlášení a identita 4.0

Vše, co drží server — uložená připojení, historie chatu na serveru, zprávy pro nepřítomné, push — patří k **účtu s passkey**. Bez přihlášení je aplikace plně použitelná v režimu *Light · P2P*; volby režimu Server-enhanced jsou neaktivní a odkazují do okna **Spojení**, jediného místa, kde se passkey vytváří a kde se přihlašuje.

Uživatelské jméno a přezdívka

	UŽIVATELSKÉ JMÉNO	PŘEZDÍVKA
Co to je	jedinečné jméno účtu, např. <code>bystry-sokol-7k3q</code> (dvě slova a 4–6 znaků z abecedy bez záměn <code>i l o 0 1</code>); vygeneruje ho server při registraci a nemění se	jméno zadané pro místnost — jen alias uživatelského jména, co uvidí ostatní
Kde je uložené	v passkey: <code>user.name</code> i <code>displayName</code> a jako <code>user.id</code> (user handle = UTF-8 jméno), takže ho ukáže správce hesel; na serveru je to ID účtu — primární klíč relací, trezoru, databáze uživatele (<code>db-<id></code>), fronty zpráv i auditu	v nastavení zařízení a zašifrovaně v trezoru účtu
Co peer vidí	v detailu uživatele (poslané v hello vedle podepsaných klíčů)	u zpráv a v seznamu lidí

- Light · P2P bez účtu:** relace dostane uživatelské jméno z přezdívky — přezdívka jako slug a 4 náhodné znaky (*Tomáš K.* → `tomas-k-7k3q`), platí jen pro tuto relaci.
- Účty z doby před 4.0** si ponechávají své dosavadní ID a to je zároveň jejich uživatelské jméno. Nepřejmenovávají se: ID je zapsané v jejich passkeys (user handle), v neměnném auditu, ve frontě i v indexu databází, a přejmenování by tyto vazby rozbilo.

Globální šifrovací klíč účtu

Klíč, kterým se šifrují data a databáze uživatele, je **kořen účtu** z rozšíření WebAuthn PRF: tentýž passkey s touž solí vrátí vždy tentýž výstup, takže klíč je pokaždé stejný a nikdy se neukládá ani neposílá. Z něj se odvozuje (HKDF) klíč trezoru, klíč databáze, podpisový klíč účtu a nově **důkaz klíče** `HKDF(kořen, "m5cet:key-proof:v1")`. Server si při registraci uloží jen `SHA-256(důkaz)`; z něj klíč nezjistí.

Přihlášení krok za krokem

KROK	CO SE OVĚŘÍ	KDYŽ SELŽE
1. Passkey	server passkey zná (je registrovaný), podpis a počítadlo sedí, user handle odpovídá účtu	neznámý passkey → chyba s doporučením registrace (<code>404 unknown-passkey</code>); audit <code>account.signin.unknown-passkey</code>
2. Globální klíč	relace je zatím zamčená ; <code>POST /api/account/unlock</code> pošle důkaz klíče, server porovná hash (v konstantním čase)	jiný klíč → relace se zruší, <code>403 wrong-key</code> , audit <code>account.signin.wrong-key</code>
3. Databáze	šifrovaná databáze uživatele se klíčem otevře	odhlášení, audit <code>account.client.database-locked</code>
4. Trezor	uložená data (profil, nastavení, připojení) se dešifrují	chyba se zapíše; data se nepřeplíšou
5. Server a verze	odezva serveru a shoda verze aplikace se serverem	jiná verze spustí kontrolu verzí, audit <code>account.client.version-mismatch</code>
6. Nastavení	načtení uživatelských nastavení do aplikace	—
7. Upozornění, připojení	propojení push, automatické připojení výchozího uloženého připojení	—

Okno Spojení ukazuje průběh (probíhá / hotovo / s výhradou / selhalo) a výsledek. Každý krok i každé selhání se zapíše do auditu v kategorii `account` (`account.signin.passkey-ok`, `account.signin.unlocked`, `account.client.signin-complete`, `account.client.signin-failed`, `account.register`, `account.register.failed` ...) s klientem a adresou — správce v konzoli vidí, proč se kdo nepřihlásil. Zamčená relace (před krokem 2) neotevře nic kromě `unlock`, `me`, odhlášení a hlášení událostí.

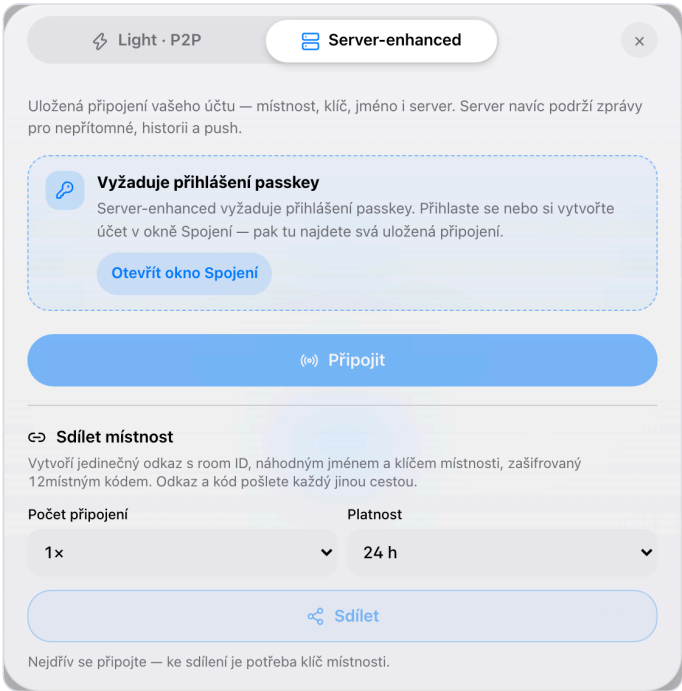
Adresy /signin a /signup

- `/signin` otevře okno Spojení a rovnou spustí přihlášení passkey.

- `/signup` otevře okno Spojení a spustí **registraci**: server vygeneruje uživatelské jméno, prohlížeč vytvoří passkey s PRF, pošle důkaz klíče a uživatel je hned **přihlášený a aktivovaný** (projde stejnými kroky jako přihlášení; nastavení zařízení se uloží do nového trezoru).



Obr. 18 — `/signup`: účet s vygenerovaným uživatelským jménem, ověřený globální klíč a průběh všech kroků.



Obr. 19 — Nepřihlášený uživatel: Server-enhanced odkazuje do okna Spojení, Připojit je neaktivní.

Úložiště

Jedna globální SQLite databáze pro server a jedna SQLCipher databáze pro každého uživatele či anonymní relaci (aplikace od 4.0 anonymní relaci nezakládá — Server-enhanced je jen pro přihlášené). Podrobně v [docs/storage.md](#).

```
$DATA_DIR/storage/
├─ m5cet.db      # globální: uživatelé, passkeys, index DB, logy, přenosy, audit, fronta
├─ storage.key   # master klíč, 0600 (nebo STORAGE_MASTER_KEY / STORAGE_KEY_FILE)
└─ db/
   ├─ db-<id>.db # účet – klíč z passkey (PRF), jen v paměti
   └─ db-<id>.db # relace – klíč zabalený master klíčem, TTL 24 h od posledního použití, max. 7 dní
```

Limity

Kvóta 64 MB na účet, 16 MB na relaci (zápis nad kvótou se vrátí); zpráva ≤ 256 KB; hodnota kv ≤ 4 MB; 30 nových relací za hodinu z jedné adresy, 5 000 živých celkem; nejvýš 200 otevřených databází (LRU).

Povýšení relace na účet

Po vytvoření passkey se data relace zkopírují do databáze účtu v jedné transakci (ATTACH s klíčem), počty se ověří před commitem; relace se smaže až potom.

Klíče

Klíč účtu drží server po držitelích (zařízeních); zapomene ho a přepíše nulami, až se odhlásí poslední, nebo po 12 h nečinnosti. ID relací jen jako HMAC.

Veřejný status

`/api/storage/status` říká jen `available`, `engine`, `caller`, `locked` — nic o ostatních uživateli. Úplný stav je v administraci.

Ochrana navigace a kontrola verzí 4.0

Nejdřív se odpojte

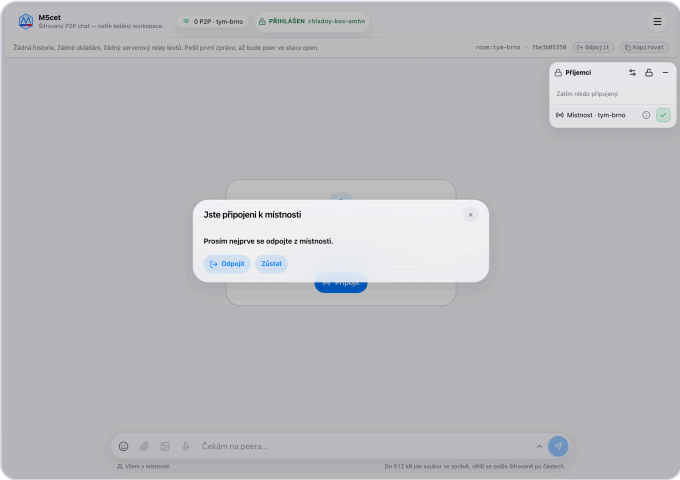
Dokud je uživatel připojený k místnosti (nebo se připojuje), aplikace brání tomu, aby stránku omylem opustil:

- **Zpět** (tlačítko prohlížeče, gesto, `Alt+←`, `Cmd+[`) a **obnovení** (`F5`, `Ctrl/Cmd+R`) otevřou okno „Prosím nejprve se odpojte z místnosti.“ s tlačítky **Odpojit** a **Zůstat**. Zpět zachytí strážný záznam v historii.
- **Zavření karty, nová adresa, obnovení z menu**: prohlížeč se zeptá vlastním dialogem (`beforeunload`); žádná stránka mu nesmí vnutit vlastní text).
- Po odpojení ochrana zmizí; *Smazat vše a odejít* ji uvolní sám.

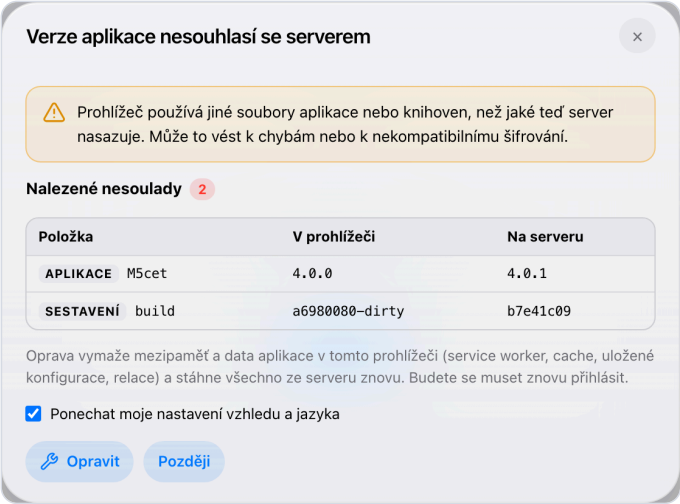
Kontrola verzí

Sestavení zapisíše `/version-manifest.json`: verzi, build, verzi protokolu, build service workeru, verze knihoven klienta a seznam souborů s SHA-256. Aplikace porovná s ním, co opravdu běží v prohlížeči — svou verzi a build, protokol, knihovny, načtené soubory a service worker — 2,5 s po startu, při návratu do okna, každých 10 minut, když server oznámí novou verzi, a při přihlášení, pokud server hlásí jiný build.

Při nesouladu otevře okno se seznamem rozdílů (aplikace, sestavení, protokol, knihovna, soubor, service worker — v prohlížeči / na serveru) a tlačítkem **Opravit**: smaže Cache Storage, odregistruje service worker, vymaže uložené konfigurace, relace a IndexedDB (kromě identity zařízení; volitelně ponechá vzhled a jazyk), požádá server o `Clear-Site-Data: "cache"` (`POST /api/clear-site-data`) a načte stránku znovu s parametrem, který obejde mezipaměti — vše včetně knihoven se stáhne čerstvé ze serveru.



Obr. 20 — F5 během spojení: nejdřív se odpojte.



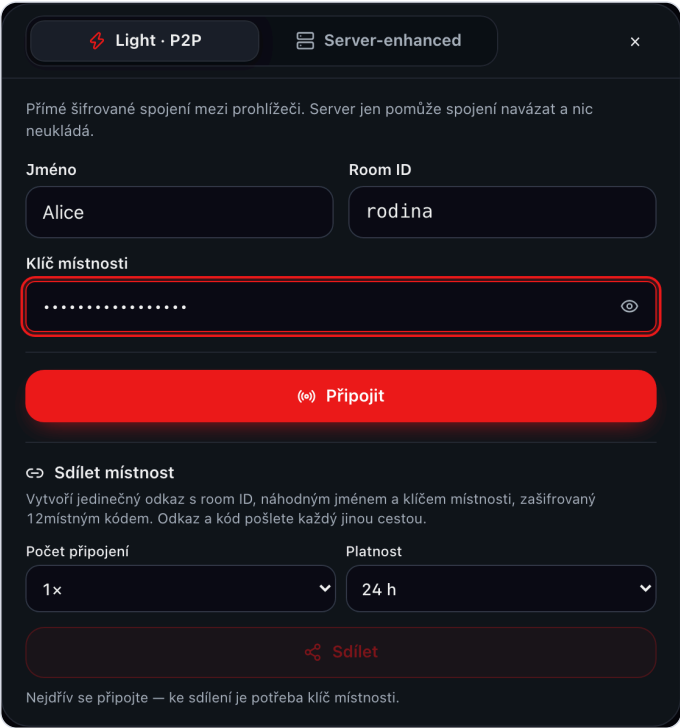
Obr. 21 — Server nasazuje jiné sestavení: seznam rozdílů a Opravit.

Okno Místnost 3.3

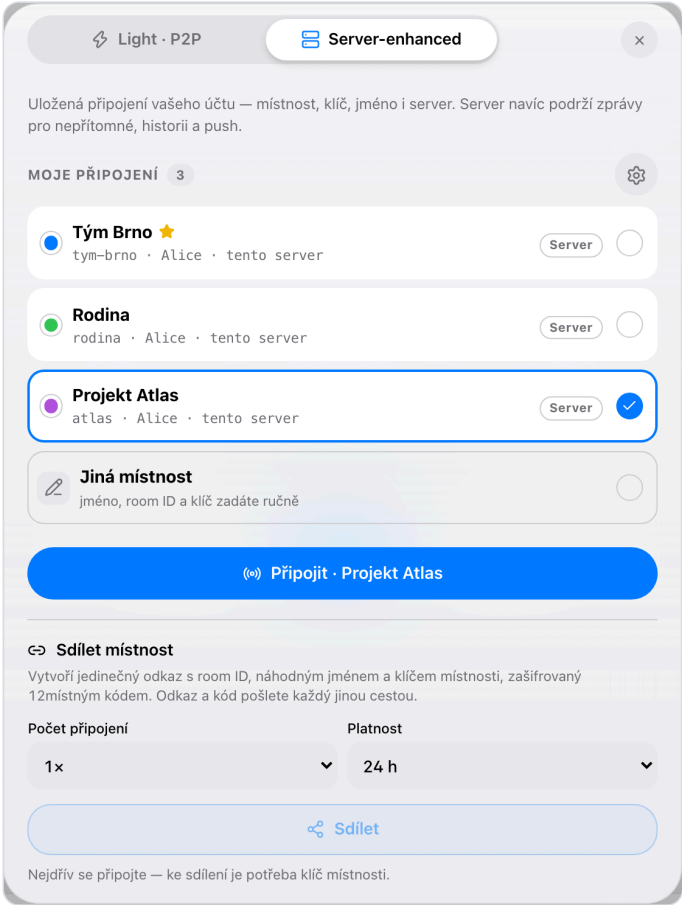
Otevře se klepnutím na logo vlevo nahoře. V záhlaví okna jsou místo názvu **záložky typu připojení** — *Light · P2P* a *Server-enhanced* (na úzkém displeji *P2P / Server*); vybraná záložka určuje režim. Pod obsahem záložky je část, která je vidět vždy: **Připojit / Reconnect, Odpojit a Sdílet místnost**.

ZÁLOŽKA	OBSAH
Light · P2P	jméno, room ID a klíč místnosti (klíč lze zobrazit okem)
Server-enhanced	uložená připojení ve stejném vzhledu jako v <i>Moje připojení</i> (barva, název, místnost · jméno · server, výchozí ★, režim) — bez tlačítek; připojení se vybere klepnutím a vybrané je orámované a zaškrtnuté. Poslední položka <i>Jiná místnost</i> ukáže pole pro ruční zadání. Nepřihlášený uživatel vidí od 4.0 jen výzvu k přihlášení s odkazem do okna Spojení a Připojit je neaktivní (Přihlášení a identita).

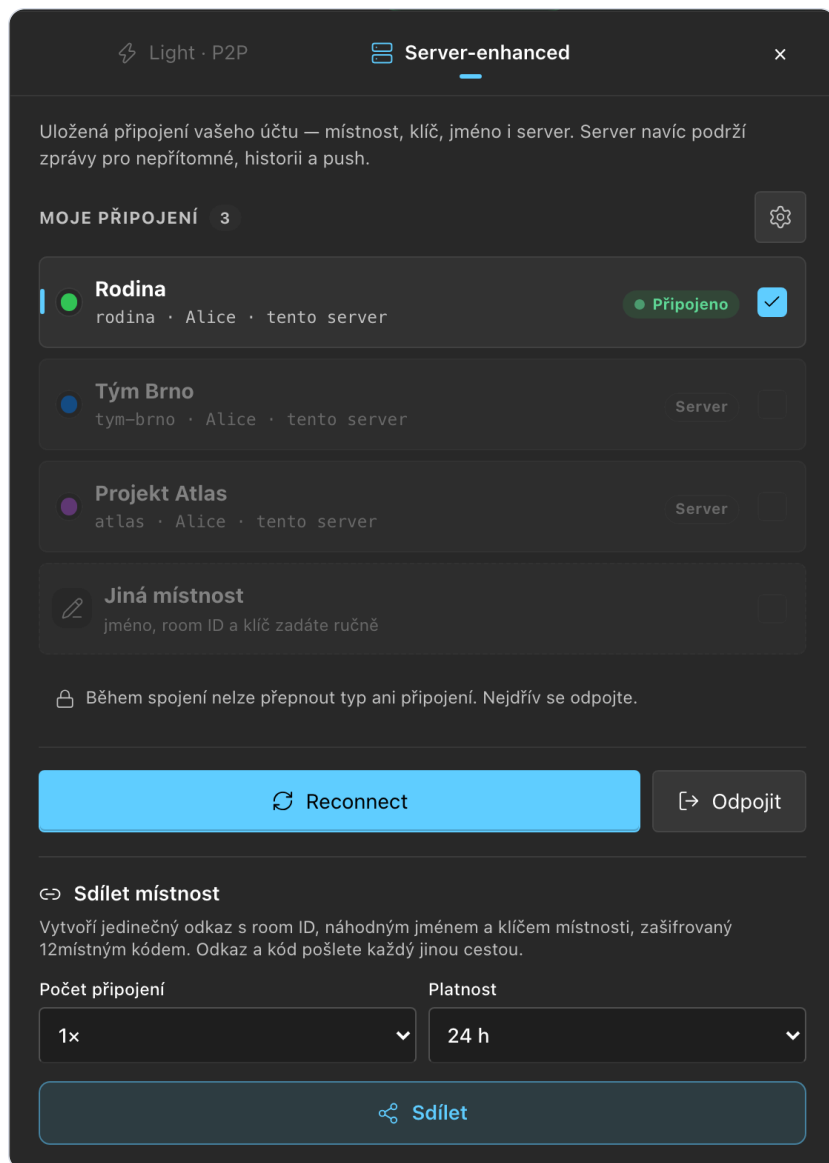
- **Ozubené kolo** vpravo nad seznamem otevře *Moje připojení* nad oknem Místnost (tam jsou Připojit / Odpojit / Upravit / Statistiky / Výchozí / Sdílet / Smazat). Po zavření se seznam obnoví: smazané zmizí, **nově vytvořené je rovnou vybrané**. Bez uložených připojení nabídne okno tlačítko *Vytvořit připojení*, které otevře rovnou formulář.
- Tlačítko Připojit zní podle výběru (*Připojit · Tým Brno*); vybrané uložené připojení se připojí se svým klíčem, jménem, serverem a nastavením, *Jiná místnost* a *Light · P2P* s tím, co je v polích.
- **Během spojení nic nepřepnete**: druhá záložka, ostatní záznamy i pole jsou neaktivní, připojený záznam ukazuje *Připojeno*; tlačítko Reconnect obnoví totéž spojení (uložené připojení i na jeho vlastním serveru). Po *Odpojit* jsou záznamy znovu k výběru.
- Okno otevřené s uloženým připojením v použití (i po obnovení stránky) začne na záložce *Server-enhanced*.
- Okna nad sebou: Escape zavře jen to horní, každé další ztmaví obrazovku jen trochu. Všechna okna se vykreslují do `<body>`, takže skleněné okno pod nimi (iOS 27) jejich pozici neovlivní.
- Ovládání z klávesnice: šipky mezi záložkami (WAI-ARIA tabs), seznam je skupina přepínačů (`radiogroup`).



Obr. 15 — Light · P2P (MotoSport Dark): pole a vždy viditelná tlačítka a sdílení.



Obr. 16 — Server-enhanced (iOS 27): výběr uloženého připojení, ozubené kolo ke správě.



Obr. 17 — Během spojení (Windows 11, tmavý tón): ostatní záznamy i záložka čekají na Odpojit.

Uložená připojení 3.2

Přihlášený uživatel v režimu **Server-enhanced** si uloží místnosti i s klíčem, jménem a nastavením, připojí se k nim jedním klepnutím (nebo z přepínače v záhlaví), vybere výchozí a nechá se k němu připojit hned po přihlášení. U každého připojení vidí statistiky a log. Menu > *Moje připojení*.

POLOŽKA PŘIPOJENÍ	VÝZNAM
Název, barva	jak se připojení jmenuje v seznamu a v přepínači; barevná tečka
Místnost, klíč, jméno	vše, co chce formulář pro připojení; klíč lze zobrazit a vygenerovat (24 znaků z 57 bez záměn, ~140 bitů, bez modulo zkreslení)
Obslužný server	<i>tento server</i> , server ze seznamu správce, nebo vlastní <code>wss://</code> adresa, pokud to správce dovolí (níže)
Režim, historie chatu	light / Server-enhanced a uchování chatu — nastaví se při připojení
Mízení zpráv	výchozí TTL zpráv odeslaných v této místnosti (nebo „nemizí“)
Relay pro nepřítomné	server drží zprávy, když uživatel není online — jen na tomto serveru
Upozornění	upozornění na nové zprávy v této místnosti
Automaticky znovu připojit	po výpadku se připojení obnoví samo (jde vypnout u jednoho připojení)
Udržování spojení	<i>šetrná</i> (ping 45 s, základ zpoždění obnovy 1,5 s), <i>vyvážená</i> (25 s, 1 s), <i>rychlá</i> (12 s, 0,5 s); zpoždění roste exponenciálně s náhodným rozptylem (full jitter), nejvýš 2 minuty

Nastavení

- Výchozí připojení** a **po přihlášení se připojit** k němu (když žádné není, k naposledy použitému) — jednou po každém načtení stránky, a jen když není připojené nic jiného (třeba místnost obnovená z relace).
- Po výpadku se připojit znovu** (vypnuté i u jednotlivého připojení = po výpadku zůstane odpojené) a **znovu připojit po návratu na stránku a do sítě** (návrat skryté nebo zmrazené stránky, událost `online`).
- Přepínač v záhlaví** vedle stavu spojení; volitelně **dotaz před přepnutím** z jiné místnosti.
- Statistiky a log** vypnout (správce je může vypnout všem).
- Tlačítko *Uložit aktuální místnost* převezme místnost, klíč, jméno a nastavení, se kterými je uživatel právě připojený.

Statistiky a log

Počet připojení, čas online (celkem, nejdelší, průměr), zprávy a soubory odeslané / přijaté, objem souborů, obnovení spojení, neúspěšné pokusy, chyby, nejvíc lidí v místnosti a kdy naposledy. Log zapisuje vytvoření a úpravy, pokusy, připojení a odpojení, obnovy, chyby, příchody a odchody lidí a soubory; zprávy se jen počítají. Filtry *Vše / Spojení / Lidé / Soubory / Chyby*, **export JSON** (bez klíče místnosti) a vymazání. Délku logu určuje správce (výchozí 200 řádků na připojení).

Kde a jak se ukládají

Seznam, nastavení, statistiky i log tvoří jeden objekt, který prohlížeč **zapečetí klíčem trezoru účtu** (AES-GCM, klíč z PRF passkey, stejně jako profil) a uloží jako třetí část trezoru: `PUT /api/account/vault s { connections: { ct, count } }`. Server drží jen šifrovaný blok (nejvýš 1,5 MB), jeho velikost, čas a počet připojení — nikdy klíč, název místnosti ani to, kdo se kam připojil. Úprava se uloží hned; počítadla (zprávy, obnovy) se sbírají a odejdou nejdéle za 8 s, nebo když se stránka skryje či uživatel odhlásí. Dokud se trezor nenačte, klient nic nezapiše (starší zařízení tak nepřepíše novější data).

Sdílet připojení 3.3

V *Moje připojení* má každý záznam mezi hvězdičkou a košem tlačítko **Sdílet**. Otevře okno *Sdílet připojení* se stejnou pozvánkou jako okno Místnost — room ID a klíč zapečetěné 12místným kódem, omezený počet použití a platnost (1 h / 24 h / 7 d) — upravenou pro tuto situaci:

- Není potřeba být připojený** — klíč je uložený v připojení.
- Karta připojení nahoře (barva, název, místnost, server) a volby jako **řada tlačítek** místo rozbalovacích seznamů.
- Jméno pro pozvaného** (jinak náhodné, např. `bystry-sokol-42`).
- Používá-li připojení **jiný signalizační server**, pozvánka ho nese (zapečetěný s klíčem) a pozvaný se připojí tam — jen pokud ho správce tohoto serveru povoluje (Client & addons). Adresa, která není čistě `wss://`, pozvánku zneplatní.
- Výsledek: odkaz, kód, sdílení přes aplikace, QR, nový odkaz, zneplatnění, *Hotovo*.

Sdílet připojení

Projekt Atlas

atlas · tento server

Pozvánka ponese místnost a klíč tohoto připojení, zašifrované 12místným kódem. Odkaz a kód pošlete každý jinou cestou. Připojení být nemusíte.

Počet připojení

1x

2x

3x

5x

10x

25x

Platnost

1 h

24 h

7 d

Jméno pro pozvaného

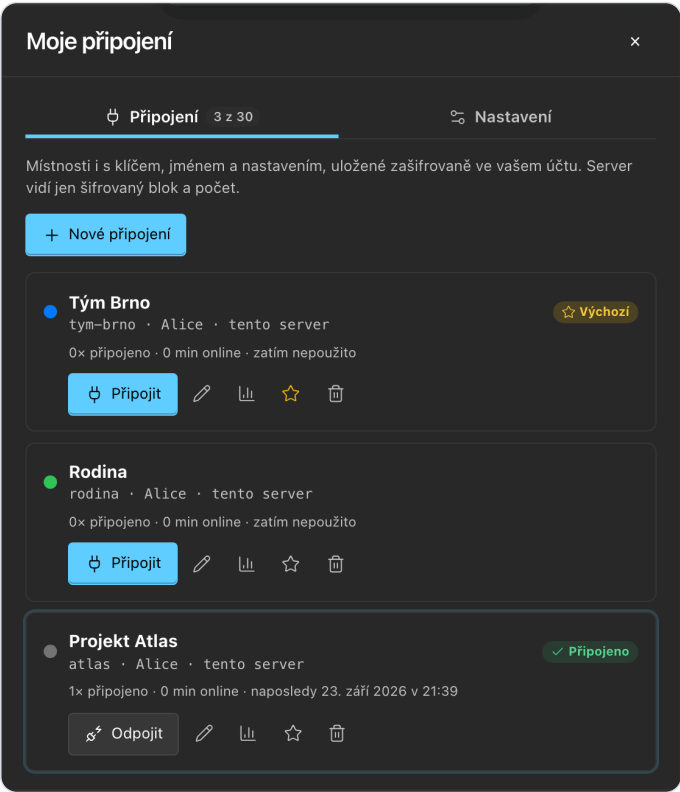
Bob

Vytvořit pozvánku

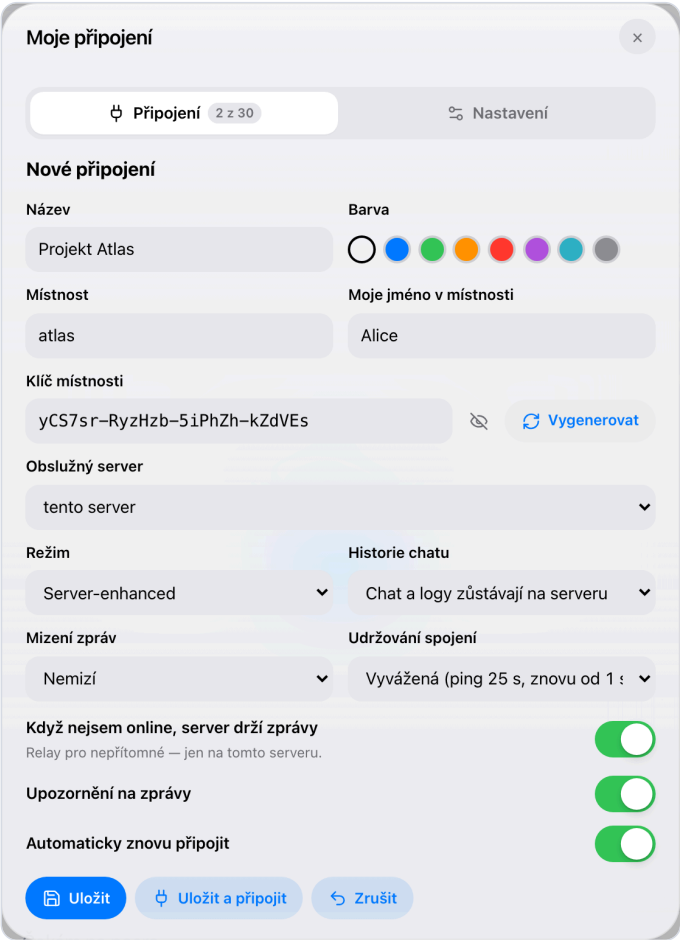
Obr. 18 — Sdílet připojení: 3 použití, 7 dní, jméno pro pozvaného.

Jiný obslužný server

Připojení může místo tohoto serveru použít jiný signalizační server M5cet: ze seznamu, který správce vede v konzoli, nebo — když to dovolí — libovolnou adresu `wss://` (`ws://` jen pro `localhost`; bez přihlašovacích údajů, dotazu a fragmentu). Tamní provozovatel musí mít tuto doménu v `ALLOWED_ORIGINS`. Na cizí server klient **neposílá token účtu**, neotevívá úložiště přes WebSocket, nevyužívá relay pro nepřítomné a **neposlouchá tamní administrátorské příkazy**. Šifrování je stejné — klíč místnosti prohlížeč neopustí nikdy. Od 3.3 si relace pamatuje i server a uložené připojení, takže **obnovení stránky** vrátí spojení na tentýž server (dříve na tento).



Obr. 10 — Moje připojení v šabloně Windows 11 (tmavý tón): výchozí, připojené, akce.



Obr. 11 — Úprava připojení v šabloně iOS 27: klíč, server, režim, historie, TTL, udržování spojení.

Šablony vzhledu 3.2

Šablona mění celé GUI: barvy, písmo, tvary, stíny, průhlednost, ovládací prvky (přepínače, segmenty, záložky, výběry), bubliny zpráv, menu a styl ikon. K šabloně se dál kombinuje barevná variace (6) a rozvržení (4). Vzhled > Šablona.

RODINA	ŠABLONA	TÓNY	IKONY
Systémové	iOS 27	světlý + tmavý	badge
	Windows 11	světlý + tmavý	thin
Klasické	Motorsport Dark	tmavý	outline
	Glass Light	světlý	outline
	Terminal Secure	tmavý	outline
	Midnight	tmavý	outline
	Paper	světlý	outline
	Vysoký kontrast	tmavý	bold
Studio	Aurora — polární záře, skleněné karty	tmavý	duotone
	Nord — arktické pastely	tmavý + světlý	thin
	Sakura — růžová, měkké tvary	světlý	duotone
	Ocean — tyrkys a hlubina	světlý + tmavý	outline
	Graphite — grafit s oranžovou	tmavý + světlý	bold

iOS 27

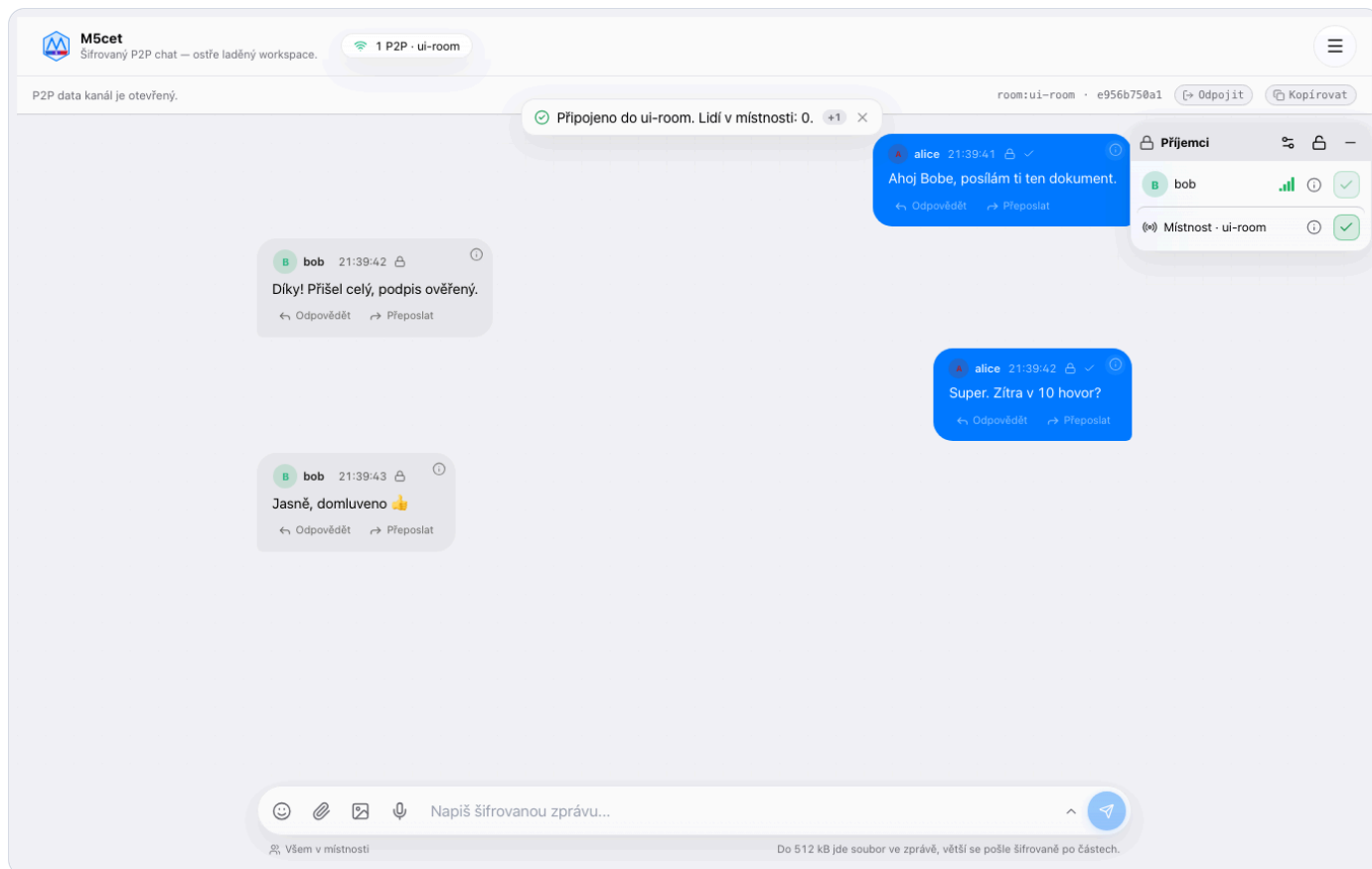
Systémové barvy iOS (modrá #007AFF, skupinové pozadí, oddělovače), písmo SF Pro (na Apple zařízeních systémové), **Liquid Glass** — průsvitné vrstvy s rozostřením a sytostí, světlý okraj a měkký stín — na záhlaví, menu, oknech a kompozéru. Bubliny jako iMessage (moje modré vpravo, cizí šedé), kompozér jako kapsle, zelené přepínače, segmentové přepínače s vystouplým vybraným segmentem, velké zaoblení, ikony menu na barevných čtverečkách jako v Nastavení; na telefonu okna jako spodní listy s úchytem.

Windows 11

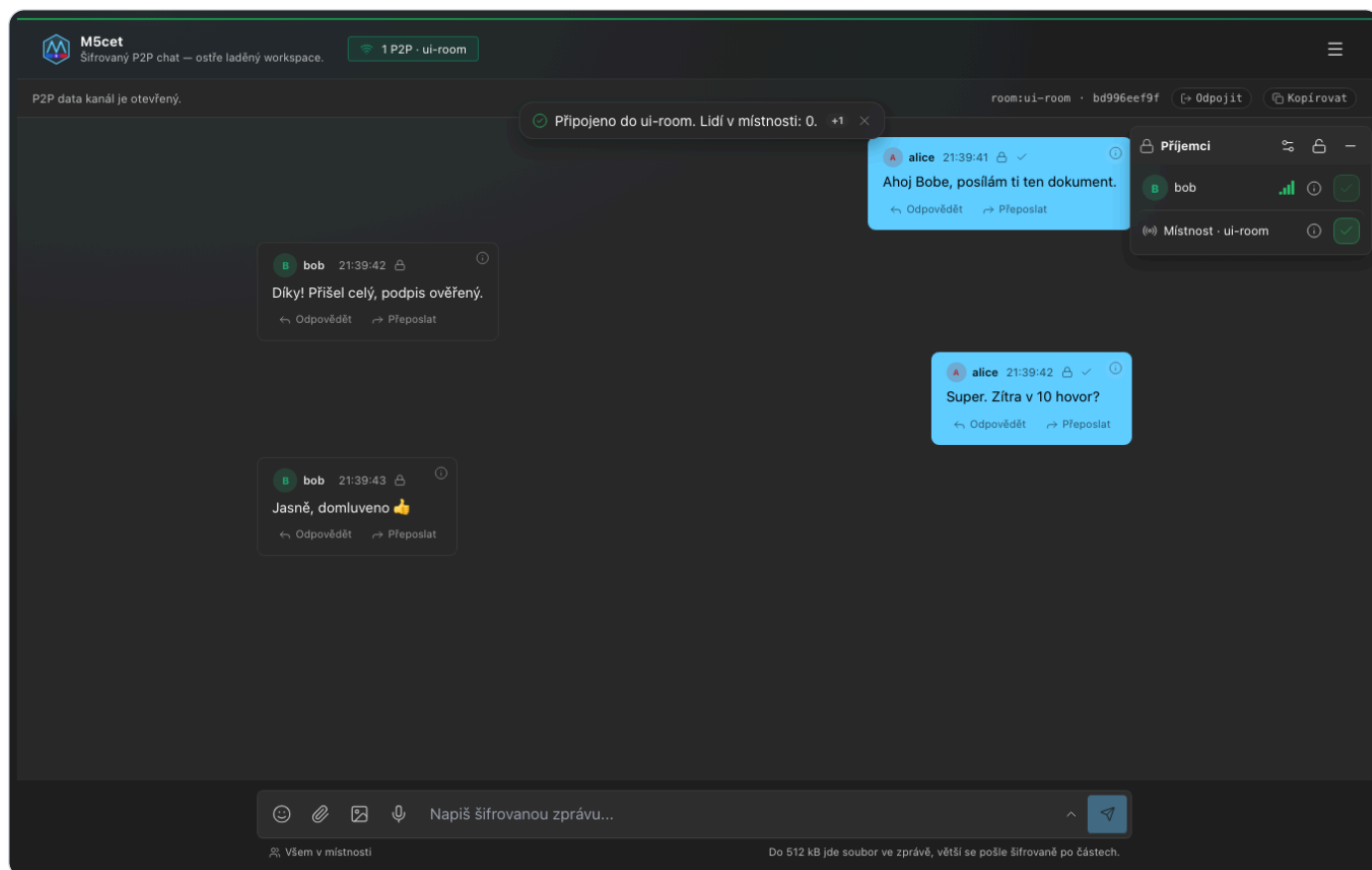
Fluent: materiál **Mica** v okně, **akrylové** nabídky a vyskakovací okna, Segoe UI Variable, zaoblení 4–8 px, jemný rámeček s elevací (spodní hrana tmavší), akcent #005FB8 / #60CDFD, pole s akcentovou linkou při fokusu, vybraná položka menu s proužkem vlevo, přepínače, segmenty a záložky (pivot) s akcentovou linkou, tenké ikony 1,5 px.

Tón a ikony

- **Tón** u šablon se dvěma tóny: *automaticky* (podle nastavení systému, mění se za běhu), *světlý*, *tmavý*.
- **Ikony**: *podle šablony*, nebo `outline` (2 px), `thin` (1,5 px), `bold` (2,5 px), `duotone` (tah a jemná výplň), `badge` (ikony menu na barevných čtverečkách).
- Technicky: atributy `data-theme`, `data-tone`, `data-icons` a `data-family` na `<html>` a CSS proměnné (`client/src/themes.css`, katalog `client/src/lib/theme-catalog.ts`). Šablona se nastaví ještě před prvním vykreslením Reactu — stránka neblíkne jinou barvou.
- Které šablony uživatel smí vybrat, výchozí šablonu, tón a ikony a zámek určuje správce (Client & addons). Zařízení, které si šablonu nevybralo, ukazuje výchozí.



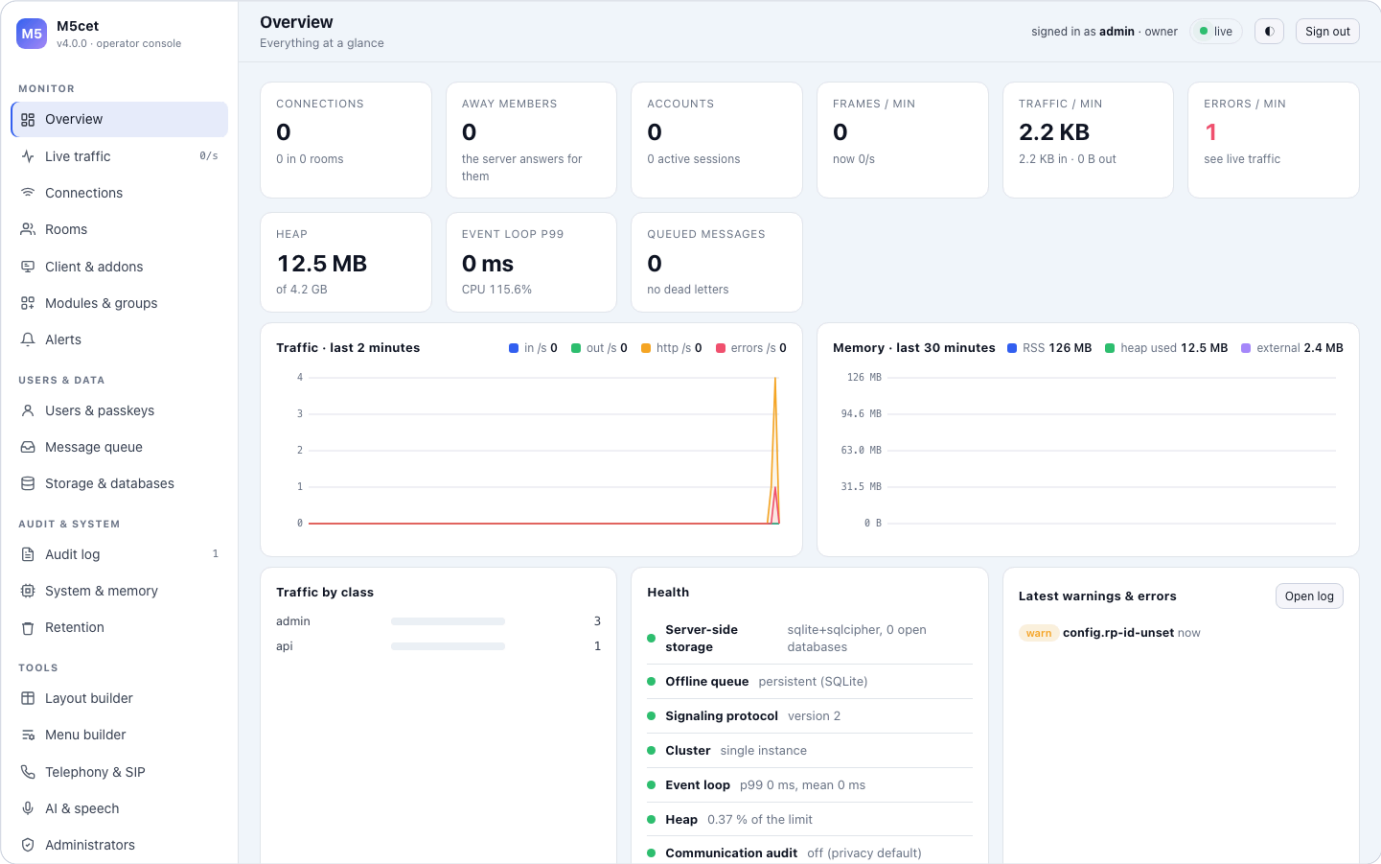
Obr. 12 — Chat v šabloně iOS 27 (světlý tón).



Obr. 13 — Chat v šabloně Windows 11 (tmavý tón).

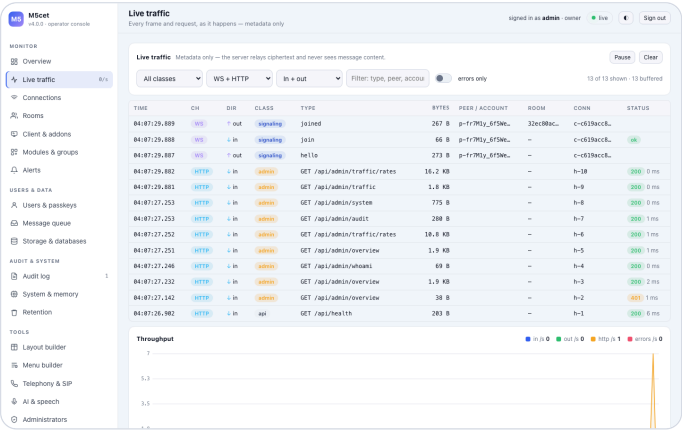
Administrace

Konzole operátora na `https://<doména>/console/`. Živá data bere z hlavní služby (`/api/admin/*`), nástroje layoutu, telefonie a AI z admin služby (`/admin/*`). Přístup mají jmenovití administrátoři s rolí; `ADMIN_API_TOKEN` zůstává jako vlastník „admin“.

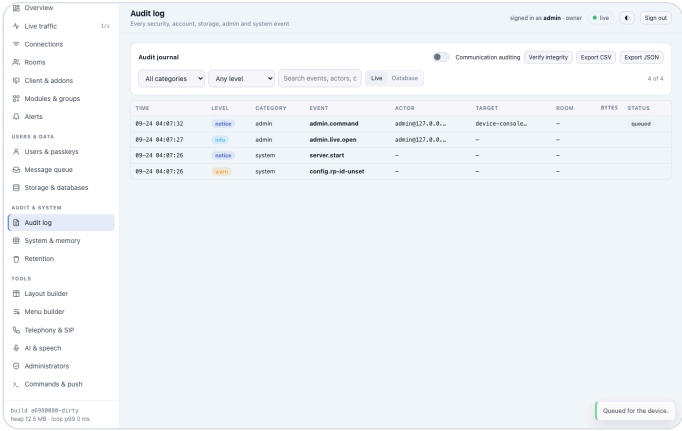


Obr. 6 — Přehled: KPI, provoz za 2 minuty, paměť za 30 minut, provoz podle tříd, zdraví, poslední varování.

PANEL	CO UKAZUJE A CO UMÍ
Přehled	spojení, místnosti, nepřítomní, účty a relace, rámce a bajty za minutu, chyby, heap, event loop p99, fronta; grafy; zdravotní kontroly (úložiště, trvalost fronty, event loop, heap, audit komunikace)
Živý provoz	každý WebSocket rámec a HTTP požadavek (čas, kanál, směr, třída, typ, bajty, peer/účet, hash místnosti, spojení, stav, doba); filtry, pauza, detail; graf propustnosti
Spojení	otevřené sockety: jméno, peer, účet, místnost, klient (prohlížeč/OS), adresa /24 (/48), bajty a rámce, doba, protokol; odpojení
Místnosti	členové online i nepřítomní (hash místnosti, nikdy název)
Uživatelé a passkeys	účty, databáze (klíčování, otevřená), trezor, fronta, relace; detail s passkeys, položkami ve frontě, auditem účtu, žurnálem a provozem; odhlásit všude, smazat (s opsáním ID)
Fronta zpráv	stav fronty (uloženo / doručuje se / mrtvé / velikost / trvalost), schránky po účtech, dead-letter s obnovou
Úložiště	globální DB (soubor, WAL, stránky, žurnál, tabulky a řádky), index šifrovaných databází
Audit	žurnál s filtry (kategorie, úroveň, text), zdroj paměť / databáze, export CSV a JSON, přepínač auditu komunikace
Systém a paměť	RSS, heap (a limit), external, event loop p99 / průměr, CPU, zátěž hostitele, otevřené zdroje; grafy 30 minut
Retence	politika uchovávání, další běh, ruční spuštění
Příkazy a push	příkaz pro zařízení (doručí se hned, je-li připojené), čekající příkazy, log doručení, stav VAPID, odběratelé, testovací push
Client & addons	využití uložených připojení; jejich politika a další signalizační servery; povolené šablony vzhledu, výchozí šablona, tón a ikony, zámek (níže) 3.2
Modules & groups	moduly aplikace i nástroje konzole: výchozí přístup, přístupové skupiny, hlavní skupina, granty na části modulu, log; přepínače služeb AI, Speech a Functions; aktivní znaky psaní zprávy; log přístupů (níže) 4.0 · 5.2
Menu builder	menu a navigace aplikace: pořadí přetažením, sekce, položky, HTML s proměnnými, oddělovače, speciální tlačítka, styly a stavy, náhled (Menu builder) 4.0
Layout builder	GUI designer rozvržení celé aplikace (hlavní obrazovka, okno Místnost, okna, dialogy, panely): paleta prvků, strom s přetahováním, vlastnosti s našeptáváním, šablony, varianty pro skupiny a šablony vzhledu, historie s návratem, sloučení po aktualizaci, vložení HTML, kontrola přístupnosti, náhled samotné aplikace; texty a chování zpráv (Layout builder) 4.0.5 · 4.13
AI & speech	poskyvatelé a klíče, modely a ceny, výchozí modely a pokyny asistenta, limity, zkušebna, test řeči, žurnál volání s živým proudem a CSV (AI a řeč) 4.14
Functions	přepínač služby, galerie vestavěných balíčků, webhooks s logem a replayem; balíčky a verze, editor CodeMirror (zvýraznění, našeptávač, šablony, nápověda, kontrola) a vizuální tvůrce, živé zkušební běhy s odpovídáním na otázky, modely (klíčové slovo, vstupy, webhook, API), plány (cron), běhy s logy, tutoriál, import/export <code>.m5pkg</code> (Funkce) 5.1 · 5.2
Telefonie	přenesené nástroje z předchozí administrace



Obr. 7 — Živý provoz.



Obr. 8 — Auditní žurnál.

Role a přístupy 3.1

ROLE	SMÍ
auditor	Číst všechno (přehled, provoz, spojení, uživatele, frontu, úložiště, audit a jeho ověření, metriky, alerty); spravovat vlastní passkey. Tlačítka, která něco mění, má zašedlá — a server je odmítne (403).
operátor	navíc zasahovat: odpojit spojení, odhlásit nebo smazat účet, obnovit položku fronty, příkazy a testovací push, retence, zálohy a kontrola integrity, prahy alertů, kontrolní bod auditu
vlastník	navíc spravovat administrátory a jejich tokeny, <code>VACUUM</code>

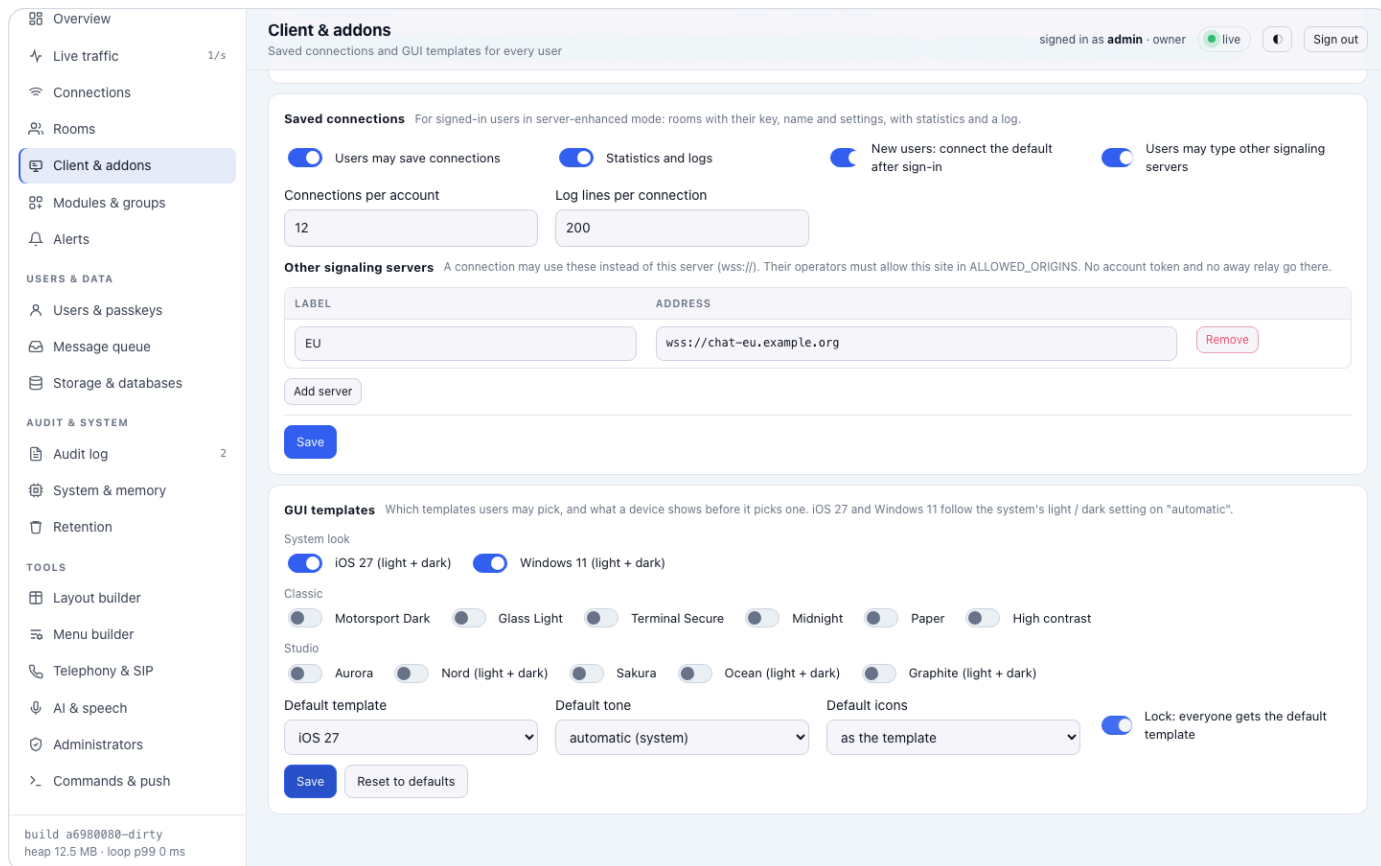
- **Odkud se berou:** `ADMIN_API_TOKEN` (vlastník), `ADMIN_TOKENS="alice:operator:token,bob:auditor:token"` v prostředí, a administrátoři založení v konzoli (panel *Administrátoři*) s pojmenovanými tokeny — v `admin-users.json` jen jejich hashe.
- **Přihlášení passkey:** administrátor si v konzoli zaregistruje passkey (*Můj passkey*) a příště se přihlásí bez tokenu; relace konzole platí 8 h.
- Každý zásah se v auditu zapíše **jménem** administrátora (`actor`), ne „admin“.

Client & addons 3.2

Co správce zapíná pro všechny klienty. Uloží se do `$DATA_DIR/client-config.json` (nebo `CLIENT_CONFIG_FILE`; sdílí ho všechny instance), klienti si ho berou z `GET /api/client-config` při startu a pak každých 5 minut. Číst smí auditor, měnit operátor; každá změna je v auditu jako `admin.client-config`.

NASTAVENÍ	VÝCHOZÍ	VÝZNAM
Users may save connections	ano	uložená připojení pro přihlášené v režimu Server-enhanced
Statistics and logs	ano	klienti vedou statistiky a log připojení
New users: connect the default after sign-in	ano	výchozí hodnota uživatelského nastavení
Users may type other signaling servers	ne	jinak jen tento server a servery ze seznamu
Connections per account, Log lines per connection	30, 200	limity (max. 200 a 2 000)
Other signaling servers	—	až 20 serverů (název, <code>wss://</code> adresa), které připojení smí použít
GUI templates	všechny	šablony, které si uživatelé smějí vybrat
Default template / tone / icons	Motorsport, automaticky, podle šablony	co ukazuje zařízení, které si nevybralo
Lock	ne	všichni dostanou výchozí šablonu, výběr je zamčený

Horní karta ukazuje využití: počet účtů, kolik z nich má uložená připojení a kolik připojení celkem — z počtů, které hlásí prohlížeč; obsah trezoru server nepřečte.

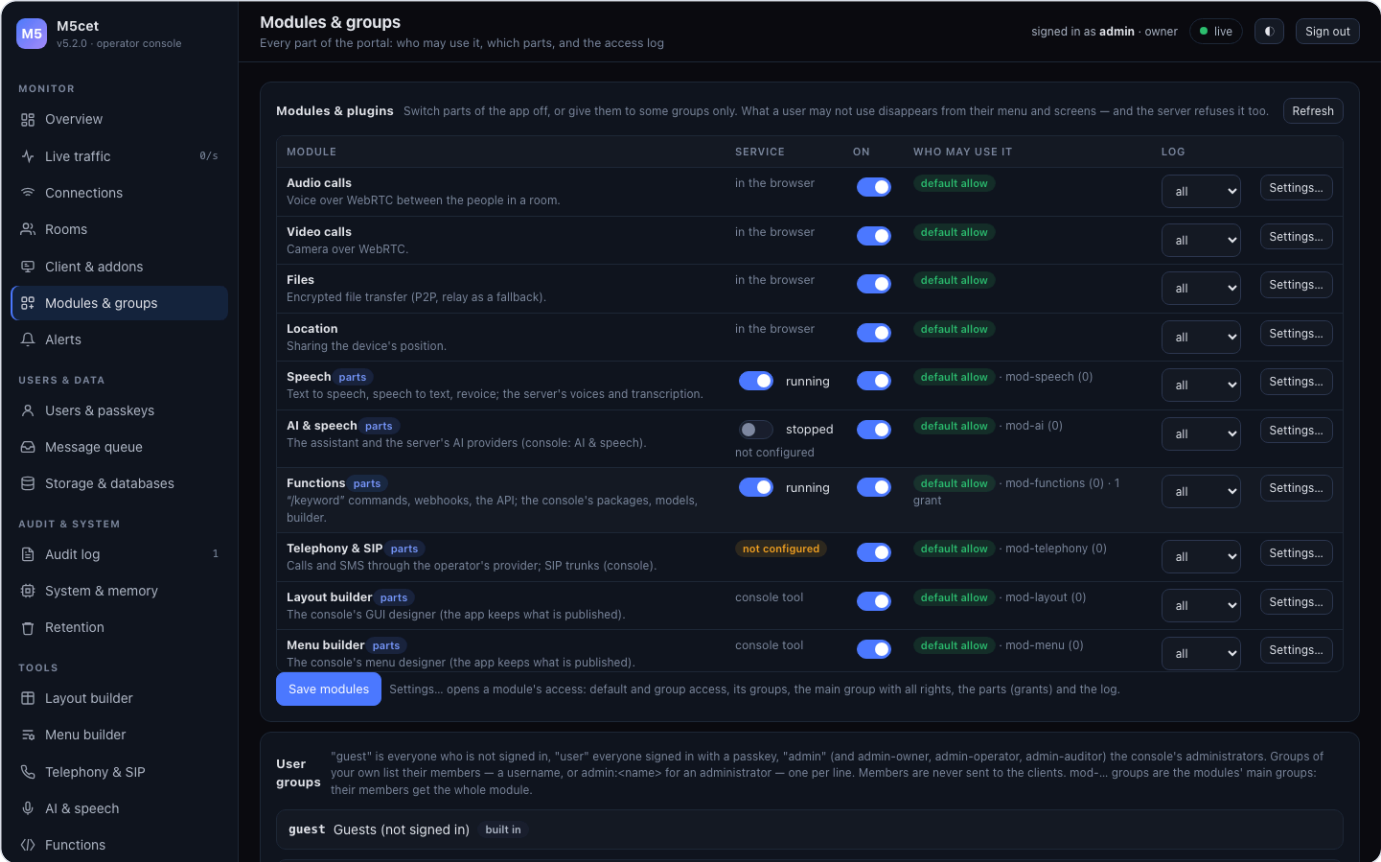


Obr. 14 — Konzole > Client & addons: politika připojení, další server, povolené šablony (jen iOS 27 a Windows 11), zámek.

Moduly a skupiny 4.0

Konzole > *Modules & groups*. Každý modul aplikace — `audio`, `video`, `files`, `location`, `speech`, `ai`, `telephony`, `nfc`, `invites`, `connections`, `notifications`, `analytics`, `appearance`, `editMode` — je buď zapnutý pro všechny, zapnutý jen pro vybrané skupiny, nebo vypnutý. Skupiny: `guest` (nepřihlášení), `user` (přihlášení passkey) a vlastní skupiny správce, jejichž členy jsou **uživatelská jména**.

- **Aplikace** modul, který uživatel nesmí, schová: položky menu (i v Menu builderu), panely, tlačítka ve psaní zprávy (soubory, obrázky, hlasové zprávy), sdílení, Edit Mode. Otevřený panel vypnutého modulu se zavře.
- **Server** to, co modul obsluhuje, odmítne (`403 module-disabled`): `/api/ai/complete`, `/api/speech/tts|stt`, `/api/telephony/sms|call`, `/api/share/create`, `/api/push/subscribe|test`, `/api/account/push`. Skupiny volajícího určí z jeho relace.
- Ukládá se do konfigurace klienta (`modules`, `groups`); **členové skupin se klientům nikdy neposílají** — účet se dozví jen své skupiny (`/api/account/me` > `groups`). Sloupec *On the server* ukazuje, zda je serverová část modulu nakonfigurovaná.

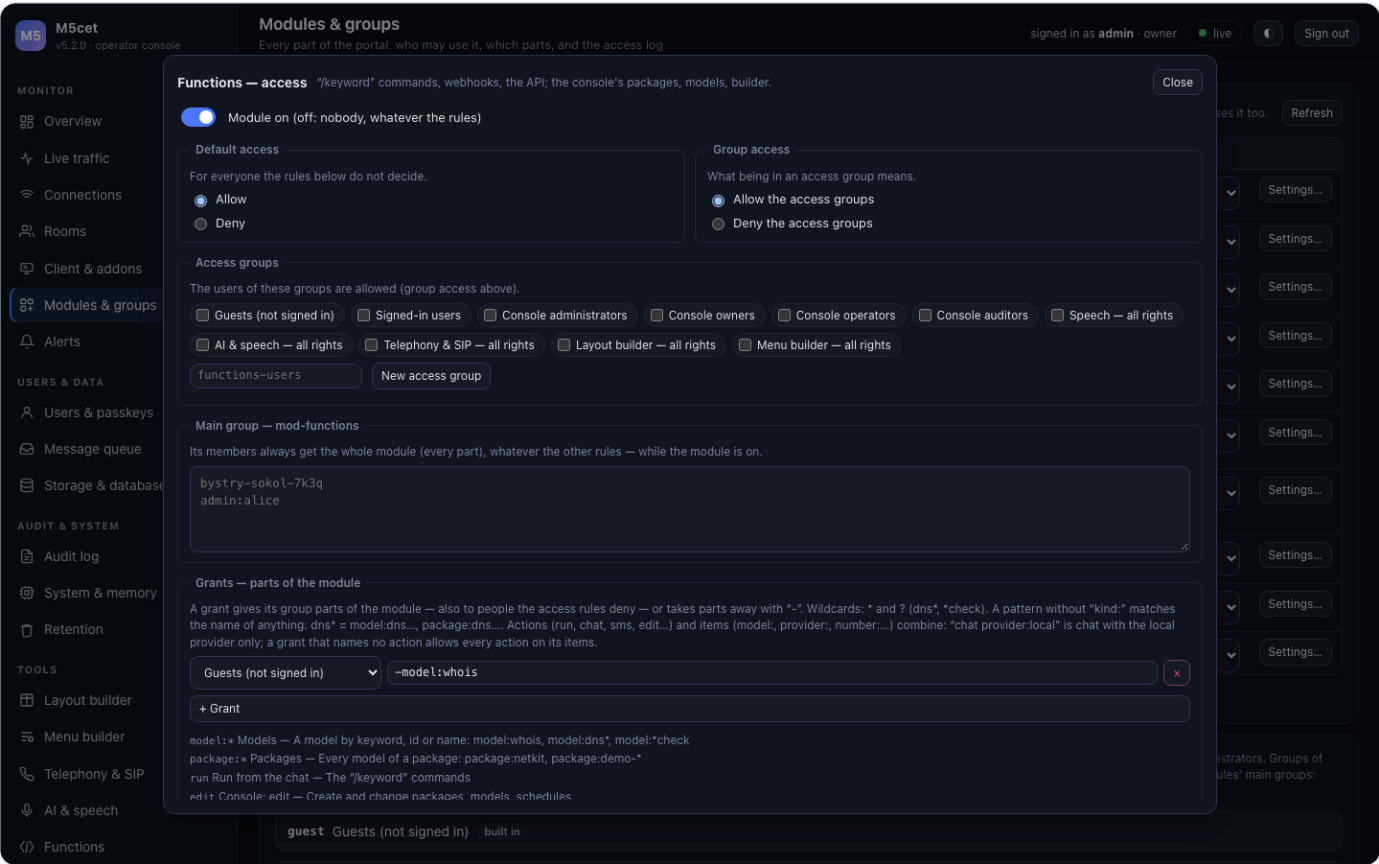


Obr. 22 — Konzole > Modules & groups (5.2): moduly aplikace i nástroje konzole, přepínače služeb (Speech a Functions běží, AI bez poskytovatele), kdo smí modul používat, hlavní skupiny (mod-...) a granty, úroveň logu.

Nástroje jako moduly 5.2

Vedle modulů aplikace jsou v *Modules & groups* i nástroje: **Functions**, **AI & speech** (moduly `ai` a `speech`), **Telephony & SIP**, **Layout builder** a **Menu builder**. Sloupec *Service* ukazuje, zda služba běží, a u AI, Speech a Functions ji rovnou zapne či vypne (`PUT /api/admin/modules/switches`) — **bez zapnuté služby Functions příkazy / v chatu nefungují**. Tlačítko *Settings* otevře nastavení modulu:

VOLBA	VÝZNAM
Enabled	modul vypnutý pro všechny (kromě vlastníka konzole)
Default access <code>allow</code> / <code>deny</code>	přístup těch, kdo nejsou v žádné skupině modulu
Group access <code>allow</code> / <code>deny</code>	členové přístupových skupin mají přístup povolený, nebo naopak zakázaný
Access groups	seznam skupin (i nová skupina přímo z dialogu) s editorem členů
Main group <code>mod-<modul></code>	hlavní skupina se všemi právy modulu; vznikne sama, když chybí (při startu služeb a při otevření stránky)
Grants	skupina → seznam práv: jen části modulu, nebo naopak odebrání (níže)
Log	<code>all</code> (povolení i odmítnutí), <code>deny</code> (jen odmítnutí), <code>off</code>
Test access	vysvětlí rozhodnutí pro uživatele (<code>alice</code>), správce (<code>admin:bob@operator</code>) nebo hosta (<code>guest</code>), volitelně s právem



Obr. 43 — Nastavení modulu Functions: výchozí a skupinový přístup, přístupové skupiny, hlavní skupina `mod-functions` a grant, který hostům odebírá `/whois`.

Pořadí rozhodnutí: vypnutý modul → hlavní skupina (vše) → základ: člen přístupové skupiny podle *Group access*, ostatní podle *Default access* → granty skupin volajícího k základu **přidají** udělené části (i tomu, kdo by jinak přístup neměl) nebo právy s **-** části **odeberou**. Členy skupin mohou být uživatelská jména aplikace i správci konzole jako `admin:jméno`; vestavěné skupiny konzole jsou `admin`, `admin-owner`, `admin-operator` a `admin-auditor`. Konzole schová nástroje, ke kterým správce nemá přístup; **vlastník konzole se nikdy nezamkne** (důvod `owner` v logu).

Práva a granty 5.2

Právo je vzor se zástupnými znaky `*` a `?`, volitelně s druhem (`model:`, `package:`, `provider:`, `number:`); vzor bez druhu platí pro hodnotu (`*check` = `model:webcheck`). Právo s **-** na začátku **odebírá** a má přednost. **Akce** (`run`, `chat`, `sms`, `edit`...) a **položky** (`model:`, `package:`, `provider:`, `number:`) se kombinují: `chat provider:local` je chat jen s lokálním poskytovatelem, `run model:dns*` spouštění jen příkazů `dns`...; grant, který nejmenuje žádnou akci, dovolí nad svými položkami všechny akce; grant, který nejmenuje nic z toho, oč jde (např. jen čísla při nastavení konzole), nestačí. Vzory se předkompilují a rozhodnutí se kešuje podle konfigurace a skupin, takže kontrola stojí jen pár porovnání.

MODUL	PRÁVA	KDE SE KONTROLUJÍ
Functions	<code>model:dns*</code> , <code>package:net*</code> , <code>-model:whois</code> , <code>run</code> ; konzole <code>edit</code> , <code>publish</code> , <code>webhooks</code>	seznam příkazů a běh v chatu, API, webhooks, <code>m5.functions.list()</code> ; <code>/admin/functions/*</code>
AI	<code>provider:openai</code> , <code>model:anthropic/claude*</code> , <code>-provider:elevenlabs</code> , <code>chat</code> ; konzole <code>settings</code> , <code>playground</code>	seznam modelů a každé volání (asistent, funkce); <code>/admin/ai*</code>
Speech	<code>provider:local</code> , <code>model:local/piper-cs*</code> , <code>tts</code> , <code>stt</code>	převod řeči a hlasy
Telephony & SIP	<code>sms</code> , <code>call</code> , <code>number:+420*</code> , <code>-number:+1900*</code> ; konzole <code>settings</code> , <code>test</code>	<code>/api/telephony/sms call</code> podle cílového čísla; <code>/admin/telephony/*</code>
Layout / Menu builder	<code>edit</code> , <code>publish</code> , <code>history</code>	zápisy v <code>/admin/layout*</code> a <code>/api/admin/menu-config</code> (čtení smí každý s přístupem)

Odmítnutí vrací `403` s kódem `module-disabled` (modul) nebo `module-denied` (právo). Pravidla nástrojových modulů a členy skupin, v nichž jsou správci konzole (`admin:...`), smí měnit **jen vlastník** (`403 owner-only`) — operátor by si jinak mohl přidělit libovolný nástroj; totéž platí pro přepínače služeb (AI a Speech potřebují `settings` v AI & speech, Functions `edit`). URL webhooku, tajemství HMAC a API token modelu vidí jen ten, kdo smí jeho webhooks měnit (`webhooks` nebo `edit` pro daný model); auditor nikdy.

Log přístupů 5.2

Každé povolení i odmítnutí (podle volby *Log* modulu) se zapíše s časem, modulem, subjektem (`alice`, `admin:bob`, `guest`, `webhook`), rozhodnutím, důvodem (`main-group`, `group-allow`, `group-deny`, `grant`, `default-allow`, `default-deny`, `right`, `off`, `owner`), právem, cestou, IP a kanálem (aplikace / konzole). Zápis jde po dávkách do denních souborů `$DATA_DIR/access/access-RRRR-MM-DD.jsonl` (`ACCESS_LOG_DAYS`, výchozí 30; `ACCESS_LOG=0` vypne). Karta *Access log* v *Modules & groups* má filtry (modul, rozhodnutí, subjekt, text), statistiku a živé obnovování; API `GET /admin/access/log`.

Access log
Every module decision — for app users, the console's administrators, webhooks and the API — allowed and refused, as each module's Log setting says. Kept 30 days (`ACCESS_LOG_DAYS`) in `$DATA_DIR/access`.
Refresh

every module
allowed and refused
who (username, admin)
2 days
live

functions 81 ✓ 0 ×
modules 1 ✓ 0 ×

TIME	MODULE	WHO	DECISION	WHY	PART / REQUEST	VIA
9/29/2026, 3:04:00 PM	functions	admin:admin admin	allow	default — allow	GET /admin/functions	con 127
9/29/2026, 2:45:13 PM	functions	admin:admin admin	allow	default — allow	GET /admin/functions/builtins	con 127
9/29/2026, 2:45:10 PM	functions	admin:admin admin	allow	default — allow	webhooks edit · GET /admin/functions/webhooks/calls/whc_mumnsffw7c83180fbb	con 127
9/29/2026, 2:45:09 PM	functions	admin:admin admin	allow	default — allow	webhooks edit · GET /admin/functions/webhooks/calls	con 127
9/29/2026, 2:45:09 PM	functions	admin:admin admin	allow	default — allow	webhooks edit · GET /admin/functions/webhooks	con 127
9/29/2026, 2:45:07 PM	functions	admin:admin admin	allow	default — allow	GET /admin/functions/builtins	con 127
9/29/2026, 2:45:07 PM	functions	admin:admin admin	allow	default — allow	GET /admin/functions	con 127
9/29/2026, 2:45:07 PM	functions	admin:admin admin	allow	default — allow	GET /admin/functions/sdk	con 127
9/29/2026, 2:45:05 PM	functions	admin:admin admin	allow	default — allow	GET /admin/functions	con 127
9/29/2026, 2:43:23 PM	functions	admin:admin admin	allow	default — allow	GET /admin/functions/builtins	con 127
9/29/2026,	functions	admin:admin	allow	default — allow	webhooks edit · GET	con

Obr. 44 — Log přístupů: kdo, kdy, rozhodnutí, důvod a část modulu (u konzole i požadované právo), filtry a statistika po modulech.

Aktivační znaky psaní zprávy 5.2

Karta *Message input* — *activation characters*: znak na začátku slova otevře našeptávač — `/` **příkazy** (funkce), `@` **zmínky** (lidé v místnosti), `#` **štítky** (předvolené a použité v konverzaci). Lze přidat další znaky (např. `!` pro příkazy, nejvýš 10) a předvolené štítky. `@jméno` a `#štítek` se ve zprávách zvýrazní; klik na štítek zobrazí jen zprávy s ním (filtr se zruší křížkem). Seznam příkazů se obnovuje každou minutu, po návratu na záložku a před odesláním neznámého příkazu; když je služba Functions vypnutá nebo nic není k dispozici, našeptávač to řekne.

Bezpečnost konzole

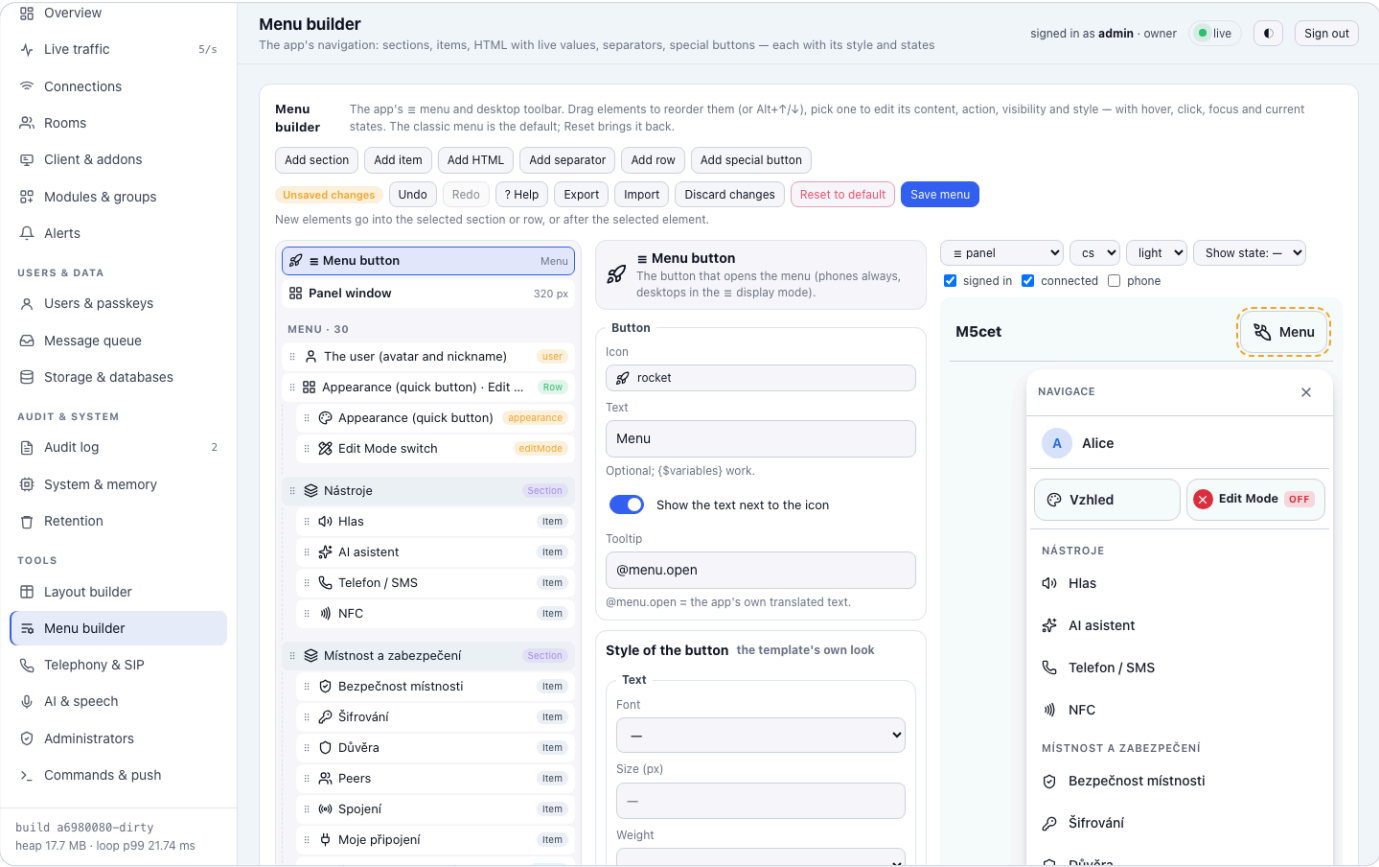
- Token je jen v paměti záložky (volitelně v `sessionStorage`), **nikdy v `localStorage`** — starý klíč `m5cet:admin:cfg` konzole při načtení smaže.
- CSP `script-src 'self'`, žádné inline skripty ani `eval`, `frame-ancestors 'none'`. Vše ze serveru jde do stránky přes `textContent`.
- Živý stream je Server-Sent Events čtený přes `fetch()` — token v hlavičce, ne v URL (nezůstane v logu proxy).
- Odmítnuté tokeny se počítají: 30 za 15 minut na adresu. Každý zásah operátora se zapíše do auditu (kategorie `admin`).
- Doporučeno omezit `/console/`, `/admin/` a `/api/admin/` v nginx na adresy správců (`allow ... ; deny all;`).

Menu builder 4.0

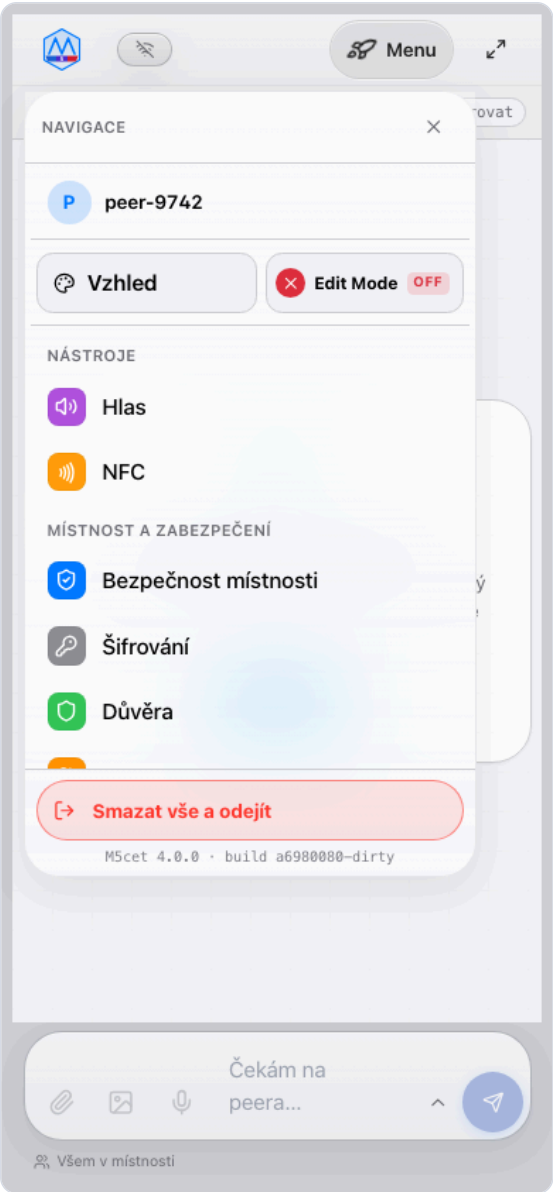
Konzole › *Menu builder*. Menu aplikace (panel za tlačítkem $\equiv$ a nástrojová lišta na desktopu) je od 4.0 **data**: `client/src/lib/menu-config.ts` je popisuje, `MainMenu.tsx` vykresluje. Výchozí konfigurace je přesně dosavadní menu — stejné položky, skupiny, třídy i test ID (ověřeno testem, který porovná DOM starého a nového menu ve všech režimech zobrazení). *Reset to default* ho kdykoli vrátí.

PRVEK	CO NASTAVÍTE
$\equiv$ tlačítko	ikona ze seznamu, text (i s proměnnými), zobrazit text, tooltip, styl a stavy
Panel	šířka (220–560 px), titulek, záhlaví a zavírací tlačítko; styl okna, záhlaví, nadpisů sekcí a výchozí styl všech položek
Sekce	nadpis (např. <i>Místnost a zabezpečení</i>), ikona, položky; prázdná sekce se nezobrazí
Položka	ikona, text, odznak, akce: panel (21 panelů), funkce (<code>openRoom</code> , <code>signIn</code> , <code>connectDefault</code> , <code>disconnect</code> , <code>toggleEditMode</code> , <code>toggleTone</code> , <code>setTheme:ios</code> , <code>setLang:en</code> , <code>toggleNotifications</code> , <code>clearQuit</code>), odkaz (<code>https://</code> nebo cesta tohoto webu, volitelně nová karta)
HTML	blok HTML s proměnnými, podmínkami a cykly (šablonovací jazyk); <code>data-action="panel:..."</code> nebo <code>"fn:..."</code> udělá z čehokoli tlačítko
Oddělovač	čára, čárkovaná, tečkovaná, dvojité nebo jen mezera; barva, tloušťka, odsazení
Řada	tlačítka vedle sebe (jako Vzhled a Edit Mode nahoře)
Speciální tlačítko	uživatel, Vzhled, Edit Mode, světlý / tmavý, oznámení, účet (uživatelské jméno nebo přihlášení), Smazat vše a odejít, verze; u přepínačů volba <i>zobrazit stav</i> (ON / OFF)

- **Viditelnost** každého prvku: modul (skrýje se, když modul uživatel nemá; položka s panelem se řídí modulem panelu sama), situace (`signedIn`, `signedOut`, `connected`, `disconnected`, `phone`, `desktop`), nebo skrytý (zůstane v builderu).
- **Styl** každého prvku: zarovnání, obtékání (zalomit, jeden řádek, se třemi tečkami), barva textu, pozadí, ikony a rámečku (barvy šablony nebo vlastní), velikost a typ písma, tloušťka, kurzíva, podtržení, verzálky, prostrkání, velikost a pozice ikony, odsazení, mezera, výška, zaoblení, rámeček, průhlednost, zvětšení, stín. Totéž (barvy, tloušťka, podtržení, průhlednost, zvětšení, stín) pro stavy **hover**, **click**, **focus** (klávesnice) a **current** (otevřený panel nebo zapnutý přepínač).
- **Ovládání**: přetažení myší (do sekce či řady, před či za prvek), `Alt + ↑/↓` z klávesnice (na kraji seznamu přesune prvek ven ze sekce), duplikace, skrytí, smazání, zpět / znovu (`Ctrl + Z`), export a import JSON. Nápoředa (*? Help*) je plovoucí okno s proměnnými, filtry, makry a příklady; klepnutí vloží ukázkou do právě upravovaného HTML.
- **Náhled** vpravo kreslí menu stejnými třídami a styly jako aplikace, s ukázkovými hodnotami (Alice, místnost *tym-brno*): panel nebo nástrojová lišta, jazyk, tón, přihlášení / připojení / telefon a vynucený stav vybraného prvku. Klepnutí v náhledu vybere prvek.
- **Uložení**: `PUT /api/admin/menu-config` (operátor; auditor builder jen čte) → server vše znovu ověří (neznámé panely, funkce, ikony, odkazy mimo `https` a tento web, velikosti) a uloží do `$DATA_DIR/menu-config.json` (nebo `MENU_CONFIG_FILE`); audit `admin.menu-config`. Aplikace si menu bere z `GET /api/menu-config` při startu a každých 5 minut a pamatuje si ho pro příští start.



Obr. 23 — Menu builder: struktura s přetahováním, vlastnosti ≡ tlačítka, živý náhled.



Obr. 24 — Totéž menu v aplikaci (telefon): nástroje nahoře, oddělovač, HTML blok, vypnuté moduly chybí.

Šablonovací jazyk

Texty položek a bloky HTML jsou šablony podobné Latte (`client/src/lib/menu-template.ts`). Text začínající `@` (např. `@menu.settings`) je překlad aplikace. Výsledek HTML se nikdy nevloží jako `innerHTML`: rozloží se na bezpečný strom (povolené značky, atributy a styly bez `url()`, odkazy jen `https:` a cesty webu, žádné `on...`, `script`, `style`, `iframe`, formuláře) a vykreslí jako prvky Reactu. Vše vypsané se escapuje.

MAKRO	VÝZNAM
<code>{\$proměnná}</code> , <code>{a.b[0]}</code>	hodnota (escapovaná)
<code>{=výraz}</code>	výraz: <code>{=\$room.peers + 1}</code> , <code>{='Místnost ' ~ \$session.room}</code>
<code>{if}...{elseif}...{else}...{/if}</code>	podmínky <code>s</code> <code>==</code> <code>!=</code> <code>===</code> <code><</code> <code>></code> <code><=</code> <code>>=</code> <code>&&</code> <code> </code> <code>!</code> a závorkami
<code>{ifset \$x}...{/ifset}</code>	když hodnota existuje
<code>{foreach \$list as \$item}...{else}...{/foreach}</code>	cyklus (i <code>\$key => \$value</code>), <code>\$iterator.counter</code> , <code>.first</code> , <code>.last</code> , <code>.odd</code> , <code>.even</code>
<code>{var \$x = ...}</code>	vlastní proměnná
<code>{icon shield-check}</code>	ikona z katalogu (128 ikon lucide)
<code>{_ 'menu.room' }</code>	přeložený text aplikace
<code>{* ... *}</code> , <code>{l}</code> <code>{r}</code>	komentář, literální složené závorky

PROMĚNNÉ	OBSAH
\$app	name, version, build, lang, online
\$global, \$now	server (host), time; \$now = čas v ms
\$user	signedIn, username, nickname, avatar, groups, keyVerified
\$session	username = current_username (účet, nebo P2P jméno relace), nickname, connected, status, room, mode, server, peers, rtt, since
\$room	name, peers, people (přezdívky)
\$connection	active, default, saved
\$settings	theme, tone, accent, lang, notifications, editMode, retention

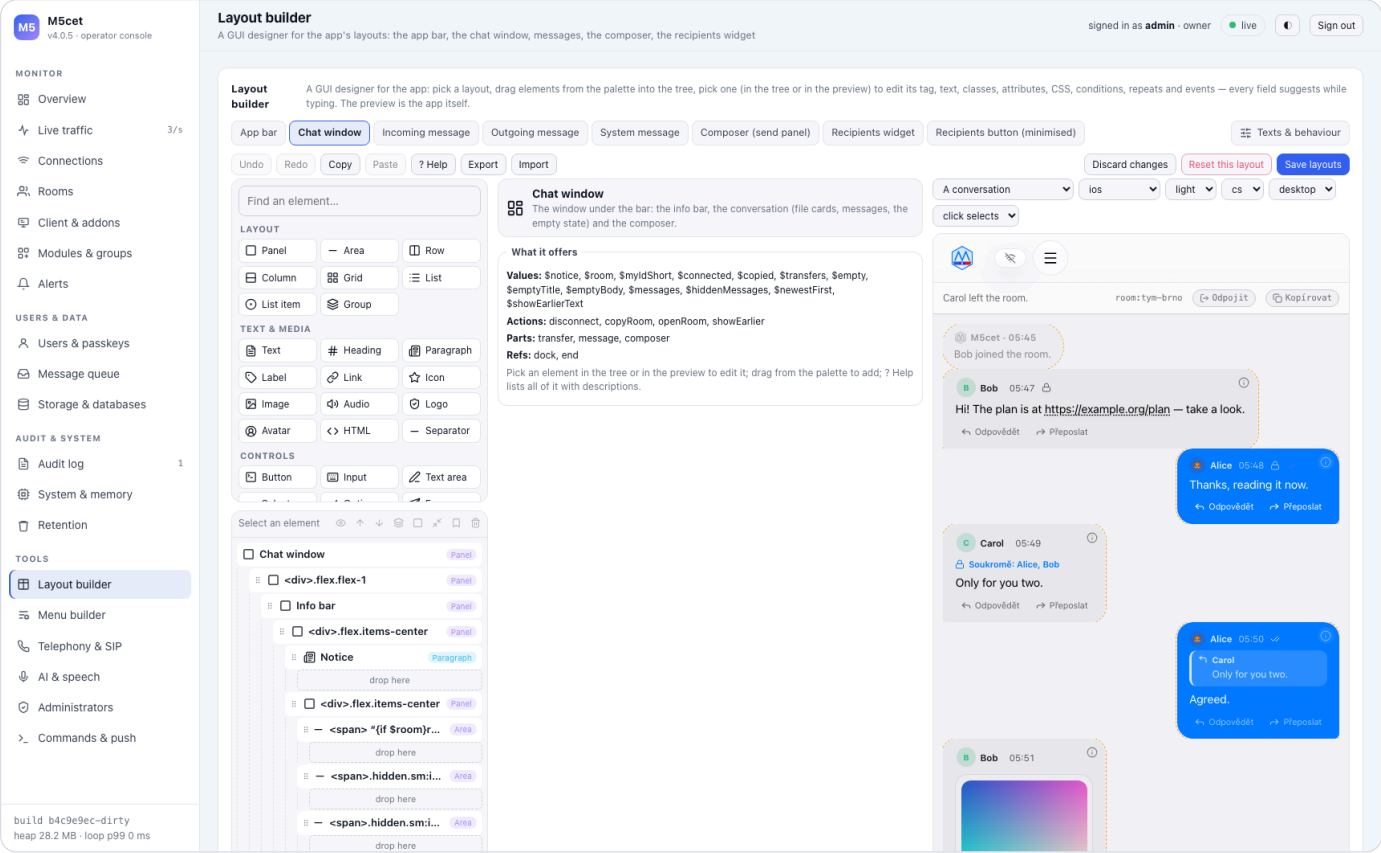
Filtry (`{ $x | filtr: arg }`, řetězí se): `upper`, `lower`, `capitalize`, `firstUpper`, `truncate`, `replace`, `default`, `date` (písmena jako v PHP, `{ $now | date: 'H:i' }`), `number`, `round`, `length`, `join`, `first`, `last`, `bytes`, `duration`, `ago`, `yesno`, `padLeft` a další — úplný seznam s příklady je v nápovědě builderu. Hodnoty se v aplikaci obnovují živě (změna místnosti, lidí, připojení; čas každých 30 s).

```
<!-- kdo a kde, tlačítko do nastavení -->
<div style="padding:6px 12px">{if $user.signedIn}<b>{$user.username}</b>{else}{$session.username|default:'host'}}{/if}
  {ifset $session.room} · {$session.room|upper} · {$room.peers} {'menu.peers'}{/ifset}</div>
<button data-action="panel:settings">{icon settings} {'menu.settings'}</button>
<button data-action="fn:setTheme:ios">iOS</button>
```

Layout builder — GUI designer4.0.5 · 4.13

Konzole > *Layout builder*. Části aplikace jsou od 4.0.5 **stromy prvků** — data, která kreslí `LayoutView.tsx`. Výchozí stromy vykreslují přesně totéž, co dřív kód komponent: při převodu se porovnal DOM starých a nových komponent v 248 situacích (typy zpráv, přílohy, zapečetění, mizení, stavy doručení, widget se všemi stavy lidí, lišta, okno chatu, psaní) a výsledek hlídají snímky v testech. Správce tak mění design i strukturu bez zásahu do kódu; nezměněná rozvržení dál sledují aktualizace aplikace. Od 4.13 jsou rozvržením i okno Místnost, okna, dialogy a panely — **45 rozvržení** v pěti sekcích (níže); ve 4.14 přibyl asistent AI.

ROZVRŽENÍ	CO KRESLÍ	ŽIVÉ ČÁSTI
App bar	logo (otevře okno Místnost), stav spojení, přepínač uložených připojení, celá obrazovka	odznak účtu, menu (Menu builder)
Chat window	informační pruh (oznámení, místnost, Odpojit, Kopírovat), konverzace, prázdný stav, „zobrazit starší“	karta přenosu souboru, zpráva, pole pro psaní
Incoming / Outgoing / System message	bublina: hlavička, čas a ikony, citace, přeposlání, soukromé, tělo s odkazy, přílohy, zapečetění, podržení, mizení, akce, stav doručení	odznak odesílatele (příchozí)
Composer	citovaná zpráva, emoji, přílohy, pole, patička s příjemci	hlasová zpráva, odeslání s volbami
Recipients widget / button	hlavička, nastavení, lidé s latencí a volbou, místnost, vše / nic; minimalizované tlačítko	—



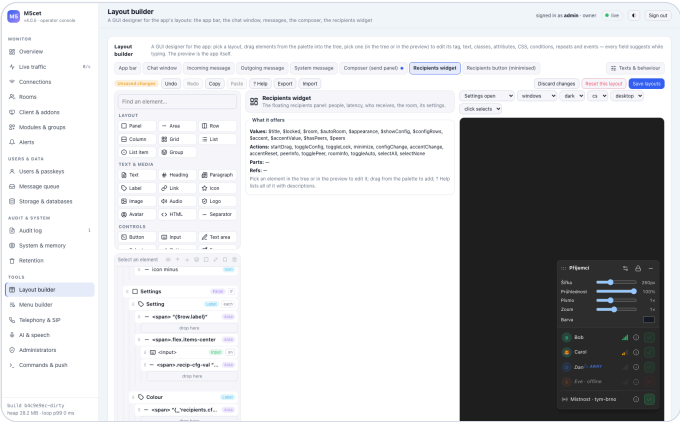
Obr. 25 — Layout builder: záložky rozvržení, paleta, strom, vlastnosti, náhled (šablona iOS 27) s vybranou zprávou.

Paleta a nastavení prvků

- Rozložení:** Panel (`div`, `section`, `header`, `nav` ...), Area (`span`, `strong`, `small` ...), Row, Column, Grid, List a List item, Group (bez vlastního prvku). **Text a média:** Text (šablona; formát *links*), Heading, Paragraph, Label, Link, Icon (207 ikon), Image, Audio, Logo, Avatar, HTML (bezpečné), Separator. **Ovládání:** Button, Input (text, password, number, email, range, color, file...), Text area, Select + Option, Form. **Logika:** App part (živá část aplikace), Template (šablona prvků). Kontejnery se libovolně vnořují.
- Každé pole našeptává během psaní** (šipky, Enter / Tab): tag, **třídy, které styly aplikace opravdu mají** (čtou se z buildu), názvy atributů podle tagu a jejich hodnoty (`type` inputu, `role`, `target`, `autocomplete` ...), 114 vlastností CSS s hodnotami (`display` : block, flex, grid, contents...), proměnné

rozvržení ({`$room`} , `$peers`), akce a živé části.

- Hodnota atributu je šablona, nebo výraz po `=` se zachovaným typem: `=$openPeerCount == 0` , `=$private ? '1' : null` (`null` atribut vynechá). Výrazy umí `&&` `||` `!` `==` `!=` `<` `>` , podmínku `?` `:` a filtry.
- **Logika:** zobrazit jen když, opakovat pro každou položku seznamu (`$peers` jako `$p` , klíč, `$iterator`), CSS z dat, ref a **události** → **akce** s argumentem (`click` → `togglePeer` , `$p.id`).
- **Styl a stavy** — týž editor jako v Menu builderu, včetně hover, click, focus a current.



Obr. 27 — Widget příjemců (Windows 11, tmavý tón) se strukturou nastavení a opakováním lidí.

view, messages, the composer, the recipients widget

Element

ID

btn-emoji

Name in the tree

Emoji

a-z, 0-9, "-"; unique in this layout

Hidden (kept here, not drawn)

Content

Text (before the children)

A template: {\$variable}, {_translation.key'}, {if \$x}...{else}...{/if}, {\$x|upper}.

Variables, filters & macros

Attributes

class

composer-icon-btn shadow-

type

button

×

data-testid

button-emoji

×

aria-expanded

=emojiOpen

×

aria-label

{_ 'chat.emoji'}

×

title

{_ 'chat.emoji'}

×

add an attribute...

A value is text with {\$variables}, or an expression after "=" (=openPeerCount == 0, =\$x ? 'on' : null — null leaves the attribute out). Links: https:// or this site's paths.

CSS

1 set

display

×

add a CSS property...

block

inline

inline-block

flex

inline-flex

grid

inline-grid

contents

flow-root

list-item

Values may use {\$variables}. No "Style and states" below.

use

Logic: when, repeat, even

Show only when

e.g. \$connected && !\$e

Repeat for each item of

e.g. \$peers

CSS from data

Key of an item

e.g. \$p.id

an object of CSS the app computes (a bubble's colours, the widget's position)

Ref

a handle the app uses (scrolling, measuring)

Events → actions

click

toggleEmoji

argument

×

add an event (click, change, submit...)

This layout's actions: submit, input, keydown, toggleEmoji, insertEmoji, pickFile, pickImage, attachment, replyJump, cancelReply.

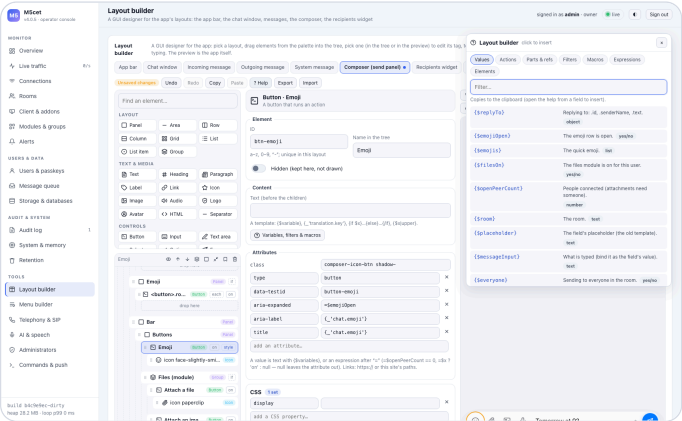
Style and states

the template's own look

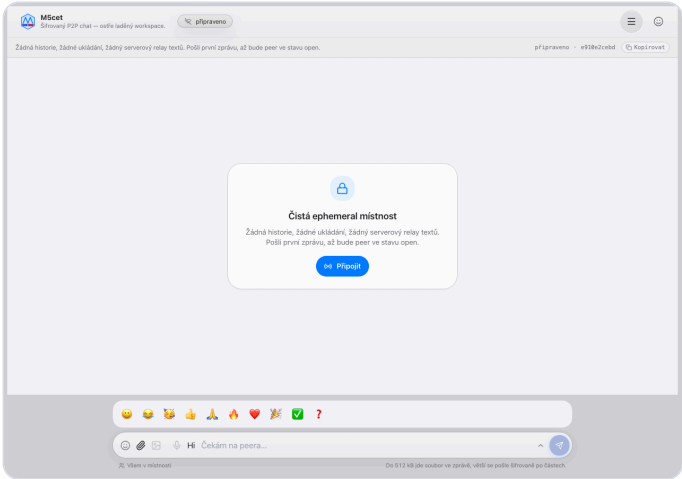
Obr. 26 — Vlastnosti tlačítka: třídy, atributy, CSS s našeptanými hodnotami `display`, událost → akce.

Strom, šablony, náhled

- Přetažení z palety do stromu (dovnitř, před, za), přesouvání, `Alt + ↑ / ↓`, duplikace, zabalení do panelu a rozbalení, skrytí, smazání, kopírovat / vložit i mezi rozvrženými, zpět / znovu, export / import.
- **Šablony:** *Save as template* uloží vybraný prvek; z palety se vloží propojeně (změna šablony se projeví všude, kde je) nebo jako kopie; šablona dostane argument jako `$arg` a upravuje se ve vlastní záložce.
- **Náhled je aplikace sama** (`layout-preview.html`): skutečné komponenty, CSS a šablony vzhledu s ukázkovými lidmi a zprávami; situace podle rozvržení, šablona, tón, jazyk, šířka (desktop / tablet / telefon). Klepnutí vybere prvek, v režimu *click tries it* náhled reaguje (emoji, příjemci, psaní). Chyby ve výrazech se ukáží u prvku.
- Rozvržení, které se od výchozího neliší, se neukládá; u uloženého builder upozorní, když aktualizace aplikace změnila výchozí stav, ze kterého vzniklo.
- Záložka *Texts & behaviour*: krátké texty (`{{placeholders}}`), includes), co zprávy ukazují, sbalování systémových zpráv a rychlé barvy komponent (dřívejší nastavení).



Obr. 28 — Plovoucí nápověda: hodnoty, akce, části, filtry, makra, výrazy, prvky.



Obr. 29 — V aplikaci: nové tlačítko v psaní (otevřít emoji) a šablona emoji tlačítka v liště.

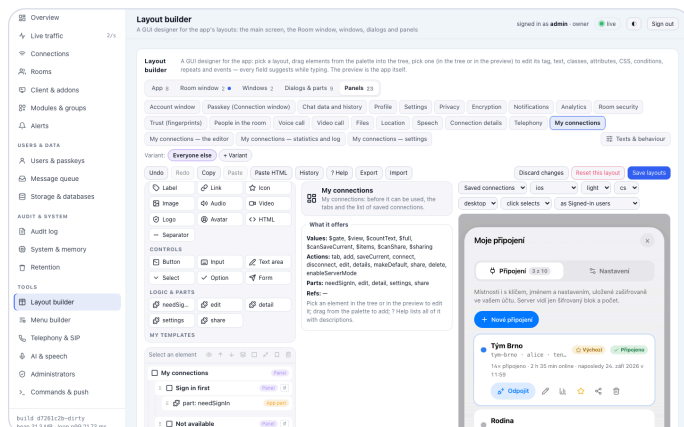
Celá aplikace v sekcích 4.13

Záložky rozvržení jsou rozdělené do sekcí; číslo u sekce říká, kolik rozvržení obsahuje, tečka, že některé z nich je vaše, a podtržení, ve které sekci je právě upravované.

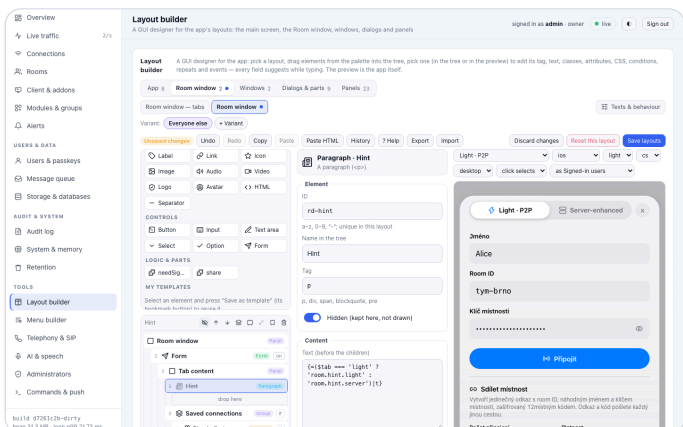
SEKCE	ROZVRŽENÍ
App (8)	lišta, okno chatu, zprávy (3), psaní, widget příjemců a jeho tlačítko — viz tabulka výše
Room window (2)	záložky Light · P2P / Server-enhanced v záhlaví; obsah okna (jméno, místnost, klíč, uložená připojení, Připojit / Odpojit, sdílení)
Windows (2)	okno panelu (záhlaví, zavření, vrstvení nad jiným oknem) a velké okno nastavení (uprostřed / šuplík vpravo)
Dialogs & parts (9)	„nejdřív se přihlaste“, odznak přihlášeného, informace o účastníkovi (bezpečnostní číslo, QR), informace o zprávě (audit, šifra), kontrola verzí, sdílení místnosti a připojení, vytvořená pozvánka, zadání kódu pozvánky
Panels (24)	asistent AI (4.14), účet, passkey (okno Připojení), historie chatu, profil, nastavení, soukromí, šifrování, oznámení, analytika, zabezpečení místnosti, důvěra (otisky), lidé v místnosti, hlasový a video hovor, soubory, poloha, řeč, podrobnosti spojení, telefonie, Moje připojení (seznam, úprava, statistiky a log, nastavení)

- Každé rozvržení má **kontrakt** (co mu komponenta dává: hodnoty, akce, živé části) a **situace náhledu** — náhled kreslí skutečnou komponentu s ukázkovými daty, v okně, ve kterém ji ukazuje aplikace. Stav, kam se komponenta dostane jen používáním (otevřený log připojení, vytvořený obnovovací kód, QR pozvánky), si náhled „doklikne“ sám; klepnutí náhledu se nepočítá jako výběr prvku.
- Převod je věrný: DOM i volané akce starých a nových komponent se porovnaly krok za krokem v desítkách situací (česky a anglicky, u části i německy). Jediný záměrný rozdíl: šest polí dostalo přístupný název (kontrola přístupnosti je našla — klíč v úpravě připojení, text SMS, odkaz pozvánky, výběr serverového hlasu, název passkey a obnovovací kód).

- Aplikace kreslí rozvržení operátora pro přihlášeného diváka (`LayoutProvider`): varianta podle jeho skupin a šablony vzhledu, jinak rozvržení operátora, jinak výchozí. Výchozí stromy se sestaví až při prvním použití (úvodní obrazovka potřebuje 8 ze 45).



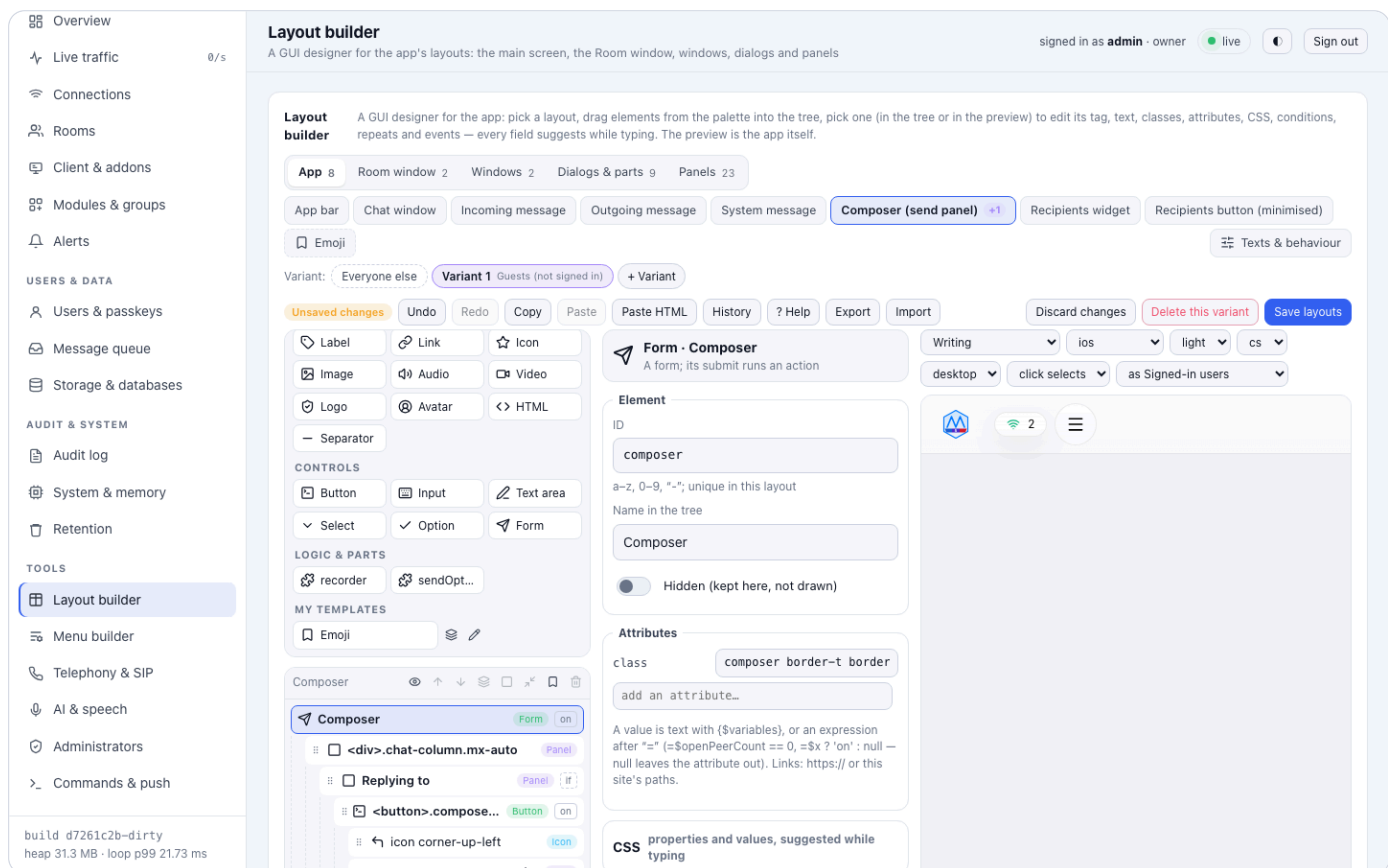
Obr. 30 — Sekce *Panels*: 23 rozvržení, v náhledu komponenta *Moje připojení s ukázkovými připojeními*.



Obr. 31 — Okno Místnost: skrytá nápověda (vybraný prvek „Hint“), v náhledu okno tak, jak ho ukáže aplikace.

Varianty pro skupiny a šablony 4.13

- Pod záložkami je řádek variant: *Everyone else* (rozvržení pro ostatní) a + *Variant*. Varianta má název, **skupiny** (vestavěné i vlastní z Modules & groups) a **šablony vzhledu**; platí, když divák patří do některé ze skupin a (jsou-li zadané) má jednu ze šablon. Bez podmínky se varianta nekreslí.
- Rozhoduje **první** vyhovující varianta (pořadí mění tlačítka *Earlier / Later*). Nejvýš 8 variant na rozvržení a 40 celkem.
- Náhled ukazuje upravovanou variantu; v náhledu lze přepnout, *jako kdo* se díváte (skupiny), a vidět, co dostane kdo jiný.

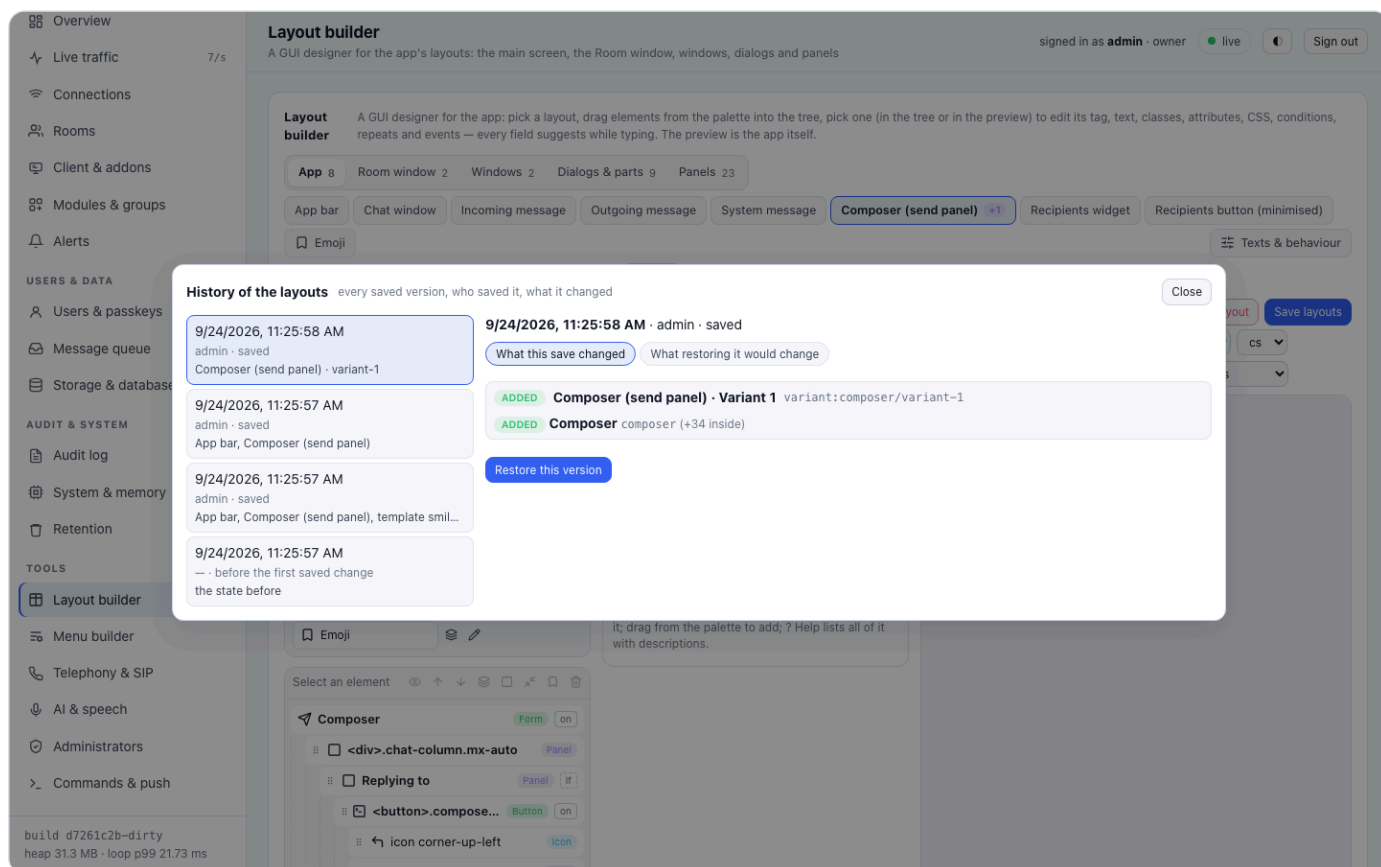


Obr. 32 – Varianta psaní pro hosty (nepřihlášené) – bez tlačítka emoji; ostatní mají výchozí.

Historie, rozdíly a návrat 4.13

- Každé uložení (i reset a návrat) je verze: kdo, kdy, co změnil (rozvržení, varianty, šablony, texty). Drží se posledních 50 verzí (nejvýš ~12 MB) v `layout-history.json` vedle `layout.json`; první uložení запиše i stav před ním.

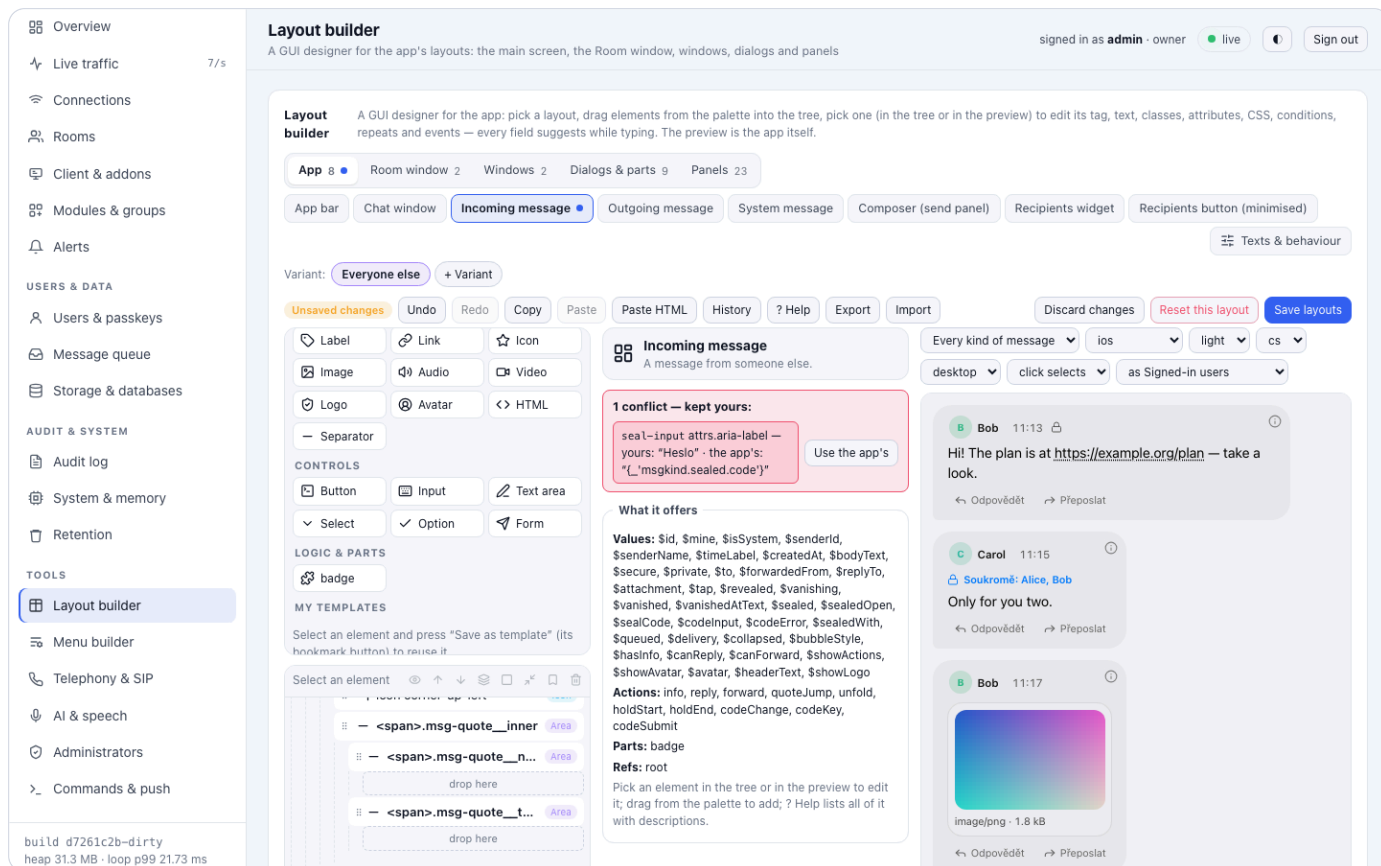
- U verze jsou **rozdíly** — co tahle změna přinesla, nebo co by návrat změnil proti dnešku: přidané, odebrané, přesunuté a změněné prvky jménem (s počtem prvků uvnitř), změněná pole.
- *Restore this version* vrátí celou konfiguraci; návrat je sám verzí, takže jde vrátit také.



Obr. 33 — Historie: kdo a kdy uložil, rozdíly verze (přidaná varianta psaní) a návrat.

Aktualizace aplikace a sloučení 4.13

- Každé vlastní rozvržení (i varianta) si pamatuje revizi výchozího stromu, ze kterého vzniklo. Všechny vydané výchozí stromy jsou v `server/layout-archive.json`, takže po aktualizaci server najde výchozí stav, ze kterého operátor vycházel, a **sloučí třicestně**: změny operátora + změny aplikace nad společným základem (podle id prvků — pole, atributy po jednom, přesuny, přidání, odebrání).
- Nekonfliktní sloučení proběhne samo při načtení a konzole ho oznámí; soubor se přepíše až při uložení. Při konfliktu (obě strany změnily totéž) zůstává operátorovo a builder nabídne *Merge with the new default* se seznamem konfliktů — u každého *Use the app's*, jinak platí vaše.



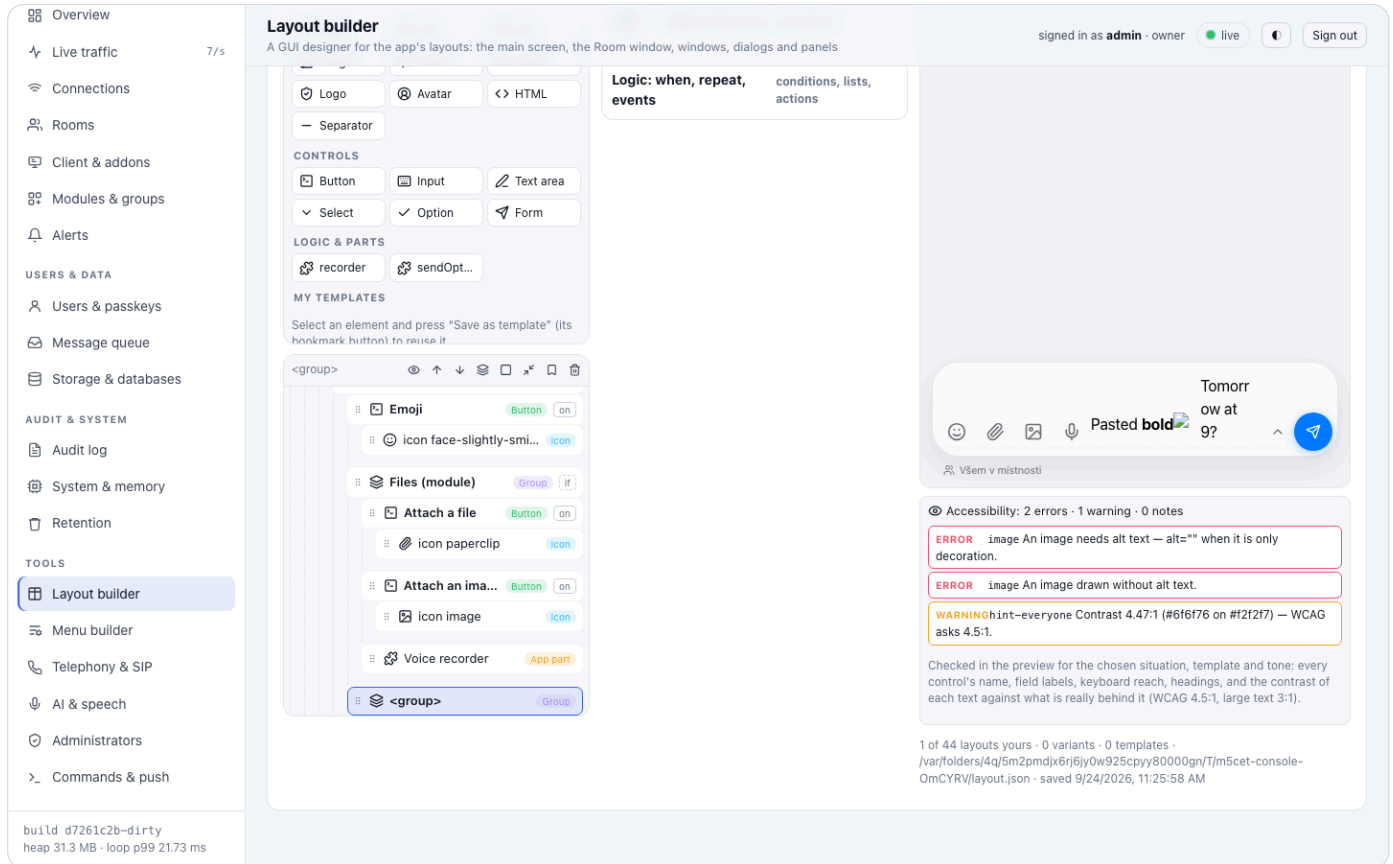
Obr. 34 — Sloučení: jeden konflikt (popisek pole kódu) — ponecháno vaše, nabídka převzít verzi aplikace.

Vložení HTML 4.13

Paste HTML převede kus HTML na prvky palety a vloží je do vybraného kontejneru: tagy na panely, oblasti, nadpisy, odkazy, seznamy, obrázky, video, tabulky, formuláře a pole, SVG ikonu lucide z katalogu na ikonu; třídy a bezpečné atributy zůstanou, `<script>`, `on*` atributy, `style`, nebezpečné adresy a neznámé tagy se vynechají a builder vypíše, co a proč (nejvýš 200 000 znaků).

Kontrola přístupnosti 4.13

- Pod náhledem je souhrn *Accessibility* a čipy u prvků ve stromu. Návrh: obrázek bez `alt`, tlačítko nebo odkaz bez názvu, pole bez popisku (placeholder popisek není), klikací prvek bez klávesnice, kladný `tabindex`, přeskočené úrovně nadpisů, duplicitní `id`, `target="_blank"` bez `rel="noopener"` / `noreferrer`.
- Vykreslený náhled: skutečný přístupný název ovládacích prvků, popisky polí a **kontrast** každého textu proti tomu, co je za ním opravdu vidět (průhledné vrstvy se skládají), podle WCAG 4.5:1 (velký text 3:1) — pro zvolenou situaci, šablonu a tón.
- Výchozí rozvržení aplikace kontrolou procházejí (hlídá to test); jedno upozornění zůstává: nápověda psaní v šabloně iOS má kontrast 4.47:1.



Obr. 35 — Po vložení HTML: dvě chyby (obrázek bez popisu) a upozornění na kontrast, u prvků ve stromu čipy.

Výkon

Šablony (`{${x}|filtr}`, podmínky, výrazy) se překládají na funkce jednou a pak se jen volají; strom rozvržení se také přeloží jednou (pro každý uzel připravené čtení dat, atributy, podmínky a opakování), neměnné části se vytvoří jednou a React je při dalším vykreslení přeskočí. Nová syntaxe: filtry nad výrazem v závorce, např. `{=( 'acc.signedInAs' |t|replace: '{name}':$userName)}`.

Bezpečnost: server i aplikace validují strom týmž modulem (`layout-tree.ts`) — jen známé prvky a tagy, žádné `on*` atributy, `style`, `srcdoc` ani `formation`, adresy jen `https:`, cesty webu, `#`, `mailto:`, `tel:` a `data:image` (při vykreslení i `data:/blob:` z dat aplikace, nikdy `javascript:`), CSS bez `url()` a výrazů, limity velikosti. Vykreslení nikdy nepoužívá `innerHTML`; prvek HTML jde přes bezpečný parser. Konzole a náhled si píší jen se stejným původem.

AI a řeč4.14

Konzole > AI & speech. Server volá AI poskytovatele za uživatele aplikace (asistent), za konzoli (zkušebna, testy) a později za funkce (framework funkcí). Každé volání prochází jednou službou (server/ai/service.ts): vybere poskytovatele a model, ověří přepínač modulu, kdo ho smí použít a limity, zavolá adaptér (i s průběžným psaním odpovědi), spočítá tokeny a cenu a zapíše volání do žurnálu. **Chat zůstává E2E šifrovaný:** do AI jde jen to, co uživatel asistentovi sám napíše, a okno asistenta to říká.

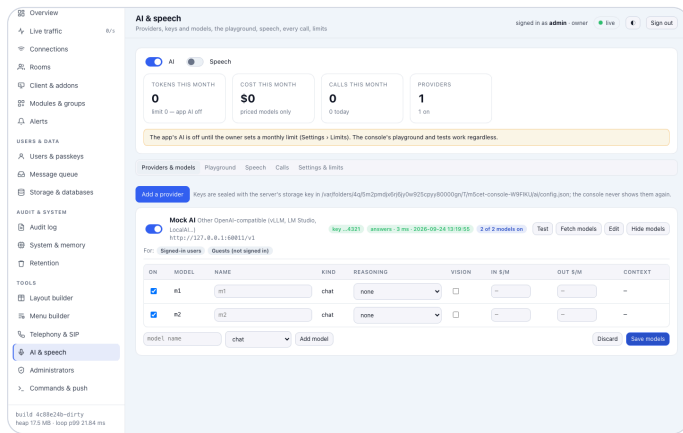
POSKYTOVATEL	PROTOKOL	KLÍČ	POZNÁMKA
Anthropic (Claude)	Messages API	ano	streamování, adaptivní uvažování s úsilím (effort) u generace Claude 5, rozpočet uvažování u starších modelů; seznam modelů i s tím, co umí
OpenAI	Chat Completions	ano	chat, řeč (tts-1 ...) a přepis (whisper-1 ...); max_completion_tokens , reasoning_effort
Open WebUI	OpenAI-kompatibilní (.../api)	ano	modely a znalosti instance Open WebUI
Perplexity	OpenAI-kompatibilní	ano	odpovědi se zdroji (citace se ukážou u odpovědi); modely sonar... se zadávají jménem
Ollama	vlastní API (/api/chat , NDJSON)	ne	lokální modely, přemýšlející modely (think)
llama.cpp server, GPT4All, jiný kompatibilní (vLLM, LM Studio, LocalAI...)	OpenAI-kompatibilní	volitelně	lokální servery
Hugging Face	Inference Providers (router.huggingface.co)	ano	chat; přepis přes hf-inference ; model může nést poskytovatele nebo politiku (...:fastest)
ElevenLabs	vlastní API	ano	řeč; hlasy účtu se načtou s modely

Poskytovatelé a klíče

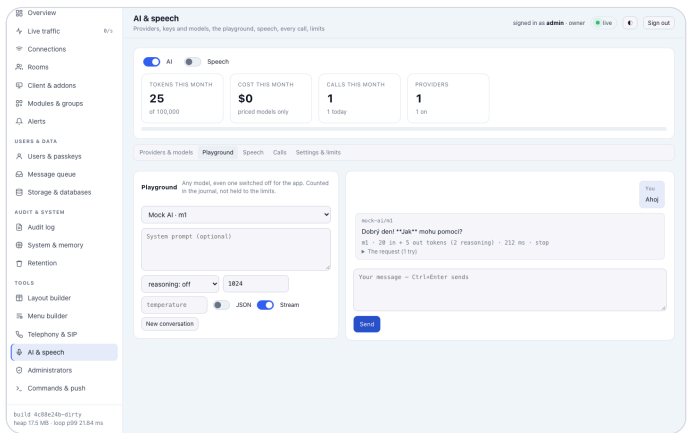
- Add a provider (vlastník): druh, jméno, adresa (prázdná = obvyklá), klíč, první model a kdo ho smí v aplikaci používat (**skupiny**: přihlášení uživatelé, hosté, vlastní skupiny z Modules & groups).
- Klíč se **zašifruje** (AES-GCM) klíčem odvozeným z master klíče úložiště serveru (STORAGE_MASTER_KEY / storage.key) a je svázaný s poskytovatelem — soubor \$DATA_DIR/ai/config.json sám klíč neprozradí a zašifrovaný klíč nejde přenést k jinému poskytovateli. Konzole klíč už nikdy neukáže (jen poslední 4 znaky); nový ho nahradí.
- **Test** zjistí, zda poskytovatel odpovídá (seznam modelů, nebo krátký dotaz) a výsledek zůstane u poskytovatele.
- Klíč z proměnné prostředí (ANTHROPIC_API_KEY , OPENAI_API_KEY , HF_API_KEY , ELEVENLABS_API_KEY , OLLAMA_URL — jako před 4.14) se ukáže jako poskytovatel „z prostředí“: klíč a adresa zůstávají v prostředí, modely a skupiny se nastavují v konzoli.

Modely

- Fetch models načte modely od poskytovatele (u Anthropicu i s tím, jak model uvažuje a kolik má kontextu); nové jsou **vypnuté**, dokud je operátor nezapne. Model jde přidat i jménem (Perplexity, jméno, které poskytovatel neuvádí).
- U modelu: zapnuto, jméno pro aplikaci, druh (chat, řeč, přepis), **uvažování** (žádné / rozpočet / adaptivní s úsilím), obrázky na vstupu a **ceny** v USD za milion tokenů (u řeči za milion znaků) — ceny zadává vlastník, protože se z nich počítá rozpočet.
- Parametr, který model odmítne (temperature u Claude 5, max_tokens u novějších modelů OpenAI, typ uvažování...), adaptér vynechá a zkusí znovu — a zkušebna ukáže, co přesně odešlo.



Obr. 36 — Poskytovatel se zašifrovaným klíčem (...4321), výsledek testu, skupiny a jeho modely.



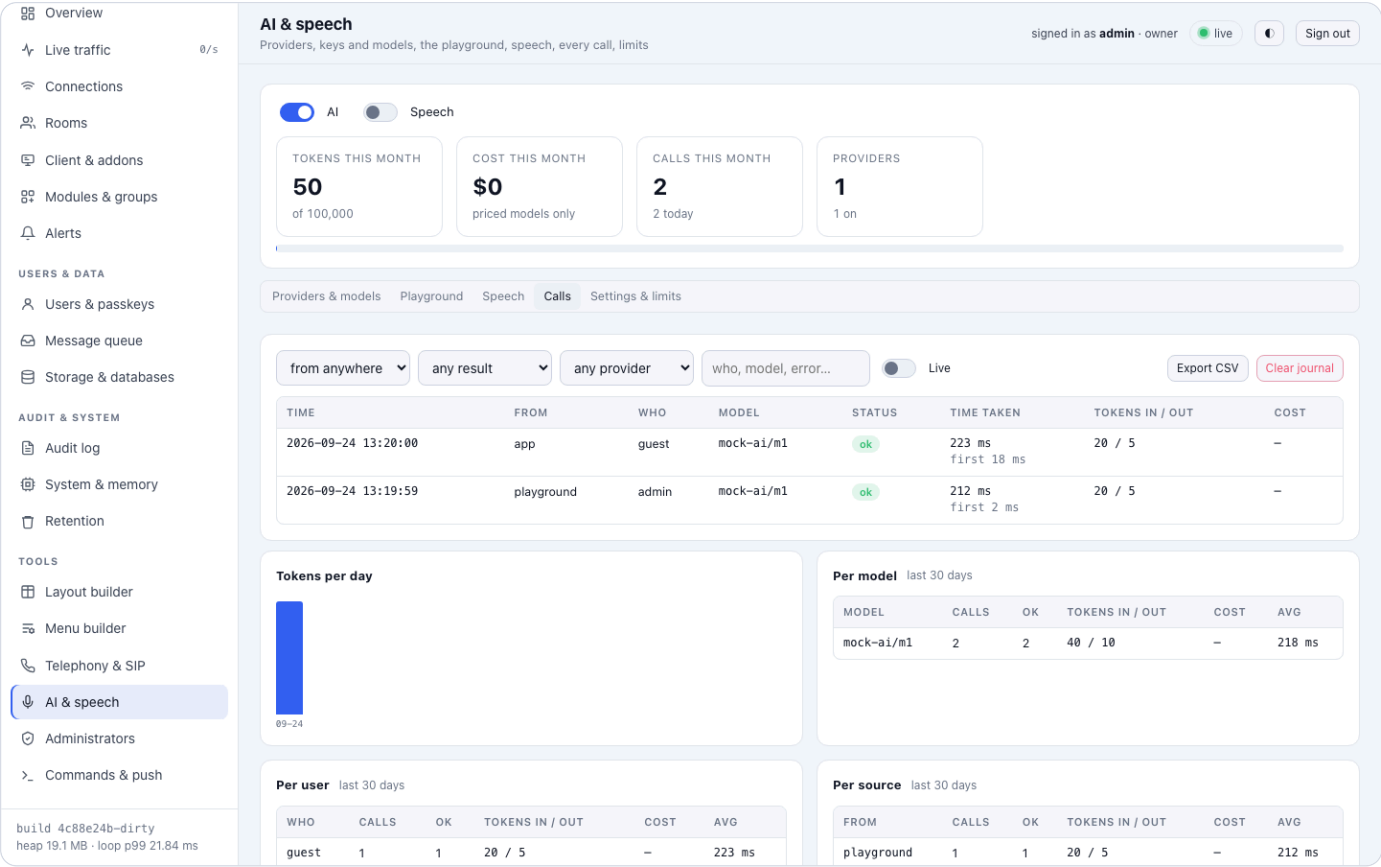
Obr. 37 — Zkušebna: odpověď psaná průběžně, tokeny, čas a požadavek, jak odešel.

Limity a náklady

- Tokeny za měsíc** pro celý server: ve výchozím stavu **0 = asistent v aplikaci je vypnutý**, dokud vlastník nenastaví limit (prázdné = bez limitu). Volitelně **USD za měsíc** (počítá se u modelů s cenou).
- Na uživatele a den: počet volání a tokeny (výchozí 200 volání). Nejdelší odpověď (výchozí 2 048 tokenů) a nejdelší odeslaná konverzace (24 000 znaků).
- Konzole (zkušebna, testy) se do limitů počítá, ale limity ji neblokují.
- Nahoře na stránce: tokeny, náklady a volání tohoto měsíce a dnes, s ukazatelem vůči limitu.

Žurnál volání

- Každé volání (aplikace, zkušebna, testy): kdy, odkud, kdo (uživatelské jméno / host / administrátor), poskytovatel a model, výsledek (ok, chyba se stavem a zprávou poskytovatele, odmítnuto limitem, zrušeno), doba i doba do prvního kousku odpovědi, tokeny (vstup, výstup, uvažování, z cache; odhad, když je poskytovatel neuvedl) a cena. V `$DATA_DIR/ai/journal.db` (SQLite, zapisuje aplikace i admin služba); bez ovladače SQLite v paměti.
- Co bylo řečeno se neukládá** — jen když vlastník zapne *content logging* na omezenou dobu (1–168 hodin, k ladění); pak obsah vidí jen vlastník a po skončení se smaže.
- Filtry, živý proud, detail volání, export CSV (buňky, které by tabulkový procesor spustil jako vzorec, dostanou apostrof), souhrny po dnech, modelech, uživateli a zdrojích; mazání (vlastník). Uchovává se 30 dní (nastavitelné).



Obr. 38 — Žurnál: volání hosta z aplikace a administrátora ze zkušebny, souhrny po dnech, modelech a uživatélích.

Zkušebna a řeč

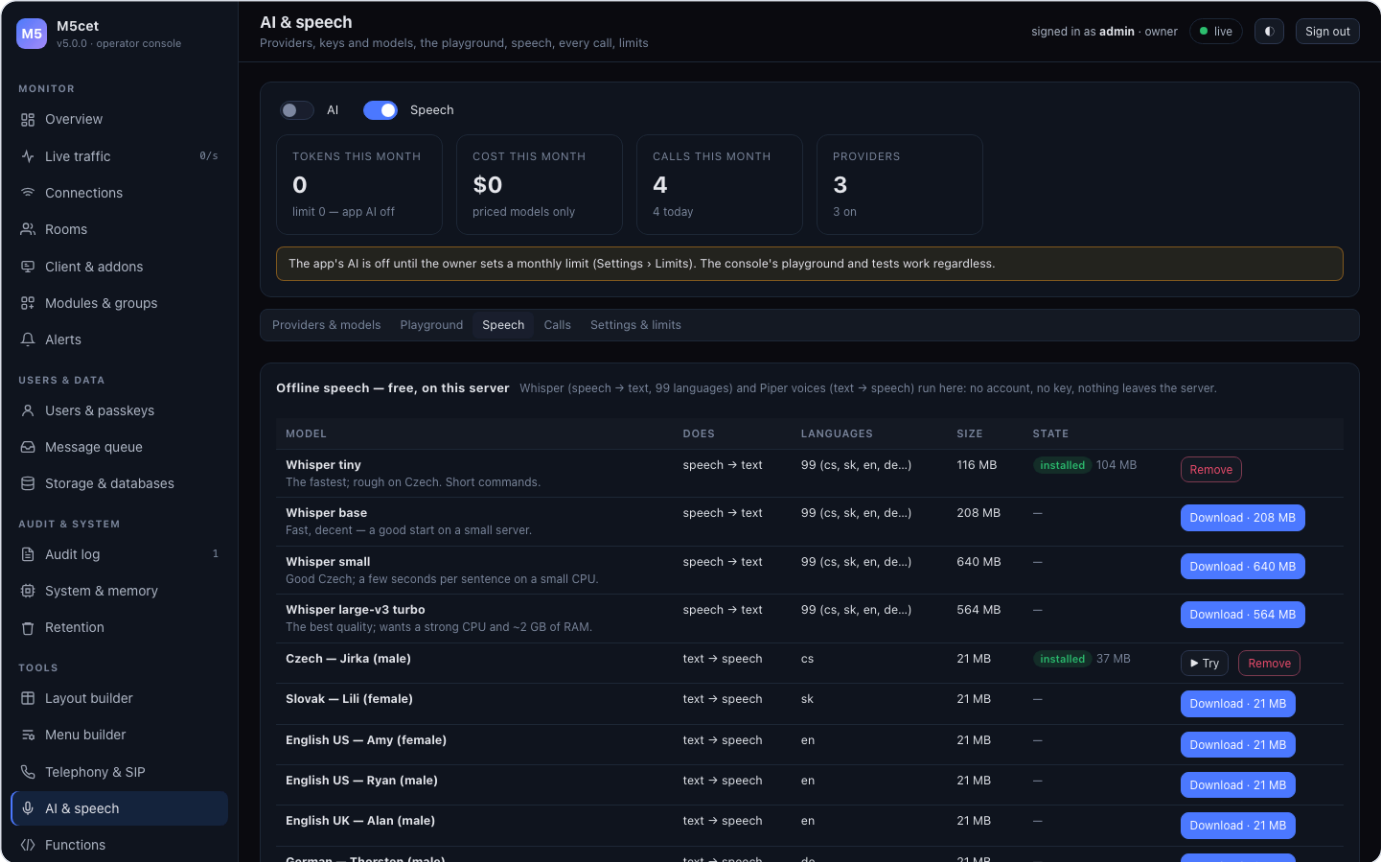
Zkušebna je konverzace s jakýmkoli modelem (i vypnutým pro aplikaci): systémový pokyn, úroveň uvažování, nejdelší odpověď, teplota, JSON, průběžné psaní; u odpovědi uvažování, zdroje, tokeny, cena, čas a *požadavek* — adresa, tělo (bez klíče), stav a počet pokusů. Karta *Speech* vyzkouší syntézu (model, hlas, text → přehrávač) a přepis (nahrávka z mikrofону nebo soubor).

Řeč zdarma a offline 5.1

Vestavěný engine **sherpa-onnx** (nativní, Apache-2.0) spouští přímo na serveru **Whisper** (řeč → text, 99 jazyků vč. češtiny) a hlasy **Piper** (text → řeč). Bez účtu a klíče, nic neodchází ven. V konzoli *AI & speech* → *Speech* je karta *Offline speech*: model se stáhne jedním klikem (průběh, velikost na disku), přidá se k poskytovateli *Built-in speech* a stane se výchozím, pokud žádný není; ► *Try* ho hned přehraje.

MODEL	STAŽENÍ	POZNÁMKA
Whisper tiny / base	116 / 208 MB	rychlé; tiny jen pro krátké povely
Whisper small	640 MB	dobrá čeština, pár sekund na větu na malém CPU
Whisper large-v3 turbo	564 MB	nejlepší kvalita, chce silnější CPU a ~2 GB RAM
Piper — čeština (Jirka), slovenština, angličtina US/UK, němčina, polština, francouzština, španělština, italština, ukrajinština	21–23 MB	jeden hlas ≈ 60 MB paměti

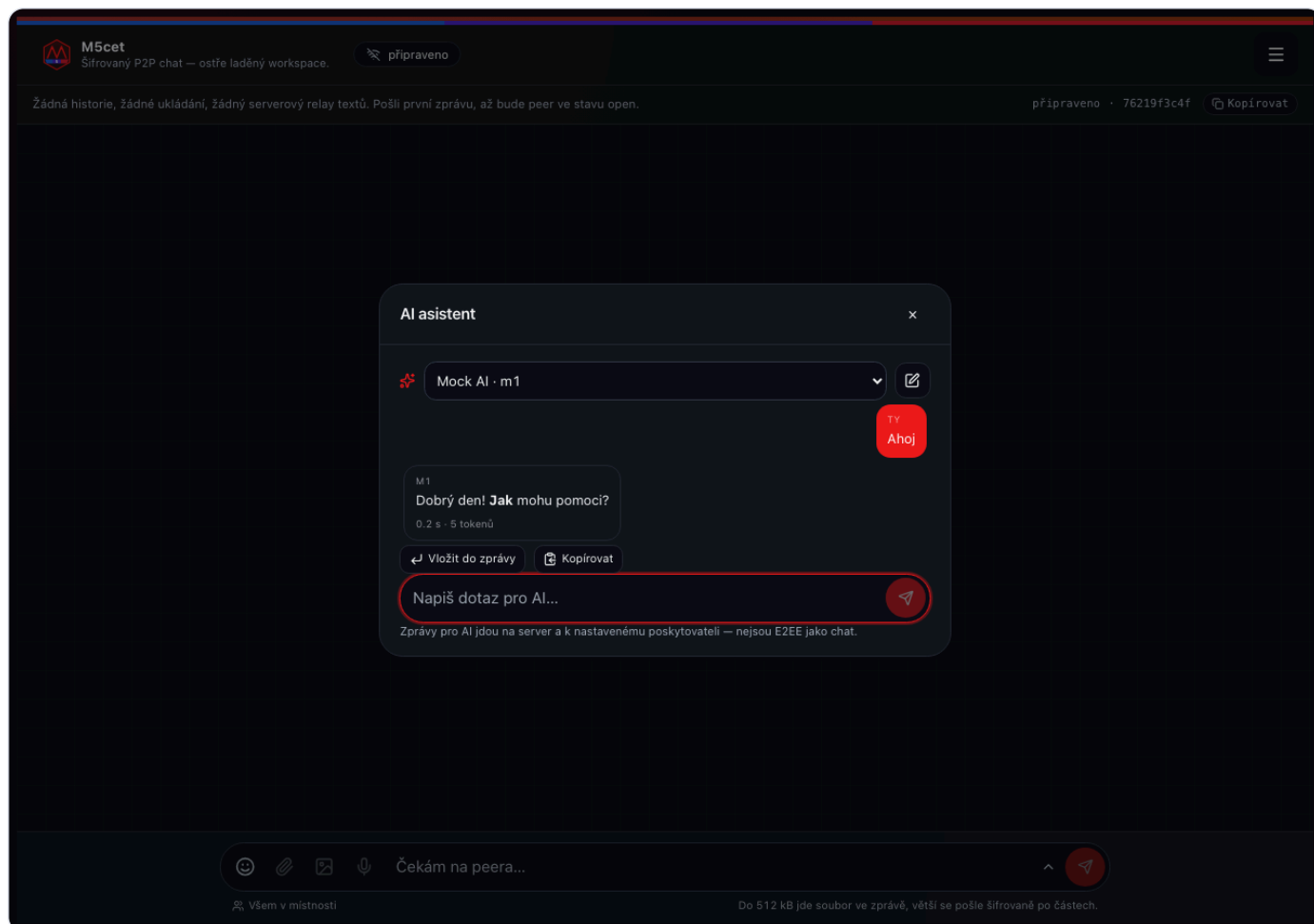
Modely leží v `$DATA_DIR/ai/speech-models`; k rozbalení je potřeba `bzip2` (instalátor i obraz Dockeru ho přidají). Vstup je WAV — aplikace i konzole nahrávku převedou v prohlížeči na 16kHz mono WAV; jiné formáty engine převede přes `ffmpeg`, pokud je na serveru. Výstup je WAV. Volání jednoho modelu jdou za sebou; počet vláken nastaví `SPEECH_THREADS` (výchozí 2). Karta *More free speech* přidá bezplatné alternativy: **Groq** (free tier, Whisper large-v3 turbo v cloudu), self-hosted **Speeches** (faster-whisper + Piper/Kokoro), **Kokoro-FastAPI** (přirozené hlasy, bez češtiny) a **whisper.cpp**. Funkce řeč používají přes `m5.ai.tts` / `m5.ai.stt` (v tvůrci uzly *Text* → *speech* a *Speech* → *text*).



Obr. 42 — AI & speech > Speech: offline modely — Whisper tiny a český hlas Jirka stažené, ostatní jedním klikem.

Asistent v aplikaci

- Menu > *AI asistent*: výběr modelu (ty, které uživatel smí), úroveň uvažování (u modelů, které uvažují), konverzace. Odpověď se píše průběžně, jako **Markdown** (nadpisy, seznamy, kód, tabulky, odkazy jen https / mailto — nic z odpovědi se nestane HTML), s uvažováním a zdroji; *Zastavit* ponechá, co přišlo; poslední odpověď jde *vložit do zprávy* nebo zkopírovat. Enter odesílá, Shift+Enter dělá nový řádek.
- Když asistent použit nejde, okno řekne proč: vypnuto, žádný model, jen pro přihlášené (s tlačítkem do okna Připojení), vlastník ještě nenastavil limit. Odmítnutí (limit, délka) je v jazyce uživatele.
- Okno je rozvržení `panel.ai` v Layout builderu (sekce Panels); vlastní pokyny asistenta nastaví operátor v konzoli (Settings > Defaults).



Obr. 39 — Asistent v aplikaci: odpověď psaná průběžně jako Markdown, zastavení, varování, že dotazy nejsou E2EE.

API

Aplikace: `GET /api/ai/status` (stav a modely pro přihlášený účet nebo hosta), `POST /api/ai/chat` (`{ model?, messages, system?, reasoning?, stream? }`); se `stream: true` Server-Sent Events (`delta`, `reasoning`, `citations`, `done`, `error`; nginx proud nebufferuje — `X-Accel-Buffering: no`, ping každých 15 s), `POST /api/ai/complete` (dřívější volání), `GET /api/speech/status`, `POST /api/speech/tts`, `POST /api/speech/stt`. Konzole: `GET /admin/ai`, `PUT /admin/ai/switches`, `POST|PUT|DELETE /admin/ai/providers[:id]`, `POST .../:id/test`, `POST .../:id/discover`, `PUT .../:id/models`, `PUT /admin/ai/defaults`, `PUT /admin/ai/limits`, `PUT /admin/ai/journal`, `POST /admin/ai/playground`, `POST /admin/ai/speech/tts|stt`, `GET /admin/ai/calls[:id]`, `GET /admin/ai/calls.csv`, `GET /admin/ai/summary`, `GET /admin/ai/stream`, `DELETE /admin/ai/calls`. Čtení: auditor; změny: operátor; klíče, adresy, ceny, limity, obsah žurnálu a mazání: vlastník.

Přechod ze 4.13

- Po aktualizaci je asistent v aplikaci **vypnutý**, **dokud vlastník nenastaví měsíční limit** (Settings & limits > Limits).
- Klíče z prostředí se ukáží jako poskytovatelé „z prostředí“ s modely z proměnných (`OPENAI_MODEL` ...); dají se nahradit poskytovatelem v konzoli a proměnnou pak odebrat.
- Hosté (nepřihlášení) asistenta používají, jen když má poskytovatel skupinu *Guests* (dřív stačilo zapnout modul).

Další etapy: framework funkcí

4.14 je první etapa návrhu `docs/functions-architecture.md`: funkce v JavaScriptu a Pythonu (QuickJS a Pyodide v izolaci WebAssembly) spouštěné přes frontu, balíčky a modely, spouštěné z chatu příkazem `/klíčové-slovo`, webhooky, IDE v konzoli a tutoriál. Tato vrstva AI je to, co funkce budou volat (`m5.ai`).

Funkce — programovatelné moduly 5.0

Konzole > *Functions*. Operátor napíše **model** v JavaScriptu nebo Pythonu, publikuje ho a kdokoli ho spustí — z chatu příkazem `/klíčové-slovo` (jako roboti v messengerech), příchozím **webhookem**, podle **plánu** nebo přes **API**. Návrh je v `docs/functions-architecture.md`; tato kapitola shrnuje hotový stav 5.0 (etapy 2–6).

Model hrozby a běh

Skripty píše **důvěryhodný operátor**, ne cizí uživatelé. Hranicí proto není obrana proti autorovi, ale **oddělený proces** na každý běh (`dist/sandbox.cjs`) a interpret ve **WebAssembly — QuickJS** pro JavaScript (ES2023, moduly, přerušení podle času, limit paměti) a **Pyodide** pro Python (CPython 3.14, standardní knihovna). Smyčka, přetečení paměti ani pád tak neohrozí hlavní službu: runner hlídá čas i paměť a proces v případě potřeby ukončí. Interpret ve WASM nevidí hostitele; ven se dostane jen přes objekt `m5` s tím, co model povolil. Server **nikdy nechte místnost** — dostane jen to, co klient u příkazu pošle, a výstup do místnosti šifruje klient.

SDK m5

Stejně rozhraní v obou jazycích (v Pythonu `snake_case`). Čisté pomocníky (`codec`, `id`, rychlé `crypto`) počítá sandbox sám; vše s I/O jde k hostiteli.

OBJEKT	CO UMÍ
<code>m5.sys</code> · <code>m5.run</code> · <code>m5.caller</code>	verze a čas, id a vstupy běhu, <code>run.progress</code> ; kdo volá, <code>caller.send</code> / <code>flash</code> / <code>openWindow</code>
<code>m5.out</code>	výstupy: <code>text</code> , <code>markdown</code> , <code>code</code> , <code>table</code> , <code>json</code> , <code>image</code> , <code>file</code> , <code>flash</code>
<code>m5.log</code>	<code>debug/info/warn/error</code> + pole, <code>trace</code> ; živě v konzoli
<code>m5.session</code> · <code>m5.cache</code>	úložiště klíč–hodnota s TTL (session = model × volající × místnost; cache s rozsahy run/session/model/global)
<code>m5.prompt</code> · <code>m5.form</code>	živá otázka za běhu (tlačítka volby / formulář) — funkce počká na odpověď
<code>m5.http</code> · <code>m5.dns</code>	HTTP bez CORS (hlavičky, JSON, limity, přesměrování) s ochranou proti SSRF (blok privátních/loopback/link-local/metadata adres, připnutí IP proti DNS rebindingu); DNS dotazy
<code>m5.crypto</code>	hash, HMAC, HKDF/PBKDF2/scrypt, AES-GCM, náhoda; JWT/JWS , X.509 , OpenPGP (OpenPGP.js), OpenSSH klíče (sshpk)
<code>m5.codes</code>	QR, Micro QR, rMQR, Aztec, Data Matrix, PDF417, MaxiCode, Han Xin, DotCode, Code 128/39/93, EAN/UPC, ITF... do SVG/PNG (bwip-js)
<code>m5.ai</code>	<code>chat</code> (text, tokeny, cena), <code>models</code> , <code>tts</code> , <code>stt</code> přes vrstvu AI jako volající „function“ (počítá se, řídí limity, zapisuje do žurnálu); <code>agent(cíl, {tools})</code> — smyčka s nástroji a potvrzováním ; rozpočet tokenů na běh
<code>m5.webhook</code>	URL vázaná na běh: <code>wait</code> (živě pokračování) nebo <code>create({durable:true})</code> pro trvalé <code>on_event</code>

Balíčky, verze a modely

Balíček je sada souborů jednoho jazyka; **koncept** se upravuje, **publikovaná verze** je neměnná a zapečetěná otiskem obsahu. Importy mezi soubory (`./util.js`) i mezi balíčky (`pkg:jméno`) s pevnou verzí. **Model** = vstupní bod + schéma vstupů (typy, validace, koerce) + limity + executory + skupiny + klíčové slovo; každé uložení je revize. Úložiště je SQLite v `$DATA_DIR/functions/functions.db`.

Chat, prompt a formuláře

Napsání `/` ukáže **našeptávač** dostupných příkazů s nápovědou argumentů; klient přeloží `klíč=hodnota` i poziční argumenty na vstupy, spustí model na serveru a výstup buď pošle do místnosti jako běžnou **šifrovanou** zprávu, nebo ukáže jen volajícímu — v obou případech jako **Markdown** v bublině. Funkce se může za běhu zeptat (`m5.prompt` s tlačítky, `m5.form`) a čekat na odpověď; čekání se nepočítá do výpočetního limitu.

Webhooky a `on_event`

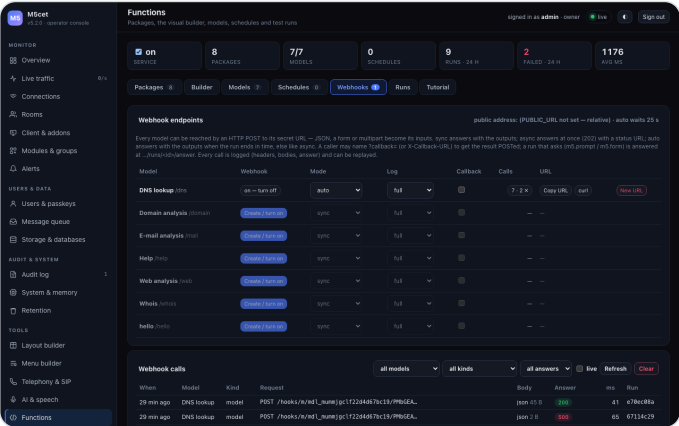
Model může být dosažitelný **příchozím webhookem** — `POST /hooks/m/:model/:token` ho spustí s tělem jako vstupy a vrátí výstupy jako JSON (mimo E2EE, pro integrace; volitelně HMAC podpis). Uvnitř běhu `m5.webhook.create()` vytvoří URL vázanou na běh; **trvalý** webhook (`durable`) po příchozím POST — i po restartu — spustí `on_event` modelu v uloženém sessionu se stavem z `m5.session`.

Webhooky: log, replay a režimy 5.2

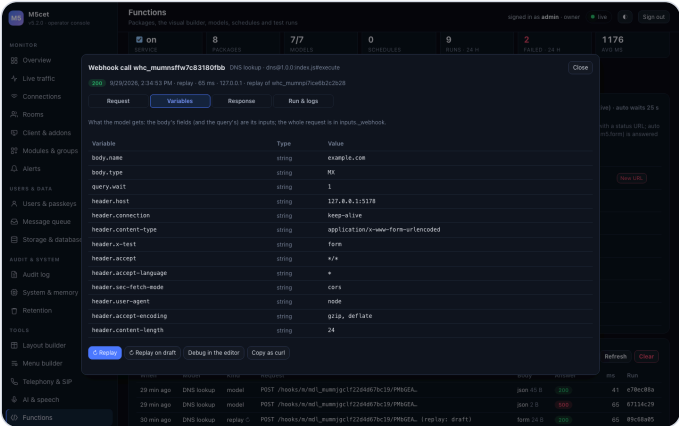
Záložka *Functions* > *Webhooks* ukazuje endpointy všech modelů: zapnutí, adresu (*Copy URL*, *curl*, *New URL* vydá nový token), **režim**, úroveň logu, zpětné volání, HMAC a statistiky. Tělo se stane vstupy: JSON (plochý, nebo `{"inputs": {...}}`), formulář, multipart i se soubory; parametry dotazu doplní zbytek.

REŽIM	ODPOVĚĎ
sync	čeká na konec běhu: 200 s výstup, 500 s chybou
async	hned 202 { runId, statusUrl }; stav a výstupy na GET .../runs/:runId, odpověď na otázku funkce POST .../runs/:runId/answer
auto	jako sync, ale nejdéle WEBHOOK_AUTO_WAIT_MS (výchozí 25 s), pak 202

?wait=1 / ?wait=0 režim přebije. Se zapnutým zpětným voláním pošle server výsledek na ?callback= nebo X-Callback-URL (jen http(s), přes ochranu SSRF). **Log volání** uloží metodu, cestu (token maskovaný), dotaz, hlavičky (Authorization, Cookie, podpisy a tokeny maskované), tělo do 256 kB, **rozparsované proměnné** (JSON, formulář, multipart, XML, text, binární data), odpověď s hlavičkami, běh s logy, dobu a chybu; **meta** vynechá těla, **off** nic. V detailu volání jsou záložky *Request*, *Variables*, *Response* a *Run & logs* a akce **Replay** (publikovaná verze), **Replay on draft** (koncept balíčku — ladění před publikací), **Debug in the editor** (vstupy volání v testovacím formuláři balíčku) a **Copy as curl**. Log se ukládí s běhy po `FUNCTIONS_RUNS_DAYS` (výchozí 30 dní).



Obr. 45 — Functions > Webhooks: endpointy modelů s režimem, logem, zpětným voláním a statistikou; pod nimi log volání.

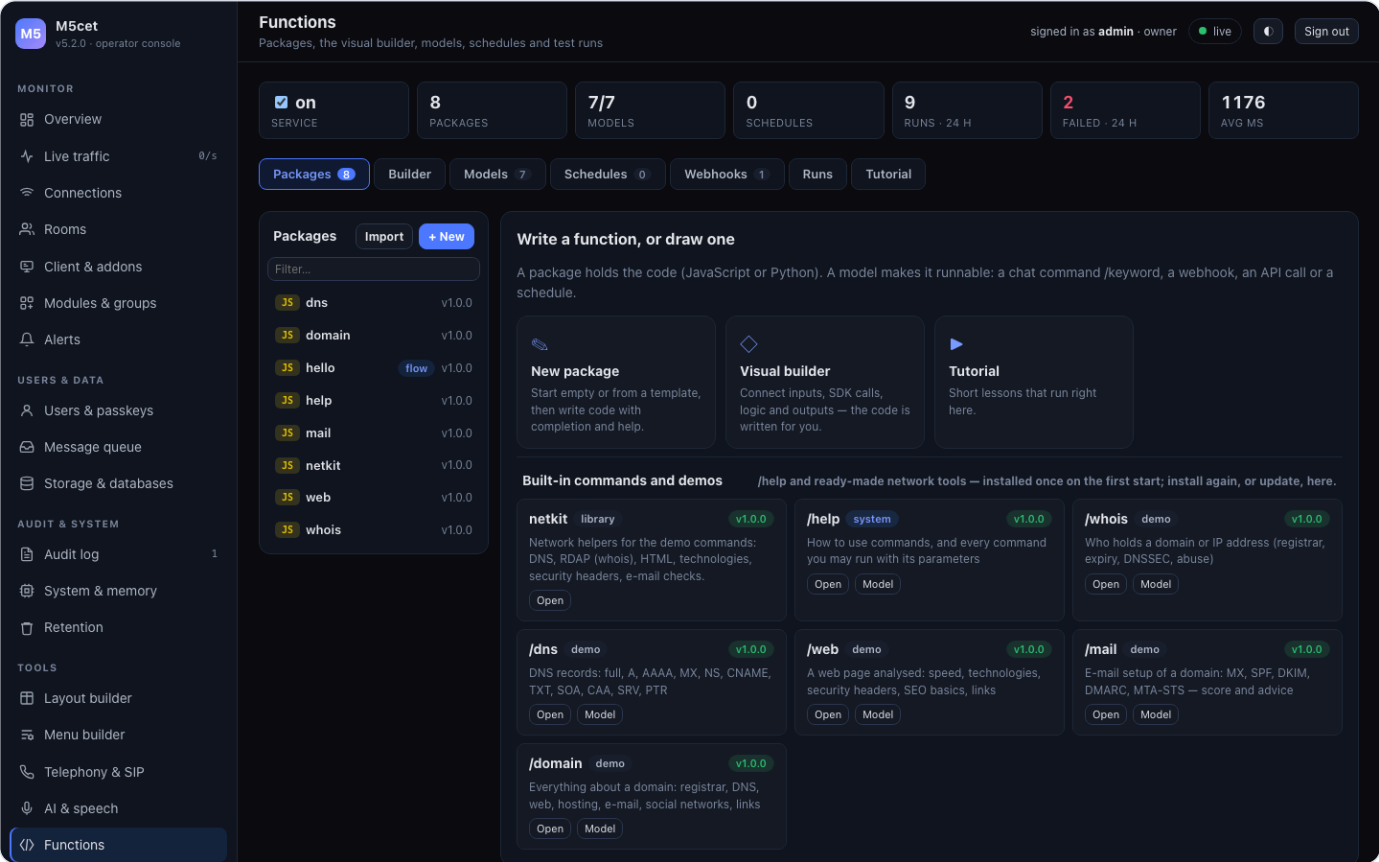


Obr. 46 — Detail volání: rozparsované proměnné těla, dotazu a hlaviček; Replay, Replay on draft, Debug in the editor, Copy as curl.

Vestavěné příkazy 5.2

Při prvním startu se nainstalují balíčky s publikovanou verzí a zapnutým modelem (`FUNCTIONS_BUILTINS=0` vypne; galerie v *Functions* > *Packages* je kdykoli doinstaluje). Jsou napsané v běžném JavaScriptu pro sandbox a slouží i jako ukázky:

PŘÍKAZ	CO DĚLÁ
/help [téma]	návod: bez tématu přehled a všechny příkazy, které smí volající spustit; /help dns parametry, typy, výchozí hodnoty, volby a příklady: syntax, webhooks, api, tags, all, ? (výběr)
/whois <doména IP>	RDAP (bootstrap IANA): registrátor, data a expirace, stav, name servery, DNSSEC, abuse kontakt
/dns <host> [type]	full (výchozí), A, AAAA, CNAME, MX, NS, TXT, SOA, CAA, SRV, PTR
/web <url>	stav, rychlost, server a technologie, bezpečnostní hlavičky, meta a SEO, robots.txt, sitemap, odkazy, sociální sítě, kontakty
/mail <doména>	MX a poskytovatel, SPF, DKIM (běžné selektory), DMARC, MTA-STX, TLS-RPT, BIMI; skóre a doporučení
/domain <doména>	celkový obraz: registrace, DNS, web, hosting, e-mail, sociální sítě a odkazy



Obr. 47 — Functions > Packages: galerie vestavěných příkazů a ukázek s verzí, otevřením balíčku a modelu.

Společné funkce jsou v knihovně `netkit` (`import ... from "pkg:netkit"`). Příkaz bez povinného vstupu ukáže formulář. Seznam příkazů dává funkcím SDK `m5.functions.list()` a `m5.functions.get(klíčové slovo)` — jen to, co smí volající spustit.

Vstupní body 5.3

Model už nemá jen jednu funkci: v *Functions > Models > Entry points* se každému typu volání přiřadí funkce z balíčku modelu (`soubor#funkce` ; konzole nabídne funkce, které verze skutečně exportuje). Každý řádek má svou sekci **Inputs** — vstupy, které se z volání přečtou, zkontrolují a otypují — a k nim pole, která typ přináší vždy:

VSTUPNÍ BOD	KDY BĚŽÍ	DOSTANE VŽDY	VSTUPY ČTE Z
execute	start: <code>/příkaz</code> v chatu, konzole, API, plán	—	argumentů příkazu
response	někdo odpověděl na zprávu modelu	<code>text</code> , <code>message</code> (<code>{ text, call }</code>), <code>event</code>	textu odpovědi (jako argumenty: <code>example.org MX</code>)
button	kliknutí na tlačítko ve zprávě modelu	<code>name</code> , <code>data</code> , <code>event</code> (<code>click</code> / <code>js</code>)	<code>data</code> tlačítka
form	odeslaný formulář ve zprávě modelu	<code>name</code> , <code>values</code> , <code>event</code>	<code>values</code>
error	jiný vstupní bod selhal (výjimka, limit, neplatný výsledek) nebo prohlížeč výsledek nezobrazil	<code>error</code> , <code>failed</code> (<code>{ call, type, parms }</code>), <code>source</code> (<code>server</code> / <code>client</code>)	—
webhook	HTTP volání na jednu z URL modelu	<code>_webhook</code> (metoda, hlavičky bez přihlašovacích údajů, dotaz, id a název webhooku)	JSON těla (<code>application/json</code> ; typy i <code>object</code> a <code>array</code>) — ostatní pole projdou, jak přišla

execute, response, button, form a error má model nejvýš jednou; **webhooků může mít víc** — každý s vlastní automaticky vytvořenou URL (kopírování v seznamu i v záložce *Webhooks*), funkcí, režimem (sync / async / auto), kontrolou HMAC, zpětným voláním a logem. Odpověď vstupního bodu **error** se ukáže místo holé chyby; jeho vlastní chyby se jen logují (nic se necyklí). V Pythonu dostane funkce pole jako pojmenované argumenty — přidejte `**inputs`. Starší modely se načtou beze změny: `entry` a `inputs` jsou jejich execute a `executors.webhook` první webhook (s původní URL).

Entry points

Which function answers which call: **execute** (the chat command, the console, the API, a schedule), **response** (a reply to the model's message), **button** and **form** (a click, a sent form), **error** (another entry point failed, or the browser could not show a result) — one of each; **webhook** — as many as you need, each with its own URL. Each has its inputs; the function also gets `m5.model` (the processing session: calls, current, last).

Package
Version

demo53 (js)
1.0.0
Open the package
Open its flow

EXECUTE
the start
index.js#execute

▼ Inputs (1) — the command's arguments

name	type	label	default	required
who	string	label	default	☐ req ↑ ×

+ Input

BUTTON
a click on its button
index.js#button
on ×

► Inputs — read from the button's data

FORM
a sent form
index.js#form
on ×

► Inputs — read from the form's values

RESPONSE
a reply to its message
index.js#response
on ×

► Inputs — read from the reply like a command's arguments

ERROR
another entry point failed
index.js#error
on ×

► Inputs

WEBHOOK
Test hook
index.js#execute
on ×

/hooks/m/mdl_mumxny7v7d8bd2adaa59/ITWhQemvd6IYjhGBam_8ovG7
Copy URL
curl

Mode
Checks
Log

sync — answers with the outputs
the secret URL
full (headers, bodies)

☐ callback (?callback= / X-Callback-URL) New URL on save

▼ Inputs (1) — the JSON body (application/json), checked and typed; other fields pass as they are
Always: `_webhook: object`

name	type	label	default	required
who	string	label	default	☒ req ↑ ×

+ Input

webhook — an inbound HTTP call
+ Entry point
functions in demo53@1.0.0: execute, button, form, response, error

Obr. 48 — Functions > Models > Entry points: execute, button, form, response, error a webhook s vlastní URL, režimem, kontrolou a logem; u každého řádku jeho vstupy.

m5.model — sezení zpracování 5.3

Všechna volání, která vzejdou z jednoho startu — odpovědi, kliknutí, formuláře, chyby — tvoří jedno **sezení zpracování** (`chain`). Funkce ho vidí v `m5.model`:

ČLEN	OBSAH
<code>m5.model.calls</code>	každé volání: <code>{ id, type, parms, result, status, err_msg, http, run, at, by }</code> ; <code>calls[0]</code> je vždy první volání (execute nebo webhook), další mají id 1, 2...
<code>m5.model.current</code> / <code>last</code> / <code>first</code>	toto volání (<code>calls[call]</code>), <code>result</code> ještě <code>null</code> , <code>status</code> „running“), předchozí a první
<code>m5.model.type</code> , <code>call</code> , <code>chain</code> , <code>endpoints</code>	typ tohoto vstupního bodu, index volání, id sezení, typy vstupních bodů modelu
<code>m5.model.session</code>	klíč–hodnota jen pro toto sezení (<code>get</code> , <code>set</code> s TTL, <code>delete</code> , <code>keys</code>)
<code>m5.model.cache</code>	cache jen pro toto sezení (<code>get</code> , <code>set</code> , <code>incr</code> , <code>delete</code>)
<code>http</code> (webhook)	URL (token maskovaný), metoda, <code>get</code> (dotaz) a <code>post</code> (tělo)

`result` jsou data, která funkce vrátila (dlouhý text a bajty zkrácené), `status` je done / failed / timed-out, `err_msg` chybová hláška. Sezení se ukládá v `$DATA_DIR/functions/functions.db` (tabulka `model_chains`), nejvýš 500 volání, a uklízí se s běhy po `FUNCTIONS_RUNS_DAYS`. Konzole ukáže sezení u každého běhu (`m5.model.calls`).

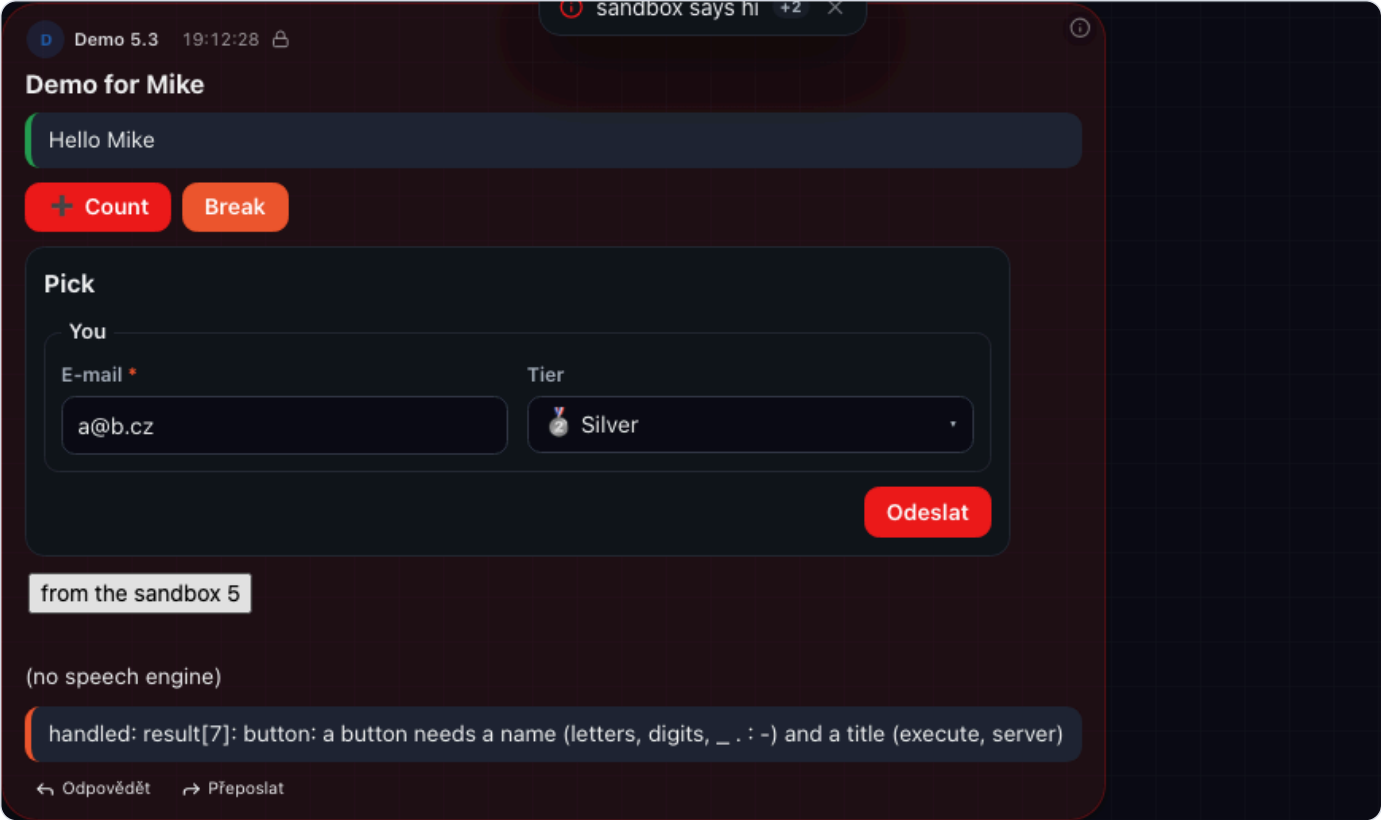
Výsledek jako seznam 5.3

Vstupní funkce vrací **jeden výstup, nebo pole výstupů**. Každá položka pole se zobrazí, přehraje nebo spustí **samostatně** — na serveru se každá zkontroluje zvlášť, v prohlížeči má každá vlastní ochranu (error boundary, try/catch): vadná položka neshodí ostatní, zaloguje se k běhu a předá se vstupnímu bodu **error**. Seznamy uvnitř seznamu (`m5.out.buttons`) se připojí. Plain objekt je výstup, když má známý `type` a klíč svého typu (`{ type: "flash", text }`); seznam obyčejných dat je jedna hodnota JSON.

```
export async function execute({ host }) {
  return [
    m5.out.markdown(`# ${host}`),
    m5.out.table(["type", "value"], rows,
    { type: "flash", text: "Hotovo", level: "success" },
    m5.out.buttons([
      { name: "type", title: "MX", data: { host, type: "MX" } }
    ]),
  ];
}
```

VÝSTUP	V APLIKACI
<code>text</code> , <code>markdown</code> , <code>code</code> , <code>table</code> , <code>json</code>	text a Markdown, blok kódu, tabulka, JSON
<code>image</code> , <code>file</code>	obrázek, soubor ke stažení (vždy jako stažení)
<code>audio</code> , <code>video</code> 5.3	přehrávač (blob); <code>autoplay</code> u nové zprávy, když to prohlížeč dovolí — jinak tlačítko přehrát
<code>flash</code>	barevné upozornění ve zprávě a notifikace aplikace (jednou, u nové zprávy)
<code>window</code>	otevře panel aplikace (files, speech, ai, phone...), pokud ho uživatel má
<code>button</code> , <code>form</code> 5.3	tlačítko a formulář — volají vstupní body button a form
<code>js</code> 5.3	kód pro prohlížeč v izolovaném rámu

Zpráva příkazu do místnosti nese výstupy end-to-end šifrovaně (`flags.fn.outputs`) a peer je před vykreslením zkontroluje stejnými pravidly (`client/src/lib/fn-outputs.ts`); média větší než se do zprávy vejde zůstanou jen volajícimu. Text zprávy je i nadále Markdown (starší aplikace, hledání, přeposlání).



Obr. 49 — Výsledek jako seznam v chatu: nadpis, notifikace, tlačítka (primary, danger s potvrzením), formulář s panelem ve dvou sloupcích a výběrem s ikonami, tlačítko z kódu v prohlížeči a odpověď vstupního bodu error na vadnou položku.

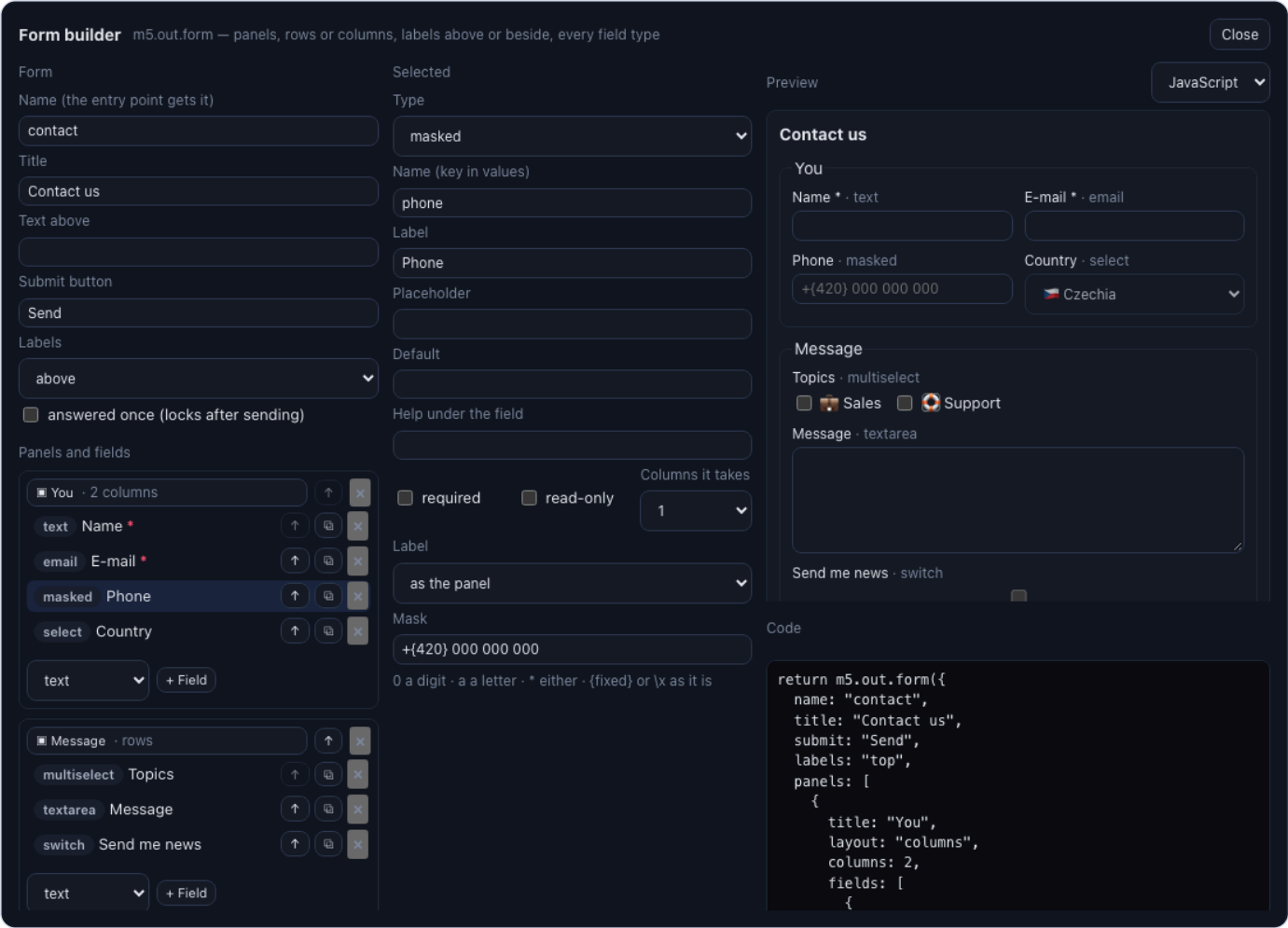
Tlačítka a formuláře 5.3

`m5.out.button({ name, title, data, css, style, icon, confirm, once })` — kliknutí spustí vstupní bod **button** se `{ name, data, event }` ve stejném sezení. `css` jsou třídy z bezpečného seznamu (primary, secondary, success, danger, warning, info, ghost, outline, link, small, large, block, round), `style` jen barvy (`color`, `background`, `border`), `icon` emoji, `confirm` se zeptá, `once` jen jednou. Tlačítka vedle sebe tvoří řadu; `m5.out.buttons([...])` jich udělá víc.

`m5.out.form({ name, title, text, submit, labels, columns, fields | panels, once })` — odeslání spustí vstupní bod **form** se `{ name, values }`. Aplikace hodnoty nejdřív zkontroluje (povinné, e-mail, čísla a meze, maska, vzor), server je pak znovu podle vstupů vstupního bodu.

ČÁST	MOŽNOSTI
Pole <code>type</code>	text, textarea, number, range, tel, email, url, password, date, time, datetime, month, color, masked , select , multiselect , radio, checkbox, switch, hidden, static (Markdown), separator
Pole	<code>name</code> , <code>label</code> , <code>placeholder</code> , <code>default</code> , <code>required</code> , <code>help</code> , <code>min / max / step</code> , <code>pattern</code> , <code>mask</code> (<code>0</code> číslice, <code>a</code> písmeno, <code>*</code> obojí, <code>{+420}</code> nebo <code>\x</code> doslova), <code>rows</code> , <code>span</code> (kolik sloupců zabere), <code>labels</code> (<code>top</code> / <code>left</code>), <code>readonly</code>
Volby	<code>options: [{ value, label, icon }]</code> — select a multiselect jsou vlastní výběry s ikonami a ovládáním z klávesnice
Panely	<code>{ title, text, layout: "rows" "columns", columns, labels, collapsed, fields }</code> — skupiny v řádcích nebo vedle sebe, popisky nad nebo vedle, sbalitelné

Form builder (editor kódu > *Tools*, a uzel *Form* ve vizuálním tvůrci) skládá formulář myši: formulář, panely a pole se všemi vlastnostmi, volby s ikonami, živý náhled a vygenerovaný kód v JavaScriptu nebo Pythonu včetně funkce vstupního bodu. Podobně **Button** a **Browser JavaScript**.



Obr. 50 — Form builder: vlastnosti formuláře, panely a pole (vybrané pole s maskou), živý náhled a kód m5.out.form s funkcí form.

Kód v prohlížeči 5.3

m5.out.js(code, args, { title, height, hidden }) a await m5.browser.run(code, args) spustí JavaScript v prohlížeči toho, kdo zprávu vidí — v rámu /fn-sandbox.html s sandbox="allow-scripts" bez allow-same-origin a s vlastní CSP (sandbox allow-scripts, i když se stránka otevře samostatně): neprůhledný origin, takže kód nesáhne na aplikaci, její úložiště, relaci ani šifrovací klíče. S aplikací mluví jen zprávami s limitem počtu:

V KÓDU	CO UDĚLÁ
m5.args, m5.root, m5.tone, m5.lang	data od funkce, prvek k vykreslení, světlý / tmavý tón, jazyk
m5.send(name, data)	vstupní bod button modelu (event.type = js)
m5.submit(name, values)	vstupní bod form
m5.flash(text, level), m5.play(bytes url)	notifikace aplikace, zvuk
m5.log(...), m5.error(e)	řádek do logu běhu; chyba (a neošetřené výjimky) → vstupní bod error
m5.resize(px?)	výška rámu (jinak podle obsahu)

hidden: true dělá z kódu **efekt** — proběhne jednou, když je zpráva nová; viditelný kód je malý widget ve zprávě a běží, kdykoli se zpráva zobrazí. m5.browser.play, flash a open pošlou zvuk, notifikaci a panel hned během běhu.

Odpověď na zprávu modelu 5.3

Odpověď (Odpověď) na zprávu příkazu, který má vstupní bod **response**, se pošle modelu: v místnosti jde zároveň jako běžná zpráva, u výsledku jen pro volajícího zůstane u něj. Odpověď modelu přijde jako nová zpráva ve stejném sezení. Tlačítka, formuláře a odpovědi procházejí stejnou kontrolou přístupu jako příkaz (modul Functions, skupiny modelu); starou zprávu, jejíž sezení už vypršelo, server odmítne (410) — stačí spustit příkaz znovu. API: POST /api/functions/event { keyword | model, chain, call, type: response | button | form | error | log, ... } (s stream: true jako /api/functions/run).

V konzoli jsou tlačítka a formuláře ve výsledcích zkušebních běhů živé — i u konceptu balíčku, lekce tutoriálu a toku ve vizuálním tvůrci (volají stejnojmenné funkce) — a pod výsledkem je pole *Reply* pro vstupní bod response.

Plány (cron) a API

Model může běžet podle **cronu** (`*/15 * * * *`, `@daily`, názvy měsíců/dnů, časové pásmo); scheduler v hlavní službě spustí splatné plány jednou za minutu. Model lze též volat programově přes `POST /api/functions/call/:id` s **API tokenem**.

ID, tutoriál a přenos

Konzole má **editor** souborů balíčku s panelem SDK, uložení konceptu, publikaci, **zkušební běh** s výstupy a živými logy; správu modelů (klíčové slovo, vstupní bod, schéma vstupů, viditelnost, skupiny, webhook a API), **plány, běhy, interaktivní tutoriál** (lekce „vložit ukázkou → spustit → kontrola“) a **šablony** i **export/import** balíčku jako přenosný bundle `.m5pkg`.

Editor kódu 5.1

Soubory balíčku se píší v editoru **CodeMirror 6**, který je přibalený do konzole (její CSP povoluje skripty jen z vlastní adresy; `npm run build` ho sestaví do `admin-ui/public/vendor/m5-editor.js`).

- **Barvy:** klíčová slova, definice a volání funkcí, proměnné, vlastnosti, řetězce, čísla, komentáře; volání SDK `m5.*` zvlášť — ve světlém i tmavém vzhledu konzole.
- **Našeptávač** (píše se sám, nebo Ctrl+Mezerník): `m5.` nabídne objekty SDK, `m5.ai.` jejich metody s podpisem a nápovědou. Vybraná metoda se vloží jako šablona s poli pro povinné argumenty (Tab mezi nimi) a u asynchronních volání se doplní `await`. Po `inputs.` nabídne vstupy modelů, které balíček spouští.
- **Šablony:** napište `execute`, `httpjson`, `table`, `ask`, `aichat`, `speak`, `counter`... (JS i Python) a vyberte je ze seznamu.
- **Nápověda:** najetím myši na `m5....` se ukáže podpis a popis, při psaní volání podpis nad řádkem. Panel vpravo má záložky *SDK*, *Templates* a *Examples* (lekce tutoriálu) s hledáním; kliknutí vloží.
- **Kontrola:** syntaktické chyby podtržené a v okraji; asynchronní volání SDK bez `await` dostane varování s opravou jedním klikem.
- Hledání a nahrazení (Ctrl+F), skládání bloků, více kurzorů, formátování (Shift+Alt+F), Ctrl+S uloží koncept, Ctrl+Enter ho spustí; stavový řádek s pozicí a stavem uložení.

The screenshot shows the M5cet editor interface. On the left is a sidebar with navigation links: Overview, Live traffic, Connections, Rooms, Client & addons, Modules & groups, Alerts, Users & data, Users & passkeys, Message queue, Storage & databases, Audit & system, Audit log, System & memory, Retention, Tools, Layout builder, Menu builder, Telephony & SIP, AI & speech, and Functions (selected). The main area is titled 'Functions' and shows a list of packages, models, schedules, runs, and tutorial. The 'Packages' tab is active, showing a package named 'hello' with version 'v1.0.0'. The code editor shows a JavaScript file named 'index.js' with the following code:

```
// Generated by the M5cet visual builder from flow.m5flow.json.
// Edit the flow in Functions -> Builder (changes here are replaced
// or use "Eject to code" there to keep working on this file by hand
4
5 const __str = (v) => (v === undefined || v === null ? "" : typeof v
6
7 export async function execute(inputs = {}) {
8   // n1 - Input "name"
9   let n_n1;
10  const r = m5.ai.
11  n_n1 = inputs["n1"]
12  // agent agent(goal, { tools, maxSteps, approve })
13  chat chat({ messages, model?, system? })
14  models models()
15  stt stt({ audio, mime })
16  tts tts({ text, voice? })
17
18  // n4 - Change text
19  let n_n4;
20  n_n4 = __str(n_n2).toUpperCase();
21  // n3 - Send Markdown
22  let n_n3;
23  n_n3 = await m5.caller.send(m5.out.markdown(__str(n_n2)));
24  return null;
25 }
```

On the right side, there is a panel with tabs for 'SDK', 'Templates', and 'Examples'. The 'SDK' tab is active, showing a search bar and a list of SDK methods: `m5.sys`, `m5.run`, `m5.caller`, `m5.log`, `m5.out`, `m5.session`, `m5.cache`, and `m5.codec`. Each method has a brief description.

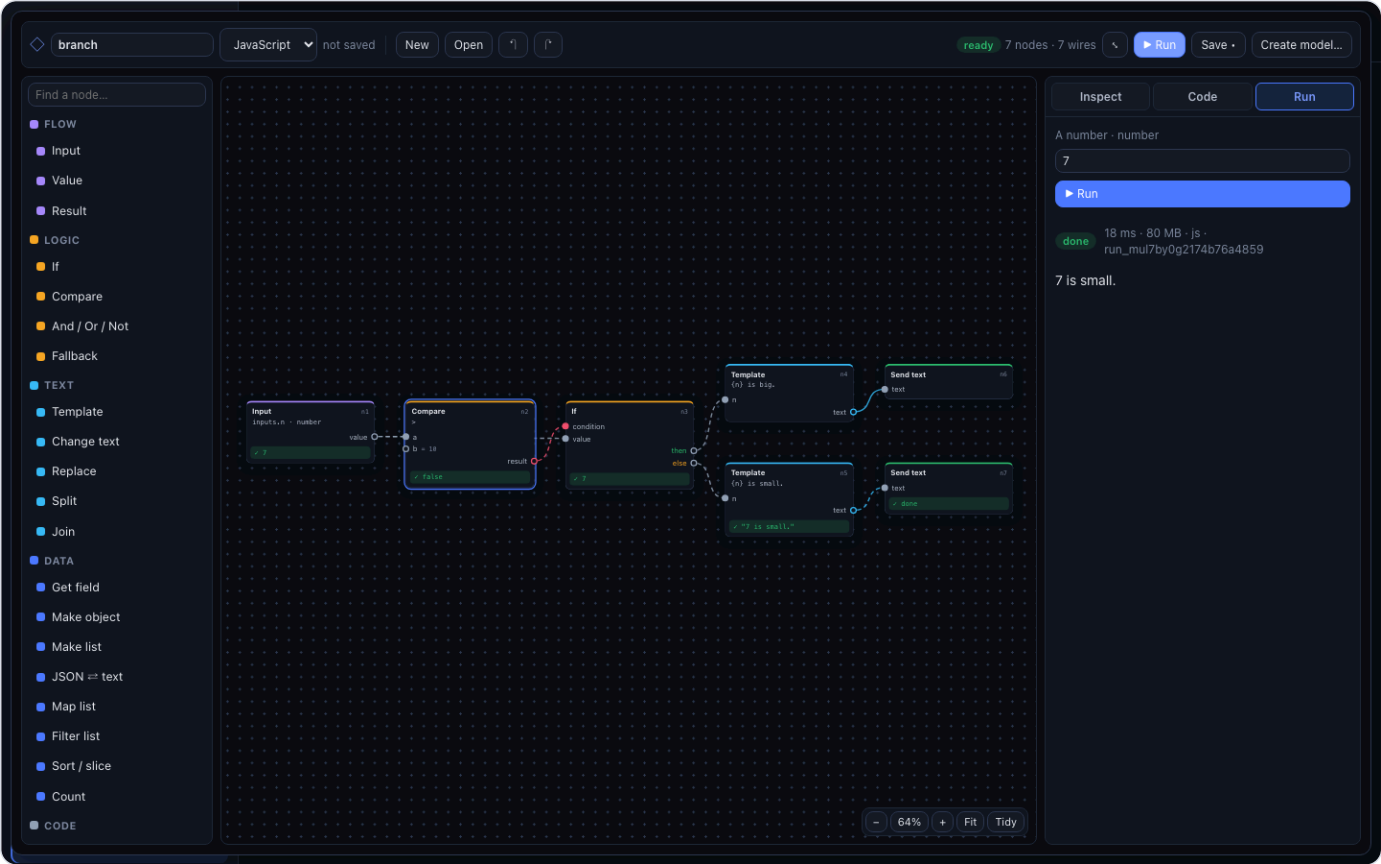
Obr. 40 — Editor: po `m5.ai.` našeptávač nabídne metody s podpisem a nápovědou; vpravo panel SDK · Templates · Examples.

Vizuální tvůrce 5.1

Záložka *Builder* skládá funkci **bez psaní kódu**: uzly z palety (přetažením nebo kliknutím, dvojklikem na plátno, puštěním drátu do prázdna — uzel se rovnou zapojí) se propojí dráty z výstupů do vstupů. Vstup bez drátu vezme hodnotu napsanou v inspektoru.

SKUPINA	UZLY
Flow	Input (vstup modelu: jméno, typ, výchozí, povinný), Value, Result (návratová hodnota pro API a webhook)
Logic	If (větve <i>then/else</i> — co na nich visí, běží jen v té větvi), Compare, And/Or/Not, Fallback
Text · Data · Code	Template s <code>{zástupci}</code> , změna textu, nahrazení, rozdělení a spojení; pole podle cesty, objekt, seznam, JSON, mapování, filtr, řazení, počet; výpočty, výraz a blok kódu v jazyce toku
Output · Ask	poslat text, Markdown, kód, JSON, tabulku (sloupce z dat), obrázek, soubor, flash; zeptat se volajícího (volby nebo text)
Store · Web	session a cache (čtení, zápis s TTL, počítadlo); HTTP (JSON se rozparsuje), DNS
AI & speech · Crypto & codes · Utility	AI chat, text → řeč, řeč → text; hash, HMAC, kódování, nové id, QR a čárové kódy; log, čas, volající, čekání

Tok se přeloží (`server/functions/flow.ts`, stejný kód na serveru i v prohlížeči) do čitelného **JavaScriptu** nebo **Pythonu** — záložka *Code* ho ukazuje živě. **Run** spustí tok s formulářem podle uzlů Input a každý uzel na plátně ukáže svou **hodnotu** nebo chybu (dráty z hotových uzlů „tečou“). Kontrola hlídá chybějící vstupy, dva dráty do jednoho vstupu i kruhy a vede k uzlu. **Save** uloží tok do konceptu balíčku (`flow.m5flow.json` + vygenerovaný `index.js` / `index.py`), **Create model...** balíček publikuje a vytvoří nebo aktualizuje model se vstupy podle uzlů Input (klíčové slovo pro chat, viditelnost). *Eject to code* převede tok na obyčejný balíček. Zpět/znovu, duplikace, *Tidy* (srovnání), přiblížení (Ctrl + kolečko), *Fit*, celá obrazovka (⌘, Esc).



Obr. 41 — Vizuální tvůrce na celé obrazovce: příklad *Branch* po běhu s hodnotou 7 — každý uzel ukazuje svou hodnotu, větve „big“ neběžela.

Víc funkcí v jednom toku 5.3 — záložky nad plátnem: *execute* a další vstupní body (*response*, *button*, *form*, *error*, *webhook*, nebo vlastní jméno), každá svůj graf; vše se přeloží do jednoho souboru a *Create model...* z nich udělá vstupní body modelu (se vstupy z uzlů Input). Nové uzly: *Play sound*, *Button* a *Form* (editované v builderech), *Browser code*, *Entry point data* (text odpovědi, jméno a data tlačítka, hodnoty formuláře, chyba) a *Model session* (`m5.model`); *Remember* umí i „conversation“ (`m5.model.session` / `cache`).

The screenshot displays the M5cet Functions Builder interface. The top bar shows the user is signed in as 'admin' (owner) and the system is 'live'. The main dashboard provides a summary of system metrics: 9 packages, 8/8 models, 0 schedules, 28 runs (24 H), 3 failed runs (24 H), and an average of 384 ms. The 'Builder' tab is active, showing a visual flowchart for a 'button' function. The flowchart includes nodes for 'Entry point data', 'Counter', 'Template', and 'Send Markdown', connected by wires. The right-hand 'Inspect' panel provides details for the 'counter-buttons' function, including its summary, description, and inputs.

Obr. 51 — Vizuální tvůrce se záložkami funkcí: funkce button čte data tlačítka a sezení modelu.

Živé běhy z konzole 5.1

Zkušební běhy (balíček, model, lekce, tok) jsou živé: `POST /admin/functions/run` s `live: true` vrátí hned `runId`, `GET /admin/functions/runs/:id/live` (SSE) přehraje události od začátku — logy, výstupy, průběh, otázky, výsledek. Když se funkce zeptá (`m5.prompt`, `m5.form`), konzole ukáže kartu s volbami nebo formulářem a odpověď pošle přes `POST /admin/functions/runs/:id/answer`. Záložka *Runs* filtruje podle stavu a modelu, umí se obnovovat sama a detail běhu ukáže vstupy, výstupy, logy i hodnoty uzlů toku.

Zapíná se přepínačem modulu *Functions* v konzoli nebo proměnnou `ENABLE_FUNCTIONS=1`. Volitelný trusted runtime (npm/PyPI v nsjail) se po zvoleném modelu hrozby nepoužívá.

Monitoring a audit

Záznam provozu

Monitor drží v paměti posledních 5 000 záznamů, počítadla po třídách, sekundové řady za 10 minut a registr živých spojení. Obsah zpráv nemá — server ho nikdy nevidí.

TrafficRecord							SSE event „traffic“	
id	at ms	channel ws http	direction in out	cls třída	type „join“, „GET /api/...“		bytes	
conn	peerId	accountId	roomHash SHA-256, 16 hex	ip /24 · /48	status ok · error · dropped · HTTP		durationMs	

Třídy: `signaling`, `presence`, `relay`, `storage`, `file-proxy`, `heartbeat`, `account`, `admin`, `push`, `api`, `static`, `error`, `other`. Heartbeaty se do živého streamu neposílají (jejich počet je v ticku).

Auditní žurnál

KATEGORIE	PŘÍKLADY UDÁLOSTÍ	ZAPNUTO
<code>security</code>	<code>ws.origin-refused</code> , <code>ws.kicked</code> , <code>auth.invalid-token</code> , <code>join.peer-id-taken</code> , <code>relay.refused</code> , <code>http.rate-limited</code>	vždy
<code>account</code>	<code>account.register</code> , <code>account.sign-in</code> , <code>account.deleted</code> , <code>session.revoked</code> , <code>relay.away</code> , <code>relay.back</code> , <code>relay.push</code>	vždy
<code>communication</code>	<code>room.join</code> , <code>room.leave</code> , <code>relay.stored</code> , <code>relay.delivered</code>	jen na vyžádání (<code>AUDIT_COMMUNICATION=1</code> nebo přepínač)
<code>storage</code>	<code>queue.sweep</code> , zakládání, povýšení a mazání databází	vždy
<code>admin</code>	<code>admin.command</code> , <code>admin.user.delete</code> , <code>admin.audit.export</code> , <code>admin.connection.close</code>	vždy
<code>network</code>	<code>ws.refused</code> , <code>ws.closed</code> , <code>ws.heartbeat-timeout</code> , <code>ws.slow-consumer</code>	vždy
<code>system</code>	<code>server.start</code> , <code>server.stop</code> , <code>http.error</code> , <code>config.rp-id-unset</code> , <code>queue.migrated</code>	vždy

Úrovně `debug < info < notice < warn < error`. Žurnál drží 3 000 posledních záznamů v paměti; s úložištěm jde každý záznam i do tabulky `audit` globální databáze, kde je volné pole `detail` **zapečetěné** master klíčem a svázané s kategorií, událostí a časem.

Audit komunikace zaznamenává, kdo koho kdy a jak velkou zprávou oslovil — tedy přesně metadata, která by šifrovaný chat neměl sbírat. Proto je vypnutý a jeho zapnutí (i vypnutí) se samo zapíše do auditu. Zapínejte jen s právním důvodem.

Živý stream

```
GET /api/admin/live?streams=traffic,audit,tick
Authorization: Bearer <ADMIN_API_TOKEN>
Accept: text/event-stream

event: traffic
data: {"id":812,"at":1790000000000,"channel":"ws","direction":"in","cls":"relay","type":"relay","bytes":1843,"conn":"c-1a82ee...","status":"ok"}

event: audit
data: {"id":77,"category":"security","level":"warn","event":"ws.kicked","status":"rate limits exceeded"}

event: tick    (každé 2 s)
data: {"counts":{"..."},"rate":{"framesIn":12,"framesOut":15,"http":1,...},"system":{"rss":..., "loopP99":1.4}}
```

Konzoli, která nestíhá (víc než 1 MB ve frontě odpovědi), server záznamy zahazuje — nikdy kvůli ní nehromadí paměť.

Neměnný audit 3.1

Každý záznam v databázi nese `hash = SHA-256(kanonický JSON [hash předchozího, čas, kategorie, úroveň, událost, ..., detail])`. Každých 500 záznamů, před promazáním starých a na požádání se hlava řetězu **podepíše** (Ed25519, `m5cet/audit-checkpoint/1|poslední_id|hash|čas`; klíč `audit-signing.key` vedle master klíče). Ověření (konzole > Audit > *Ověřit integritu*, `GET /api/admin/audit/verify`) přepočítá řetěz a řekne, kde byl záznam změněn, odstraněn nebo kontrolní bod přepodepsán. Záznamy z doby před 3.1 hlásí jako „před řetězem“.

Metriky a alerty 3.1

`GET /metrics` (token `METRICS_TOKEN` nebo administrátor) vrací Prometheus text: spojení, místnosti, členy, účty a relace, rámce a bajty po třídách, frontu, heap, event loop, CPU, audit, zálohy, cluster (`m5cet_cluster_*`) a počet spuštěných alertů.

PRAVIDLO	VÝCHOZÍ PRÁH	PROMĚNNÁ
bezpečnostní varování za minutu	20	<code>ALERT_SECURITY_WARNINGS</code>
chyby za minutu	120	<code>ALERT_HTTP_ERRORS</code>
event loop p99	200 ms	<code>ALERT_EVENT_LOOP</code>
heap z limitu	85 %	<code>ALERT_HEAP</code>
mrtvé položky fronty	1	<code>ALERT_DEAD_LETTERS</code>
selhání zálohy, integrity nebo řetězu auditu	1	<code>ALERT_STORAGE</code>

Alert se spustí a vyřeší jednou (žádné opakování každou minutu); s `ALERT_WEBHOOK_URL` ho server pošle jako JSON (Slack/Teams/vlastní přijímač). Prahy jde měnit v konzoli (panel *Alerty*, uloží se do `alert-rules.json`).

Zálohy a integrita 3.1

Záloha je adresář `$BACKUP_DIR/m5cet-<čas>/`: globální databáze zkopírovaná online backup API SQLite (za běhu konzistentní), kopie šifrovaných databází uživatelů (zůstávají šifrované), index účtů se zapečetěnými trezory, administrátoři (jen hashe tokenů) a `manifest.json` s velikostí a SHA-256 každého souboru. **Master klíč do zálohy nepatří** — uložte ho zvlášť. S `BACKUP_DIR` běží zálohy samy (`BACKUP_INTERVAL_HOURS`, výchozí 24; drží se `BACKUP_KEEP`, výchozí 7) a s nimi `PRAGMA quick_check`; jinak na požádání z konzole (Úložiště). Tamtéž optimalizace a `VACUUM` (jen vlastník).

API reference

Veřejné REST API hlavní služby

METODA A CESTA	POPIS	AUTORIZACE
GET /api/health	živost, verze, build, <code>protocol</code> , počet místností	—
POST /api/functions/event 5.3	odpověď na zprávu modelu, tlačítko, formulář, chyba nebo log z prohlížeče — vstupní bod ve stejném sezení (<code>stream: true</code> jako <code>/run</code>); <code>410 expired</code> , když sezení skončilo	jako příkaz (účet / host)
GET /fn-sandbox.html 5.3	izolovaný rám pro kód funkcí v prohlížeči (vlastní CSP, <code>sandbox allow-scripts</code>)	—
GET /api/modules	manifest volitelných modulů	—
GET /api/turn	ICE server; s <code>TURN_SECRET</code> krátkodobé přístupy a <code>expiresAt</code>	—
GET /metrics 3.1	Prometheus metriky	<code>METRICS_TOKEN</code> nebo administrátor
GET /api/layout	vzhled a šablony od operátora	—
GET /api/client-config 3.2	konfigurace klienta od správce: politika uložených připojení a další servery, povolené a výchozí šablony (Client & addons), od 4.0 moduly a skupiny (bez členů); nic tajného	—
POST /api/share/create redeem revoke	pozvánky (split-key, kód 12 číslic); limit 30 / 40 za 15 min. Zapečetěný obsah: místnost, klíč, jméno a od 3.3 volitelně <code>server</code> (<code>wss://</code>)	—
GET /api/push/status , POST /api/push/subscribe test	Web Push pro zařízení bez účtu (jen známé push služby)	test sobě bez tokenu, všem s admin tokenem
/api/account/*	registrace a přihlášení passkey, <code>me</code> , trezor, push, odhlášení, smazání. Od 4.0 vrací <code>register/options</code> vygenerované <code>username</code> , <code>register/verify</code> chce <code>keyProof</code> a <code>signin/verify</code> vydá zamčený token (<code>locked:true</code>); neznámý passkey → <code>404 unknown-passkey</code>	Bearer token relace
POST /api/account/unlock 4.0	<code>{ keyProof }</code> — odemkne relaci, když důkaz globálního klíče sedí; jinak zruší token a vrátí <code>403 wrong-key</code>	zamčený token relace
GET /api/menu-config 4.0	menu aplikace z Menu builderu	—
POST /api/clear-site-data 4.0	odpověď <code>Clear-Site-Data: "cache"</code> — krok opravy v kontrole verzí	—
GET /version-manifest.json 4.0	verze, build, protokol, service worker, knihovny a soubory sestavení (SHA-256)	—
GET PUT /api/account/vault — část <code>connections</code> 3.2	zapečetěná uložená připojení: <code>{ connections: { ct, count } }</code> , <code>ct</code> ≤ 1,5 MB	token relace
POST /api/account/passkeys/options verify , GET passkeys/:id/wrapped , DELETE passkeys/:id 3.1	další passkey účtu, jeho zapečetěný kořen, odebrání (poslední nejde)	token relace
PUT DELETE /api/account/recovery , POST recovery/start finish 3.1	obnovovací kód: nastavit, zrušit, obnovit účet (10 pokusů / h)	token relace / důkaz z kódu
GET /api/account/sessions , DELETE sessions/:id , PUT identity 3.1	zařízení účtu a jejich odhlášení; veřejný klíč účtu (Ed25519)	token relace
/api/storage/*	úložiště (i přes WebSocket rámec <code>storage</code>)	relace účtu nebo <code>X-M5cet-Session</code>

METODA A CESTA	POPIS	AUTORIZACE
GET POST /api/settings , POST /api/audit/purge , /api/analytics/consent	nastavení a souhlasy vázané na deviceId	znalost deviceId
/api/ai/* , /api/speech/* , /api/telephony/*	volitelné moduly (AI a Speech zapíná konzole nebo ENABLE_AI , ENABLE_SPEECH ; ENABLE_TELEPHONY)	limity na IP
GET /api/events/recent , GET /api/transfers/stats	metadata událostí, statistika proxy	admin token (od 3.0)

Operátorské API (/api/admin/*)

Všechny cesty vyžadují Authorization: Bearer <token administrátora> (nebo relaci konzole po přihlášení passkey). Čtení (GET) stačí auditorovi, změny operátorovi, správa administrátorů vlastníkovi — viz Role.

METODA A CESTA	POPIS
GET overview	verze, proces, počty, provoz, fronta, úložiště, audit, zdraví
GET live	SSE: traffic, audit, tick (výběr ?streams=)
GET traffic	záznamy provozu; filtry cls, channel, direction, conn, peer, account, room, type, since, before, errors=1, limit
GET traffic/rates?seconds=	sekundové řady (max 600 s)
GET connections, POST connections/:id/close	živá spojení, odpojení
GET rooms	místnosti (hash), členové, nepřítomní
GET users, GET users/:id	účty; detail s passkeys, frontou, auditem, žurnálem, provozem
POST users/:id/signout	odvolat všechny relace (okamžitě i na socketech)
DELETE users/:id?confirm=:id	smazat účet se vším (ID se musí zopakovat)
GET queue, GET queue/dead, POST queue/:id/revive	fronta po účtech, dead-letter, obnova položky
GET system, GET db	proces a paměť (+ historie 30 min); databáze
GET audit (?source=db), GET audit/export?format=csv json	žurnál z paměti nebo databáze; export (CSV s ochranou proti vzorcům)
PUT audit/settings	{ communication: bool }
GET POST commands	povolené příkazy, čekající, log; zařazení příkazu {kind, deviceId, payload?}
GET push, POST push/test	stav VAPID a odběratelé; testovací push
GET events	metadata událostí (LOG_EVENTS=1)
GET retention, POST retention/run	politika uchovávání, ruční běh
GET storage, GET storage/logs	index a statistika úložiště, tabulka logů
GET whoami 3.1	kdo jsem a s jakou rolí
GET POST admins, PATCH DELETE admins/:name, POST DELETE admins/:name/tokens[:id] 3.1	administrátoři, role, pojmenované tokeny (vlastník)
POST me/passkeys/options verify, POST auth/passkey/options verify signout 3.1	vlastní passkey administrátora; přihlášení do konzole passkey
GET audit/verify, POST audit/checkpoint 3.1	ověření hash řetězu a podpisů; podepsat hlavu teď
GET POST backups, POST db/integrity, POST db/optimize 3.1	zálohy, kontrola integrity, optimalizace ({vacuum:true} jen vlastník)
GET metrics, GET alerts, PUT alerts/rules/:id 3.1	metriky; pravidla, spuštěné alerty, historie; práh a zapnutí pravidla
GET PUT client-config 3.2	konfigurace klienta (+ výchozí hodnoty, cesta k souboru, využití, katalog šablon a od 4.0 modulů, stav serverových modulů features); PUT operátor
GET PUT menu-config 4.0	menu aplikace (+ výchozí, cesta k souboru, katalog: ikony, panely, funkce, speciální tlačítka, písma, barvy, moduly, nápověda šablon); PUT operátor, tělo do 1 MB
GET modules/state, PUT modules/switches 5.2	přepínače služeb ({ai?, speech?, functions?}) a hlavní skupiny modulů (chybějící založí)

METODA A CESTA	POPIS
POST menu-config/render 4.0	náhled builderu: { config, lang } → přeložené texty a HTML bloky jako bezpečné stromy (ukázkové hodnoty); nic nemění

```
# export auditu bezpečnostních událostí od úrovně warn do CSV
curl -H "Authorization: Bearer $ADMIN_API_TOKEN" \
  "https://chat.example.com/api/admin/audit/export?format=csv&category=security&minLevel=warn" -o audit.csv

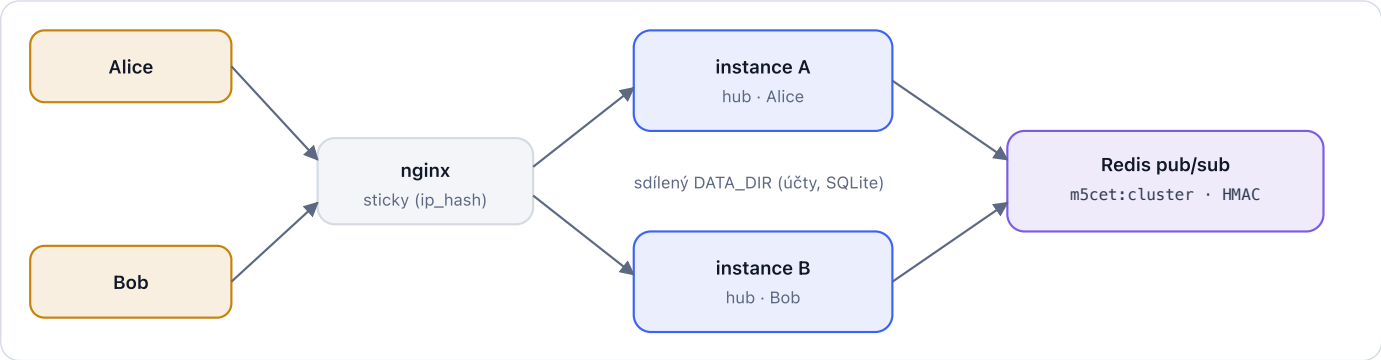
# příkaz pro zařízení (doručí se hned, je-li připojené)
curl -X POST -H "Authorization: Bearer $ADMIN_API_TOKEN" -H "Content-Type: application/json" \
  -d '{"kind":"show-notification","deviceId":"dev-3f9c0a","payload":{"title":"Údržba","body":"Ve 22:00 restart"}}' \
  https://chat.example.com/api/admin/commands
```

Admin služba (/admin/*)

Layout builder: GET /admin/layout (konfigurace, výchozí hodnoty a katalog — paleta, atributy, CSS, třídy aplikace, ikony, rozvržení s výchozími stromy a kontrakty), PUT /admin/layout (operátor, tělo do 4 MB; zapíše verzi do historie), POST /admin/layout/reset ; od 4.13 historie GET /admin/layout/history , GET /admin/layout/history/:id , rozdíly GET /admin/layout/history/:id/diff?against=current|previous , návrat POST /admin/layout/history/:id/restore , sloučení s výchozím POST /admin/layout/merge a převod HTML POST /admin/layout/from-html ; náhled /layout-preview.html (frame-ancestors 'self') a /assets/* z dist/public . Telefonie a SIP (/admin/telephony/* včetně sip/trunks), AI a řeč (/admin/ai* — viz AI a řeč > API; GET /admin/plugins a PUT /admin/plugins/switches zůstávají jako dřívější adresy, /admin/logs/stream). Od 5.2 přístupy: GET /admin/access/me (k čemu mám přístup), GET /admin/access/log (filtry module, decision, subject, kind, text, since, limit, days), POST /admin/access/explain ; webhooky funkcí: GET /admin/functions/webhooks , PUT /admin/functions/webhooks/:modelId , GET|DELETE /admin/functions/webhooks/calls , GET /admin/functions/webhooks/calls/:id , POST /admin/functions/webhooks/calls/:id/replay ; vestavěné balíčky GET /admin/functions/builtins , POST /admin/functions/builtins/:name/install . Od 5.3 vstupní body: GET /admin/functions/exports?package=&version= (exportované funkce po souborech a pole typů), GET /admin/functions/chains?model=|id= (sezení zpracování), POST /admin/functions/event (klik, formulář, odpověď nebo chyba ve výsledku zkušebního běhu; live), PUT /admin/functions/webhooks/:modelId s endpoint / create a DELETE /admin/functions/webhooks/:modelId:endpoint . Nástroje konzole kontroluje modul (403 module-denied). Dřívější /admin/commands/* , /admin/clients , /admin/test/push , /admin/metrics a /admin/logs/recent dnes preposílají na hlavní službu — ve 2.x četly prázdnou kopii stavu v jiném procesu, takže příkazy nikdy nedorazily.

Více instancí 3.1

S `REDIS_URL` se místnost rozprostře přes všechny instance za load balancerem: lidé na různých instancích se vidí, spojí a pošlou si soubor přes relay.



Obr. 9 — Dvě instance, jedna místnost: členství, signály a relay jdou přes Redis.

ZPRÁVA	KDY	CO UDĚLÁ DRUHÁ INSTANCE
<code>join</code> / <code>leave</code> / <code>update</code>	člen vstoupí, odejde, přihlásí se	<code>peer-joined</code> / <code>peer-left</code> / <code>peer-updated</code> svým socketům; reference účtů si spočítá sama
<code>hello</code> → <code>state</code>	start instance, obnovené spojení s Redisem	pošle seznam svých členů po místnostech
<code>beat</code> / <code>bye</code>	každých 5 s / při zastavení	instance tichá 20 s (nebo s <code>bye</code>) zmizí i s členy
<code>signal</code>	SDP/ICE pro člena jinde	doručí adresátovi
<code>room</code> / <code>chunk</code>	rámec relay pro celou místnost	rozešle; binární kus binárně, starším klientům JSON
<code>to-sender</code>	<code>proxy-need</code> / <code>proxy-cancel</code> od příjemce	předá odesílateli přenosu, je-li u ní
<code>evict</code>	tentýž klient se vrátil (resume) na jinou instanci	starému socketu pošle <code>replaced</code>
<code>revoke</code> / <code>away</code>	odhlášení, smazání účtu / nepřítomnost	ukončí relace na svých socketech / srovná stav relay

Podpis zpráv

S `CLUSTER_SECRET` nese každá zpráva HMAC-SHA256; nepodepsané a padělané se zahodí (počítadlo `dropped`). Kdo smí do Redisu publikovat, bez tajemství místnosti neovládne. Bez tajemství server při startu varuje.

Sdílená data

Instance sdílejí `DATA_DIR` (jeden hostitel nebo sdílený svazek): SQLite úložiště a fronta jsou bezpečné pro více procesů (WAL), účty jsou JSON — instance si před každou operací načte, co zapsala jiná, a zapisuje hned.

Redis klient

Vlastní RESP klient bez závislostí: `redis://` i `rediss://` (TLS), `AUTH`, `SELECT`, reconnect s backoffem, znovu-přihlášení k odběru; zprávy odeslané při výpadku se po návratu odešlou (max. 1 000).

Omezení

Limity spojení a rámců jsou na instanci. Dvě instance měnící tentýž účet ve stejné milisekundě mohou jednu změnu ztratit — sticky sessions to prakticky vyloučí.

```
# nginx: dvě instance, sticky podle adresy klienta
upstream m5cet_app {
    ip_hash;
    server 127.0.0.1:5000;
    server 127.0.0.1:5001;
}
# každá instance
REDIS_URL=redis://:heslo@127.0.0.1:6379/0 CLUSTER_SECRET=$(openssl rand -base64 32) DATA_DIR=/var/lib/m5cet
```

Stav clusteru ukazuje konzole (Přehled › Zdraví) a metriky `m5cet_cluster_*`.

Nasazení

Proměnné prostředí

PROMĚNNÁ	VÝCHOZÍ	VÝZNAM
PORT , HOST	5000, 0.0.0.0	kde poslouchá hlavní služba
DATA_DIR	./m5cet	data: účty, úložiště, signaling.secret (systemd: StateDirectory)
TRUST_PROXY	loopback	kterým proxy věřit s X-Forwarded-For (limity, brána spojení, audit)
ALLOWED_ORIGINS 3.0	—	další originy směřující otevřít /ws (čárkami); vlastní doména je povolena vždy
ADMIN_API_TOKEN	—	token konzole a admin API (bez něj vrací 503)
ENABLE_ADMIN , ADMIN_PORT , ADMIN_BIND	—, 5050, 127.0.0.1	admin služba
MAIN_URL 3.0	http://127.0.0.1:\$PORT	kam admin služba přeposílá /api/admin/*
AUDIT_COMMUNICATION 3.0	vypnuto	1 = audit komunikace od startu
LOG_HTTP 3.0	ve vývoji ano	1 = řádek na každý API požadavek i v produkci (bez těl)
PUSH_ENDPOINT_HOSTS 3.0	—	další povolené push služby (.example.com = i subdomény)
VAPID_PUBLIC_KEY , VAPID_PRIVATE_KEY , VAPID_SUBJECT	—	Web Push
WEBAUTHN_RP_ID , WEBAUTHN_ORIGINS , PUBLIC_BASE_URL	—	doména passkeys a povolené originy; bez nich produkce varuje
STORAGE_DIR , STORAGE_MASTER_KEY , STORAGE_KEY_FILE	—	úložiště a jeho master klíč
ACCOUNTS_DIR , RELAY_RETENTION_DAYS	—, 30	účty; jak dlouho čeká položka fronty bez vlastního TTL
TURN_SERVER_URL , TURN_USERNAME , TURN_CREDENTIAL	—	TURN pro striktní NAT (statické přístupy — server varuje)
TURN_SECRET , TURN_TTL_SECONDS , STUN_URLS 3.1	—, 3600, Google STUN	krátkodobé TURN přístupy (coturn use-auth-secret); vlastní STUN servery
ADMIN_TOKENS 3.1	—	další administrátoři: jméno:role:token,... (role owner , operator , auditor)
METRICS_TOKEN 3.1	—	token pro /metrics (Prometheus); bez něj jen administrátor
ALERT_WEBHOOK_URL , ALERT_<PRAVIDLO> 3.1	—	kam posílat alerty; prahy pravidel (tabulka)
BACKUP_DIR , BACKUP_INTERVAL_HOURS , BACKUP_KEEP 3.1	—, 24, 7	plánované zálohy a kontroly integrity
REDIS_URL , CLUSTER_SECRET , CLUSTER_CHANNEL 3.1	—, —, m5cet:cluster	více instancí (cluster); tajemství pro podpis zpráv
WS_CONNECTS_PER_MINUTE , WS_CONNECTIONS_PER_CLIENT , WS_CONNECTIONS_TOTAL 3.1	30, 20, 5000	brána WebSocket spojení (na adresu klienta / celkem)

PROMĚNNÁ	VÝCHOZÍ	VÝZNAM
<code>CLIENT_CONFIG_FILE</code> 3.2	<code>\$DATA_DIR/client-config.json</code>	konfigurace klienta z konzole (Client & addons, moduly a skupiny)
<code>MENU_CONFIG_FILE</code> 4.0	<code>\$DATA_DIR/menu-config.json</code>	menu aplikace z Menu builderu
<code>ACCESS_LOG_DAYS</code> , <code>ACCESS_LOG</code> 5.2	30, zapnuto	log přístupů: kolik dní souborů držet; <code>ACCESS_LOG=0</code> vypne
<code>WEBHOOK_AUTO_WAIT_MS</code> 5.2	25000	jak dlouho čeká webhook v režimu <code>auto</code> , než odpoví <code>202</code>
<code>FUNCTIONS_RUNS_DAYS</code> , <code>FUNCTIONS_BUILTINS</code> 5.2	30, zapnuto	úklid běhů a logu webhooků; <code>FUNCTIONS_BUILTINS=0</code> nenainstaluje vestavěné příkazy
<code>LOG_EVENTS</code>	vypnuto	metadata událostí (<code>/api/admin/events</code>)
<code>ENABLE_AI</code> , <code>ENABLE_SPEECH</code> , <code>ENABLE_FUNCTIONS</code> , <code>ENABLE_TELEPHONY</code> + klíče poskytovatelů	vypnuto	volitelné moduly; AI a Speech lze zapnout i v konzoli (<code>\$DATA_DIR/plugins.json</code> , <code>PLUGINS_SETTINGS_FILE</code>) — nastavená proměnná má přednost. Od 4.14 se poskytovatelé a klíče nastavují v konzoli (<code>\$DATA_DIR/ai/</code> ; <code>AI_DATA_DIR</code> , <code>AI_CONFIG_FILE</code> , <code>AI_JOURNAL_FILE</code>); klíč v proměnné prostředí dál funguje jako poskytovatel „z prostředí“. HuggingFace: <code>HF_API_KEY</code> , <code>HF_TEXT_MODEL</code> (i s poskytovatelem, např. <code>...:novita</code> , <code>...:cheapest</code>), <code>HF_ASR_MODEL</code> , <code>HF_BASE_URL</code> (výchozí <code>https://router.huggingface.co</code>)

nginx

Konfigurace je v `deploy/nginx/m5cet.conf`. Pro 3.0 přibyl blok pro operátorské API (SSE potřebuje vypnutý buffering a dlouhý timeout):

```
location /api/admin/ {
    # allow 203.0.113.10; deny all; # doporučeno
    proxy_pass http://m5cet_app;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header Connection "";
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_buffering off;
    proxy_cache off;
    proxy_read_timeout 3600s;
}
```

Od 3.2 build ukládá ke každému souboru v `/assets/` i předkomprimované `.br` a `.gz`. nginx je posílá direktivou `gzip_static on;` (s modulem `ngx_brotli` i `brotli_static on;`); bez nginx je posílá sám server. Soubory s hashem v názvu mají `Cache-Control: public, max-age=31536000, immutable`, HTML, service worker a `build.json` dál `no-store`.

Upgrade z 3.2 na 3.3

Jen klient: `git pull`, `npm ci`, `npm run build`, restart `m5cet`. Protokol, šifrování ani data se nemění; otevřená okna nabídnou obnovení. Starší klient by server v pozvánce přehlédl a připojil by se sem — proto po nasazení obnovit okna.

Upgrade z 3.1 na 3.2

- 1. `git pull`, `npm ci`, `npm run build` (nebo `./update.sh`) — build nově předkomprimuje assets.
- 2. nginx (doporučeno): `gzip_static on;` v bloku `/assets/` (hotové v `deploy/nginx/m5cet.conf`); `nginx -t && systemctl reload nginx`.
- 3. Restart `m5cet` a `m5cet-admin`. Bez `client-config.json` platí výchozí hodnoty: uložená připojení zapnutá, všechny šablony povolené, výchozí Motorsport.
- 4. Protokol ani šifrování se nemění: **klienti 3.1 a 3.2 se v místnosti potkají**. Otevřená okna nabídnou obnovení (nový build). Kdo si šablonu už vybral, má ji dál.
- 5. V konzoli > *Client & addons* nastavit šablony a případné další signalizační servery.

Upgrade z 3.0 na 3.1

- 1. `git pull`, `npm ci` (nová závislost `hash-wasm`), `npm run build` (nebo `./update.sh`).

2. nginx: v hlavičce CSP `script-src 'self' 'wasm-unsafe-eval'` (hotová konfigurace v `deploy/nginx/m5cet.conf`); `nginx -t && systemctl reload nginx`. Bez toho Argon2id neběží a nikdo se nepřipojí.
3. Doporučeno: `TURN_SECRET` místo statických TURN přístupů, `BACKUP_DIR`, `METRICS_TOKEN`, `ALERT_WEBHOOK_URL`; jmenovití administrátoři místo sdíleného tokenu.
4. Restart `m5cet` a `m5cet-admin`. Migrace proběhnou samy: fronta dostane evidenci relayovaných zpráv a zapečetěná metadata (staré položky se dál přečtou), audit hash řetěz (staré záznamy „před řetězem“), účty nová pole.
5. **Všechna okna obnovit.** Klient 3.1 se připojuje pod slepým ID a jinými klíči — s klientem 3.0 se v místnosti nepotká. Zprávy čekající ve frontě z 3.0 se otevrou. Relace z 3.0 byly jen v paměti, takže se každý jednou znovu přihlásí passkey.

Upgrade z 2.x na 3.0

1. `git pull`, `npm ci`, `npm run build` (nebo `./update.sh`).
2. Do nginx přidat blok `/api/admin/`; `nginx -t && systemctl reload nginx`.
3. Běží-li hlavní služba jinde než na `127.0.0.1:$PORT`, nastavit admin službě `MAIN_URL`.
4. Restart `m5cet` a `m5cet-admin`. Při startu se staré schránky převedou do fronty (`queue.migrated` v auditu) a vznikne `signaling.secret`.
5. Otevřená okna nabídnou obnovení (nový build). Klient 2.x se s 3.0 nespojí — všichni obnoví.
6. Dřívější admin stránka ukládala token do `localStorage`; nová konzole ho při prvním otevření smaže. Zvažte výměnu `ADMIN_API_TOKEN`.

Ověření

```
npm run check          # TypeScript
npm test -- --no-file-parallelism # 1107 unit testů (vč. fuzz, upgrade z 3.0, cluster s Redisem, připojení, konfigurace klienta, všechna roz
npm run build
npm run test:e2e        # 67 testů: zprávy, soubory P2P i relay, šifrovaný hovor, účty a obnova, uložená připojení, okno Místnost, sdílení př
npm run load-test -- --url ws://127.0.0.1:5000/ws --clients 500 # zátěž (zvednout WS_* limity na testovacím serveru)
curl -s https://chat.example.com/api/health # {"ok":true,"protocol":2,"version":"5.3.0","cluster":"local",...}
```

Bezpečnostní opatření serveru

HTTP

Helmet, CSP v produkci `script-src 'self' 'wasm-unsafe-eval'` (jen WebAssembly pro Argon2id, žádný `eval`) a `object-src 'none'`, workery jen z vlastního originu; `Cache-Control: no-store` všude; limity **před** parsery těl (veřejné API 100 / 15 min, trezor 300, úložiště 1 200, admin 30 odmítnutí / 15 min); JSON do 256 KB (trezor 8 MB, úložiště 12 MB); chyby 500 bez interních detailů.

Logy

Řádek požadavku = metoda, cesta, stav, doba, velikost. **Nikdy těla** — odpovědi nesou session tokeny a šifrované texty (2.x je logovala). Surové tělo se drží jen pro webhooky poskytovatelů kvůli podpisům.

WebSocket

Origin, brána spojení, maxPayload 256 KB, validace rámců, limity tříd, vyhazování opakovaných narušitelů, heartbeat, zpětný tlak, vlastnictví přenosů, rozesílání jen ověřených polí.

Identita a soukromí

Pseudonymy účtů v místnosti, obecná odmítnutí relay, IP v monitoru zkrácené na /24 (/48), místnosti jako hash, audit komunikace jen na vyžádání, zapečetěné detaily v databázi.

SSRF

Server POSTuje jen na push endpointy známých služeb (https, bez portu a přihlašovacích údajů); příkaz ke stažení souboru přijme jen https adresu a klient se uživatele zeptá.

Ukončení

SIGTERM/SIGINT: sockety se zavřou (klienti se vrátí přes resume), účty se zapíší, databáze se zavřou a klíče vynulují, pak konec (nejpozději za 8 s).

Návrhy a roadmapa

Návrhy z dokumentace 3.0 a jak dopadly ve 3.1; na konci to, co zbývá. Priorita **P1** = nejvyšší přínos pro bezpečnost či spolehlivost, náročnost S / M / L.

Stav návrhů z 3.0

OBLAST	NÁVRH	STAV	KDE
Kryptografie	Forward secrecy a odebrání člena	hotovo 3.1	klíče odesílatele s ratchetem, rotace, <i>Vyloučit z mých zpráv</i> — Šifrování v3. MLS zůstává v plánu pro velké skupiny.
	Slepá ID místností	hotovo 3.1	HKDF místo HMAC názvu, r3....
	Párové šifrování soukromých zpráv	hotovo 3.1	ECDH P-256 (WebCrypto nemá X25519 všude), podepsaný hello
	Identita svázaná s účtem	hotovo 3.1	Ed25519 z kořene účtu, certifikát zařízení, pin podle účtu
	Argon2id	hotovo 3.1	64 MiB / 3 průchody ve Web Workeru, CSP <code>'wasm-unsafe-eval'</code>
	Ověření QR kódem	hotovo 3.1	zobrazení i skenování (<code>BarcodeDetector</code>), potvrzení
Protokol	Více instancí	hotovo 3.1	Redis pub/sub, podepsané zprávy — Více instancí; fronta dál v SQLite (sdílený disk)
	Binární rámce	hotovo 3.1	vlastní binární formát kusů (jednodušší než CBOR) po DataChannelu i socketu; WebTransport zatím ne
	SFU s E2EE	částečně	rámce hovoru jsou šifrované end-to-end a připravené pro SFU; SFU samotné součástí není
	TURN s krátkodobými přístupy	hotovo 3.1	<code>TURN_SECRET</code> , obnova před vypršením
Fronta	Trvalá evidence účtenek	hotovo 3.1	tabulka <code>relay_ledger</code>
	Neutrální text probuzení	hotovo 3.1	push bez jména a místnosti
	Metadata fronty	hotovo 3.1	odesílatel a stav zapečetěné master klíčem, odesílatel jen jako hash (v globální DB — do DB uživatele se zapisovat nedá, když je zamčená)
Účty	Více passkeys a obnovovací klíč	hotovo 3.1	zapečetěný kořen pro každý passkey i kód
	Relace přežívající restart, seznam zařízení	hotovo 3.1	hashe v <code>sessions.json</code> , klouzavá platnost
	Zálohy a integrita	hotovo 3.1	online backup, manifest, quick_check, VACUUM
Administrace	Role, jmenné přístupy, přihlášení passkey	hotovo 3.1	vlastník / operátor / auditor; OIDC zatím ne
	Neměnný audit	hotovo 3.1	hash řetěz + podepsané body
	Prometheus, OpenTelemetry, alerty	částečně	<code>/metrics</code> a alerty s webhookem ano; trasování OpenTelemetry ne
Klient	Rozdělit <code>App.tsx</code> , error boundary	částečně	panely, ovládání hovorů, dialog a pomocné funkce v modulech, error boundary; hooky spojení a peerů zbývají
	Kryptografie ve Web Workeru	hotovo 3.1	KDF ve workeru, rámce hovoru ve workeru
	Rozdělení bundlu, dlouhé seznamy	hotovo 3.1	hlavní chunk 685 → 440 kB, React zvlášť; okno 200 zpráv + <code>content-visibility</code>
	Dokončit i18n	hotovo 3.1	hlášky chatu cs/en/de + test úplnosti

OBLAST	NÁVRH	STAV	KDE
Testy	Fuzzing a property testy	hotovo 3.1	vlastní seedovaný generátor (bez závislosti); našel chybu v RESP parseru
	Zátěžové testy	hotovo 3.1	<code>npm run load-test</code> (1 000 klientů lokálně: RTT p99 3 ms)
	E2E konzole v CI, test upgradu	hotovo 3.1	konzole s rolemi, obnova účtu, relay souborů, hovor; upgrade dat z 3.0

Co dál

NÁVRH	PROČ	PRIORITA	NÁROČNOST
Relay a fronta pod klíči odesílatelů — předem rozeslané řetězy nebo prekeys (X3DH) pro nepřítomné	Zprávy pro nepřítomné a fronta odchozích jsou dnes pod klíčem místnosti: forward secrecy pro ně neplatí.	P1	L
Účty v SQLite místo JSON souborů	Bezpečné souběžné zápisy více instancí bez spoléhání na sticky sessions; transakce.	P2	M
SFU pro skupinové hovory (mediasoup / LiveKit) s již šifrovanými rámci	Mesh se nad 4–5 účastníků neškáluje; rámce jsou připravené.	P2	L
Hooky spojení a WebRTC peerů z <code>App.tsx</code> + jejich unit testy; stavový automat spojení	Zbytek logiky chatu je testovaný jen end-to-end.	P2	M
MLS (RFC 9420) pro velké místnosti	Párová distribuce klíčů odesílatelů roste s n^2 ; MLS s $n \log n$.	P3	L
Seznam zařízení účtu viditelný protistranám (podepsaný účtem, transparentní změny)	Upozornit, když účet získá nové zařízení, ne až když napíše.	P3	M
OIDC pro konzoli, OpenTelemetry trasování	Napojení na firemní identitu a tracing.	P3	M
Fronta v Postgresu , WebTransport	Cluster přes více hostitelů bez sdíleného disku; nižší latence relay.	P3	L
Import / export uložených připojení zapečetěným souborem; <i>uložit jako připojení</i> po přijetí pozvánky	Sdílení pozvánkou je hotové (3.3); přenos celé sady mezi účty zatím ne.	P3	S
Vlastní šablona od správce (barvy, písmo, zaoblení) v konzoli	Firemní vzhled bez úprav CSS; dnes jen výběr z 13 šablon.	P3	M
Post-kvantový hybrid (ML-KEM + ECDH) pro párové klíče	„Sklidit teď, rozšifrovat později“; až to WebCrypto nabídne nativně.	P3	M

Změny v 5.3

Přidáno

Vstupní body modelu — execute, response, button, form, error a víc webhooků s vlastními URL — se vstupy u každého.

`m5.model` (calls, current, last, first, session, cache). **Výsledek jako seznam** výstupů, každý vykreslený samostatně; nové výstupy audio, video, flash, window, **button, form, js**; `m5.browser`. Odpověď na zprávu modelu. Konzole: editor vstupních bodů, **Form builder**, Button, Browser JavaScript, živá tlačítka, formuláře a Reply ve zkušebních bězích, sezení běhu. Vizuální tvůrce s víc funkcemi a novými uzly. `/help` a ukázky 1.1.0, tutoriál 10–16.

Změněno

Model ukládá `endpoints` (`entry`, `inputs` a `executors.webhook`) jsou jejich zrcadlo — starší modely beze změny). Běh ukládá sezení a volání. Text zprávy v rozvržení je `<div>`. CSP aplikace: `frame-src 'self'`. Nová API `/api/functions/event`, `/fn-sandbox.html` a konzolová `/admin/functions/event|exports|chains`.

Opraveno

Výstupy příkazů se v chatu ukazovaly jen jako Markdown (obrázky, soubory, notifikace a okna chyběly). Neplatný výstup z `m5.caller.send` tiše mizel. Data s polem `type` se brala jako výstup. Konzole neukázala odpověď vstupního bodu error.

Bezpečnost

Kód z funkcí běží v prohlížeči jen v izolovaném rámu (neprůhledný origin, vlastní CSP) s omezenými zprávami. Tlačítka, formuláře a odpovědi mají stejnou kontrolu přístupu jako příkaz; ID sezení má 96 náhodných bitů; výstupy od peerů se kontrolují znovu (typy, velikosti, třídy a barvy, pole). Hlášení z prohlížeče mají limit. Protokol, šifrování ani data účtů se nemění.

Podrobný seznam: `CHANGELOG.md`. Architektura: `docs/functions-architecture.md`. Tato dokumentace odpovídá kódu verze 5.3.0.

Změny v 5.2

Přidáno

Nástroje jako moduly (Functions, AI & speech, Telephony & SIP, Layout a Menu builder): výchozí přístup, přístupové skupiny allow/deny, hlavní skupina `mod-<modul>` (založí se sama), **granty** na části modulu se zástupnými znaky a odebráním, test přístupu. **Log přístupů** (povolení i odmítnutí) s kartou v konzoli. **Webhooky**: plný log s proměnnými, replay i na konceptu, ladění v editoru, režimy sync/async/auto, stav a odpovědi běhu, zpětné volání. **Aktivační znaky** `/`, `@`, `#` (a vlastní). Vestavěné `/help`, `/whois`, `/dns`, `/web`, `/mail`, `/domain` a knihovna `netkit`; SDK `m5.functions`.

Změněno

Přepínače služeb AI, Speech a Functions v *Modules & groups*. Nový tvar pravidel modulů (staré se načtou beze změny chování). Seznam příkazů v chatu se obnovuje; příkaz „do místnosti“ bez příjemců se spustí lokálně. Běhy funkcí a log webhooků se uklízejí (`FUNCTIONS_RUNS_DAYS`). Nové proměnné `ACCESS_LOG_DAYS`, `ACCESS_LOG`, `WEBHOOK_AUTO_WAIT_MS`, `FUNCTIONS_BUILTINS`.

Opraveno

`/` **v chatu nespouštěl příkazy** — služba Functions byla vypnutá bez přepínače v konzoli a seznam příkazů se neobnovoval. Webhooky dostávaly prázdné tělo (spotřebovaly ho globální parsery), takže nešel ověřit ani HMAC. Úklid starých běhů se nikdy nespouštěl.

Bezpečnost

Token webhooku se porovnává v konstantním čase a v logu je maskovaný; `Authorization`, `Cookie` a podpisy se maskují a funkci se nepředávají. Zpětné volání prochází ochranou SSRF. Nástroje konzole hlídá modul i role; vlastník se nezamkne. Protokol, šifrování ani data účtů se nemění.

Podrobný seznam: `CHANGELOG.md`.

Změny v 5.1

Přidáno

Editor CodeMirror 6 v konzoli Functions (zvýraznění, našeptávač SDK s šablonami a `await`, šablony, nápověda při najetí i psaní, kontrola syntaxe a chybějícího `await`, hledání, formátování). **Vizuální tvůrce** (uzly a dráty → JS/Python, hodnoty na plátně, uložení do balíčku, model jedním klikem, příklady). **Živé běhy** s odpovídáním na `prompt` / `form`. **Řeč zdarma a offline** (Whisper + Piper přes sherpa-onnx) a předvolby Groq, Speeches, Kokoro, whisper.cpp.

Změněno

Přehled konzole Functions (statistiky za 24 h), dialogy místo `prompt()`, typovaný testovací formulář modelu, filtr a automatické obnovování běhů. Nahrávky k přepisu jdou jako 16kHz WAV. Funkce spuštěná z konzole je pro AI i přihlášený uživatel. Instalátor a obraz Dockeru mají `bzip2`. Nové endpointy konzole: `/admin/functions/runs/:id/live`, `/runs/:id/answer`, `/flow/compile`, `/packages/:id/flow`, `/admin/ai/local`.

Opraveno

Záložka *Runs* ukazovala „[object Promise]“. Markdown výstupů: nadpis či seznam na prvním řádku bloku.

Bezpečnost

Editor se nenačítá z CDN (CSP konzole zůstává `script-src 'self'`). Offline řeč běží na serveru a nic neposílá ven; modely se stahují jen na pokyn vlastníka z vydání sherpa-onnx na GitHubu. Protokol, šifrování ani data účtů se nemění.

Podrobný seznam: `CHANGELOG.md`.

Změny v 5.0

Přidáno

M5cet Functions (`server/functions/`): sandbox (QuickJS + Pyodide v odděleném procesu s limity), SDK `m5` stejné v JS i Pythonu (výstupy, log, session, cache, `http` se SSRF ochranou, `dns`, `crypto` vč. JWT/PGP/SSH/X.509, `codes`, `ai` a agenti, `webhook`, `prompt / form`), balíčky a neměnné verze, modely se schématem vstupů; konzole s IDE, zkušebním během, tutorialem, plány a běhy. Chat: `/klíč` s našeptávačem a bohatým Markdown výstupem, živé `prompt / form`. Executory: webhook (příchozí i běhu), plán (cron), API token; trvalé `on_event`. Šablony a export/import `.m5pkg`.

Změněno

Nový přepínač modulu `functions` (`ENABLE_FUNCTIONS`). `npm run build` staví i `dist/sandbox.cjs` a kopíruje běhové balíčky do `dist/node_modules` (ovladač SQLCipher, Pyodide, QuickJS WASM). Nová API aplikace `/api/functions/*` a `/hooks/*`.

Opraveno

Ovladač SQLCipher se teď dostane do `dist/node_modules`, takže úložiště a žurnál AI přežijí i výchozí instalaci a obraz Dockeru (dřív běžely jen v paměti, když se `node_modules` po sestavení mazal).

Bezpečnost

Model hrozby: skripty píše důvěryhodný operátor; hranicí je oddělený proces a interpret ve WASM, ne obrana proti autorovi. `m5.http` má SSRF ochranu (bez privátních a metadatových adres, připnutí IP). Server nechte místnost; výstup do ní šířuje klient. Protokol, šifrování ani data účtů se nemění.

Podrobný seznam: `CHANGELOG.md`. Architektura: `docs/functions-architecture.md`. Tato dokumentace odpovídá kódu verze 5.0.0.

Změny ve 4.14

Přidáno

Vrstva AI a řeči (`server/ai/`): poskytovatelé Anthropic, OpenAI, Open WebUI, Perplexity, Ollama, llama.cpp, GPT4All, Hugging Face, jiný OpenAI-kompatibilní a ElevenLabs; klíče zašifrované na serveru; modely načtené od poskytovatele s tím, co umí, a s cenami; skupiny, které poskytovatele smí použít; limity (měsíc, uživatel a den, velikosti); žurnál všech volání s tokeny, cenou, dobou a obsahem jen při dočasném ladění. Konzole > AI & speech: poskytovatelé, modely, zkušebna se streamem a požadavkem, test řeči, volání (živě, filtry, CSV, souhrny), výchozí modely a pokyny asistenta, limity, žurnál. Aplikace: asistent se streamovanými odpověďmi v Markdownu, uvažováním, zdroji a zastavením (rozvržení `panel.ai`).

Změněno

`/api/ai/status` a `/api/speech/status` podle přihlášeného účtu a jeho skupin; `/api/ai/complete` zůstává. Manifest `/api/modules` hlásí AI a řeč podle přepínače v konzoli a nastavených modelů (dřív jen podle `ENABLE_AI=1`). Konektory z `server/plugins/` nahradily adaptéry v `server/ai/providers/`.

Kompatibilita

Po aktualizaci je asistent vypnutý, dokud vlastník nenastaví měsíční limit; hosté potřebují skupinu *Guests* u poskytovatele. Klíče z prostředí dál fungují. Protokol, šifrování ani data účtů se nemění.

Bezpečnost

Klíče jen na serveru, zašifrované a svázané s poskytovatelem, nikdy zpět do konzole; adresy poskytovatelů jen http(s) bez přihlašovacích údajů a mění je jen vlastník; obsah konverzací se neukládá (kromě dočasného ladění, které vidí jen vlastník); odpovědi se kreslí bez HTML; export CSV odolný vůči vzorcům.

Podrobný seznam: `CHANGELOG.md`. Tato dokumentace odpovídá kódu verze 4.14.0.

Změny ve 4.13

Přidáno

Okno Místnost, okna, dialogy a panely jako rozvržení Layout builderu (36 nových, celkem 44, v sekcích), náhled každého skutečnou komponentou v jejích situacích. Varianty rozvržení pro skupiny uživatelů a šablony vzhledu. Historie uložených verzí s rozdíly a návratem. Archiv vydaných výchozích stromů a třicestné sloučení vlastního rozvržení s novým výchozím (samo, nebo s řešením konfliktů). Vložení HTML jako prvků. Kontrola přístupnosti návrhu i vykresleného náhledu (názvy, popisky, klávesnice, nadpisy, kontrast WCAG). Prvky tabulka a video.

Změněno

Šablony a rozvržení se překládají na funkce (rychlejší vykreslení, neměnné části se nevytvářejí znovu). Filtry nad výrazem v závorce. Komponenty oken, okna Místnost, dialogů a panelů kreslí rozvržení (DOM a akce ověřeně stejné). Výchozí stromy se staví až při prvním použití. Katalog ikon 239 a skript ikon čte stromy rozvržení.

Přístupnost

Přístupný název dostalo šest polí: klíč místnosti v úpravě připojení, text SMS, odkaz pozvánky, výběr serverového hlasu, název passkey a obnovovací kód — a ve zprávách pole kódu zapečetěné zprávy.

Kompatibilita

`layout.json` dostal `variants`; starší soubor platí beze změny (vlastní rozvržení z 4.0.5 se sloučí s novými výchozími). Historie vzniká vedle něj (`layout-history.json`). Protokol, šifrování ani data účtů se nemění; klienti 4.0 a 4.13 se v místnosti potkají.

Podrobný seznam: `CHANGELOG.md` .

Změny ve 4.0.6

Opraveno

Anthropic: konektor posílal vždy `temperature: 0.7`, což modely vydané po Claude Opus 4.6 (např. `claude-sonnet-5`) odmítají (chyba 400). Parametry vzorkování se posílají jen na vyžádání a odmítnutý parametr se zkusí znovu bez něj; z odpovědi se berou jen textové bloky. HuggingFace: text i přepis řeči volaly zaniklý `api-inference.huggingface.co` („fetch failed“) — dnes `router.huggingface.co/v1/chat/completions` (OpenAI kompatibilní) a `router.huggingface.co/hf-inference/models/<model>`. OpenAI: `max_completion_tokens` u `api.openai.com`.

Přidáno

Konzole › AI & speech: přepínače modulů AI a Speech (uloženo v `plugins.json`), sdíleno aplikací i admin službou; proměnné prostředí mají přednost, panel se načte hned po otevření, test ukazuje dobu odezvy. Chyby spojení říkají proč (DNS, odmítnuté spojení, časový limit, certifikát) a u poskytovatele jeho vlastní zprávu.

Podrobný seznam: `CHANGELOG.md`.

Změny ve 4.0.5

Přidáno

Layout builder jako GUI designer: rozvržení lišty, okna chatu, zpráv (3), psaní a widgetu příjemců (2) jako stromy prvků; paleta 28 prvků, strom s přetahováním, vlastnosti s našeptáváním (třídy z buildu, atributy, CSS, proměnné, akce), podmínky, opakování, události, šablony prvků, kopírování mezi rozvrženími, náhled samotné aplikace (`layout-preview.html`). Šablonovací jazyk: podmínka `? :`, filtr `t`.

Změněno

MessageBubble, RecipientsWidget, lišta, okno chatu a pole pro psaní se kreslí z rozvržení (DOM beze změny, ověřeno). Menu builder a Layout builder sdílejí prvky formulářů, výběr ikon a barev, editor stylů a nápovědu (`builder-kit.js`). Katalog ikon 207 (přejmenované ikony lucide se hledají i pod starým jménem).

Kompatibilita

`layout.json` dostal `layouts` a `blocks`; dřívější barvy, texty a chování platí dál. Protokol, šifrování ani data účtů se nemění.

Bezpečnost

Stromy validuje server i klient, vykreslení bez `innerHTML`, adresy a CSS se kontrolují i při vykreslení. Náhled smí zarámovat jen konzole (stejný původ).

Podrobný seznam: `CHANGELOG.md`.

Změny ve 4.0.0

Přidáno

Jedinečné uživatelské jméno účtu (primární klíč, uložené v passkey), přezdívka jako alias, P2P jméno relace z přezdívky. Důkaz globálního klíče a zamčená relace do jeho ověření; přihlášení krok za krokem s logem v auditu; `/signin` a `/signup`. Ochrana navigace během spojení. Kontrola verzí aplikace, knihoven, souborů a service workeru proti serveru s opravou. Moduly a skupiny v konzoli, vynucené i serverem. Menu builder se šablonovacím jazykem, styly a stavy.

Změněno (nekompatibilně)

Server-enhanced vyžaduje přihlášení passkey — nepřihlášený uživatel už nedostane anonymní relaci úložiště ani ruční připojení na server; volby jsou neaktivní a vedou do okna Spojení. Vytvoření passkey a přihlášení jsou jen v okně Spojení. Registrace chce důkaz klíče (starší klient se nezaregistruje) a přihlášení vydá zamčený token, který odemkne až `/api/account/unlock`.

Stávající účty

Fungují dál: jejich ID je jejich uživatelské jméno (nepřejmenovávají se), hash důkazu klíče se uloží při prvním přihlášení 4.0. Menu vypadá stejně, dokud ho správce v builderu nezmění.

Kompatibilita

Signalizační protokol a šifrování místností beze změny; hello nově nese uživatelské jméno (starší klienti ho ignorují). Nová data: `menu-config.json`, `modules` a `groups` v konfiguraci klienta, `username` a `keyVerifier` u účtu.

Podrobný seznam: `CHANGELOG.md`.

Změny ve 3.3.0

Přidáno

Okno Místnost se záložkami typu připojení v záhlaví, výběrem uložených připojení (bez tlačítek, vybraný orámovaný), položkou *Jiná místnost*, ozubeným kolem k *Moje připojení* nad oknem a vždy viditelnými tlačítky a sdílením. Tlačítko *Sdílet* u uloženého připojení a okno *Sdílet připojení* (bez nutnosti spojení, jméno pro pozvaného, jiný server v pozvánce).

Změněno

Během spojení nelze přepnout typ, připojení ani pole — nejdřív Odpojit. Okna se vykreslují do `<body>` a skládají se nad sebe (Escape zavře horní). Pole klíče má oko pro zobrazení.

Opraveno

Obnovení stránky vracelo spojení uloženého připojení s jiným serverem na tento server; Reconnect takového připojení také. Pozvánka přijatá po připojení k jinému serveru zůstala na něm. Uložené připojení s režimem light po připojení skrylo seznam připojení.

Kompatibilita

Protokol, šifrování i data beze změny.
Relace si navíc pamatuje server a uložené připojení; pozvánka může nést `server`.

Podrobný seznam: `CHANGELOG.md`.

Změny ve 3.2.0

Přidáno

Uložená připojení přihlášených uživatelů (místa, klíč, jméno, server, režim, historie, TTL, relay, upozornění, znovupřipojení, udržování spojení) se statistikami, logem a exportem; výchozí připojení, připojení po přihlášení a po návratu, přepínač v záhlaví; jiný obslužný server. Šablony iOS 27 a Windows 11 (světlý i tmavý tón), Aurora, Nord, Sakura, Ocean, Graphite; tón podle systému; pět stylů ikon. Konzole > Client & addons, `/api/client-config`.

Optimalizováno

Předkomprimované assety (brotli: hlavní JS 484 → 122 kB) s ročním cache; zprávy chatu se nepřekreslují každou sekundu (časovač jen k nejbližší expiraci) a řádky zpráv jsou memoizované; odvozené klíče v LRU cache; konfigurace layoutu se čte jen při změně souboru.

Bezpečnost

Na jiný obslužný server nejde token účtu, úložiště, relay ani administrátorské příkazy; adresy serverů jen `wss://` bez přihlašovacích údajů; trezor s připojeními se nepřepíše, dokud se nenačte; export logu bez klíče.

Kompatibilita

Protokol a šifrování beze změny — klienti 3.1 a 3.2 se potkají. Trezor účtu má třetí část `connections`; starší klient ji ignoruje.

Podrobný seznam: `CHANGELOG.md`.

Změny ve 3.1.0

Přidáno

Šifrování v3 (Argon2id ve workeru, slepá ID místností, klíče odesílatelů s ratchetem, párové klíče, identita účtu, QR bezpečnostní čísla), binární přenos kusů, relay souborů při selhání P2P s řízením rychlosti, E2EE rámců hovoru, krátkodobé TURN přístupy, cluster přes Redis, více passkeys a obnovovací kód, trvalé relace a seznam zařízení, role a přihlášení passkey do konzole, neměnný audit, zálohy, `/metrics` a alerty, error boundary, dělení bundlu, kompletní i18n, fuzz / zátěžové / upgrade testy.

Opraveno

Worker „background tick“ z `blob:` URL blokový produkční CSP (skrytá záložka neudržovala spojení); zvuk toho, kdo se k hovoru připojil druhý a nebyl iniciátorem, se nikdy nevyjednal; soubor bez P2P spojení nešel poslat vůbec; dva E2E testy sdílely port.

Změněno

Živé zprávy už nešifruje klíč místnosti; server zná jen slepé ID; metadata fronty zapečetěná; push probuzení neutrální; relace 12 h klouzavě (max. 7 dní) místo pevných 12 h v paměti; CSP povoluje WebAssembly.

Nekompatibilní

Klient 3.0 a 3.1 se v místnosti nepotkají (jiné ID i klíče) — po nasazení obnovit všechna okna. nginx potřebuje `'wasm-unsafe-eval'` v CSP.

Podrobný seznam: `CHANGELOG.md`.

Změny ve 3.0.0

Přidáno

Protokol v2 (validace, limity, brána, Origin, resume, pseudonymy, `auth`, odvolání, heartbeat, zpětný tlak), šifrování v2 (klíče podle účelu, AAD, zapečetěná signalizace, podpisy, TOFU, ověřené soubory), SQLite fronta s lease a dead-letter, nová konzole s živým provozem, auditem, uživateli, frontou, úložištěm a systémem, dokumentace HTML + PDF.

Opraveno

XSS přes MIME souboru, DoS přes metadata souboru, převzetí peer ID, podvržené účtenky a položky schránky, logování session tokenů, SSRF přes push endpoint, token admina v localStorage, příkazy z admin služby, které nikdy nedorazily, vlastní zprávy po obnovení označené jako cizí, 18 oprav úložiště.

Změněno

Veřejný status úložiště bez statistik ostatních, metadata endpointy jen s tokenem, kód zapečetěné zprávy 12 znaků, transfer log bez jmen souborů, historie serveru bez passkey šifrovaná v prohlížeči.

Nekompatibilní

Klienti 2.x se s 3.0 nespojí (nezapečetěná signalizace) a nepřečtou obálky v2. Po nasazení obnovit všechna okna; přidat blok nginx `/api/admin/`.

Podrobný seznam: `CHANGELOG.md`.